# Supplementary Material: Knowledge-Aided Space-Time Adaptive Processing for Functional Ultrasound

Arthur Shune*
Independent Researcher

**Supplementary material.** This document contains appendices for the main paper (derivations, diagnostic panels, and reproduction commands).

## References

[1] Imbault, M., et al. Intraoperative functional ultrasound imaging of human brain activity. *Scientific Reports* 7, 7304 (2017).

[2] Macé, E., Montaldo, G., Pernot, M., Cohen, I., and Tanter, M. Functional ultrasound imaging of the brain. *NeuroImage* 54(4), 2720–2731 (2011).

[3] Urban, A., et al. In vivo whole brain microvascular imaging in mice using ultrafast ultrasound. *EBioMedicine* 76, 103861 (2022).

[4] Alexandrov, A. V., et al. Velocity measurements in the middle cerebral arteries of healthy adults by transcranial Doppler. *Journal of Neuroimaging* 1, 166–172 (1991).

[5] Alexandrov, A. V., et al. Optimal values of flow velocity on transcranial Doppler in patients with intracranial arterial stenosis. *Stroke* 33, 1757–1761 (2002).

[6] Macé, E., et al. Functional ultrasound imaging of the brain: theory and basic principles. *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* 60(3), 492–506 (2013).

[7] Rabut, C., et al. Functional ultrasound imaging of human brain activity through the skull. *Science Translational Medicine* 12(572), eaav4392 (2020).

[8] Rabut, C., Norman, S. L., Griggs, W. S., Russin, J. J., Jann, K., et al. Functional ultrasound imaging of human brain activity through an acoustically transparent cranial window. *Science Translational Medicine* 16, eadj3143 (2024).

[9] Deffieux, T., Demené, C., and Tanter, M. Functional ultrasound imaging: A new imaging modality for neuroscience. *Neuroscience* 474, 110–121 (2021).

*Work performed in personal capacity. Correspondence: `arthur.shune@gmail.com` (alt: `arthur@skymesasystems.com`). ORCID: `0009-0004-6433-3856`. Current employment: BlackSky, Inc. Founder: SkyMesa Systems; Valence Intelligence Systems. No external funding.

[10] Demeulenaere, O., Bertolo, A., Pezet, S., Osmanski, B.-F., Tanter, M., and Deffieux, T. In vivo whole brain microvascular imaging in mice using transcranial 3D ultrasound localization microscopy. *EBioMedicine* 79, 103995 (2022).

[11] Demené, C., et al. Spatiotemporal clutter filtering of ultrafast ultrasound data highly increases Doppler and fUltrasound sensitivity. *IEEE Transactions on Medical Imaging* (2015). DOI: `https://doi.org/10.1109/TMI.2015.2428634`.

[12] Baranger, J., et al. Adaptive spatiotemporal SVD clutter filtering for ultrafast Doppler imaging using similarity of spatial singular vectors. *IEEE Transactions on Medical Imaging* (2018). DOI: `https://doi.org/10.1109/TMI.2018.2789499`.

[13] Yu, A. C. H., and Løvstakken, L. Eigen-based clutter filter design for ultrasound color flow imaging: a review. *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* (2010). DOI: `https://doi.org/10.1109/TUFFC.2010.1521`.

[14] Song, P., et al. Ultrasound small vessel imaging with block-wise adaptive local clutter filtering. *IEEE Transactions on Medical Imaging* (2017). DOI: `https://doi.org/10.1109/TMI.2016.2605819`.

[15] Kasai, C., Namekawa, K., Koyano, A., and Omoto, R. Real-time two-dimensional blood flow imaging using an autocorrelation technique. *IEEE Transactions on Sonics and Ultrasonics* 32(3), 458–464 (1985). DOI: `https://doi.org/10.1109/T-SU.1985.31615`.

[16] Chee, A. J. Y., Yiu, B. Y. S., and Yu, A. C. H. A GPU-parallelized eigen-based clutter filter framework for ultrasound color flow imaging. *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* (2017). DOI: `https://doi.org/10.1109/TUFFC.2016.2606598`.

[17] Lok, U.-W., Tang, J., Kilic, K., Szabo, T. L., and Boas, D. A. Real time SVD-based clutter filtering using randomized SVD and spatial downsampling for microvessel imaging. *Ultrasonics* 107, 106163 (2020). DOI: `https://doi.org/10.1016/j.ultras.2020.106163`.

[18] Demene, C., et al. Improved color Doppler for cerebral blood flow axial velocity estimation (iCD-fUS). *IEEE Transactions on Medical Imaging* 40(5), 1498–1510 (2021).

[19] Tang, J., Kilic, K., Szabo, T. L., and Boas, D. A. Improved color Doppler for cerebral blood flow axial velocity imaging. *IEEE Transactions on Medical Imaging* 40(2), 758–764 (2021).

[20] Urban, A., et al. Mobile human brain imaging using functional ultrasound. *Nature Biomedical Engineering* 8, 534–546 (2024).

[21] Aaslid, R., Markwalder, T.-M., and Nornes, H. Noninvasive transcranial Doppler ultrasound recording of flow velocity in basal cerebral arteries. *Journal of Neurosurgery* 57(6), 769–774 (1982).

[22] Gao, S., Lam, W. W.-K., Chan, Y.-H., Liu, J., Wong, K.-P., Leung, T. W.-S., et al. Optimal values of flow velocity on transcranial Doppler in grading middle cerebral artery stenosis in comparison with magnetic resonance angiography. *Journal of Neuroimaging* 12(3), 213–218 (2002).

[23] Estrada, H., Gottschalk, S., Reiss, M., Neuschmelting, V., Razansky, D., et al. Observation of guided acoustic waves in a human skull. *Ultrasound in Medicine & Biology* 44(7), 1438–1447 (2018).

[24] Candès, E. J., Li, X., Ma, Y., and Wright, J. Robust principal component analysis? *Journal of the ACM* 58(3), 11 (2011).

[25] Solomon, O., et al. Deep unfolded robust PCA with application to clutter suppression in ultrasound. *IEEE Transactions on Medical Imaging* 39(4), 1051–1063 (2020). DOI: `https://doi.org/10.1109/TMI.2019.2941271`.

[26] Lin, Z., Chen, M., and Ma, Y. The augmented Lagrange multiplier method for exact recovery of corrupted low-rank matrices. *arXiv preprint* arXiv:1009.5055 (2010).

[27] Halko, N., Martinsson, P. G., and Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53(2), 217–288 (2011).

[28] Functional ultrasound imaging of the brain. UNC/NCSU Joint Department of Biomedical Engineering teaching material, online resource, accessed 2024. Available at `https://bme.unc.edu/wp-content/uploads/sites/917/2021/08/functional_ultrasound.pdf`.

[29] Soloukey, S., Vincent, A. J. P. E., Smits, M., Slump, C. H., Dirven, C. M. F., et al. Functional ultrasound (fUS) during awake brain surgery: the clinical potential for intra-operative functional and vascular brain mapping. *Frontiers in Neuroscience* 13, 1384 (2019). Available at `https://www.frontiersin.org/articles/10.3389/fnins.2019.01384`.

[30] Rapid full-wave phase aberration correction method for transcranial ultrasound. Online resource, accessed 2024. Available at `https://jtultrasound.biomedcentral.com/articles/10.1186/s40349-016-0074-7`.

[31] Ultrafast Ultrasound Imaging. Online resource, accessed 2024. Available at `https://mdpi-res.com/bookfiles/book/756/Ultrafast_Ultrasound_Imaging.pdf`.

[32] Staggered multiple-PRF ultrafast color Doppler. Online resource, accessed 2024. Available at `https://www.biomecardio.com/publis/ieeetmi16.pdf`.

[33] Functional ultrasound imaging: a useful tool for neuroscience. Online resource, accessed 2024. Available at `https://www.sciencedirect.com/science/article/pii/S1053811921009940`.

[34] Shin, Y., et al. Context-Aware Deep Learning Enables High-Efficacy Localization of High Concentration Microbubbles for Super-Resolution Ultrasound Localization Microscopy. Zenodo dataset, DOI 10.5281/zenodo.10711806 (2024).

[35] Shin, Y., et al. Context-Aware Deep Learning Enables High-Efficacy Localization of High Concentration Microbubbles for Super-Resolution Ultrasound Localization Microscopy. *Nature Communications* (2024). Available at `https://www.nature.com/articles/s41467-024-47154-2`.

[36] Chavignon, A., Heiles, B., Hingot, V., Lopez, P., Teston, E., and Couture, O. In vivo rat brain for Ultrasound Localization Microscopy: raw and beamformed data. Zenodo dataset, DOI 10.5281/zenodo.7883227 (2023).

[37] Heiles, B., Chavignon, A., Hingot, V., Lopez, P., Teston, E., and Couture, O. Performance benchmarking of microbubble-localization algorithms for ultrasound localization microscopy. *Nature Biomedical Engineering* (2021). DOI: `https://doi.org/10.1038/s41551-021-00824-8`.

[38] Errico, C., et al. Ultrafast ultrasound localization microscopy for deep super-resolution vascular imaging. *Nature* 527(7579), 499–502 (2015). DOI: `https://doi.org/10.1038/nature16066`.

[39] Hingot, V., et al. Subwavelength motion-correction for ultrafast ultrasound localization microscopy. *Ultrasonics* 77, 17–21 (2017). DOI: `https://doi.org/10.1016/j.ultras.2017.01.008`.

[40] Leonov, D. V., Reshetnikov, R. V., Kulberg, N. S., Nasibullina, A. A., and Gromov, A. I. Doppler Twinkling Artifact Observations: an Open-Access Database of Raw Ultrasonic Signals. *Digital Diagnostics* (2021). DOI: `https://doi.org/10.17816/DD76511`.

[41] Ozgun, K., and Byram, B. C. Multidimensional clutter filtering of aperture domain data for improved blood flow sensitivity. *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* 68(8), 2645–2656 (2021). DOI: `https://doi.org/10.1109/TUFFC.2021.3073292`.

[42] Huang, Z., et al. Adaptive higher-order singular value decomposition clutter filter for ultrafast Doppler imaging of coronary flow under non-negligible tissue motion. *Ultrasonics* (2024). DOI: `https://doi.org/10.1016/j.ultras.2024.107307`.

[43] Garcia, D. SIMUS: an open-source simulator for medical ultrasound imaging. Part I: theory and examples. *Computer Methods and Programs in Biomedicine* 218, 106726 (2022). DOI: `https://doi.org/10.1016/j.cmpb.2022.106726`.

[44] Cigier, A., Varray, F., and Garcia, D. SIMUS: an open-source simulator for medical ultrasound imaging. Part II: comparison with four simulators. *Computer Methods and Programs in Biomedicine* 220, 106774 (2022). DOI: `https://doi.org/10.1016/j.cmpb.2022.106774`.

[45] Urban, A., et al. Real-time imaging of brain activity in freely moving rodents using functional ultrasound. *Nature Methods* 12(9), 873–878 (2015). DOI: `https://doi.org/10.1038/nmeth.3482`.

[46] Demené, C., et al. 4D microvascular imaging based on ultrafast Doppler tomography. *NeuroImage* 127, 472–483 (2016). DOI: `https://doi.org/10.1016/j.neuroimage.2015.11.014`.

[47] Slepian, D. Prolate spheroidal wave functions, Fourier analysis, and uncertainty—V: The discrete case. *Bell System Technical Journal* 57(5), 1371–1430 (1978). DOI: `https://doi.org/10.1002/j.1538-7305.1978.tb02104.x`.

[48] Thomson, D. J. Spectrum estimation and harmonic analysis. *Proceedings of the IEEE* 70(9), 1055–1096 (1982). DOI: https://doi.org/10.1109/PROC.1982.12433.

[49] Percival, D. B., and Walden, A. T. *Spectral Analysis for Physical Applications: Multitaper and Conventional Univariate Techniques*. Cambridge University Press (1993).

[50] Guerci, J. R. *Space-Time Adaptive Processing for Radar*. Artech House (2003).

[51] Wicks, M. C., Rangaswamy, M., Adve, R., and Hale, T. B. Space-time adaptive processing: a knowledge-based perspective for airborne radar. *IEEE Signal Processing Magazine* 23(1), 51–65 (2006). DOI: https://doi.org/10.1109/MSP.2006.1593337.

[52] Melvin, W. L., and Guerci, J. R. Knowledge-aided signal processing: a new paradigm for radar and other advanced sensors. *IEEE Transactions on Aerospace and Electronic Systems* 42(3), 983–996 (2006). DOI: https://doi.org/10.1109/TAES.2006.248215.

[53] Gurram, P. R., and Goodman, N. A. Spectral-domain covariance estimation with a priori knowledge. *IEEE Transactions on Aerospace and Electronic Systems* 42, 1010–1020 (2006). DOI: https://doi.org/10.1109/TAES.2006.248209.

[54] Van Veen, B. D., and Buckley, K. M. Beamforming: a versatile approach to spatial filtering. *IEEE ASSP Magazine* 5(2), 4–24 (1988). DOI: https://doi.org/10.1109/53.665.

[55] Kuglin, C. D., and Hines, D. C. The phase correlation image alignment method. In *Proceedings of the IEEE International Conference on Cybernetics and Society* (1975).

[56] Stone, H. S., Orchard, M. T., Chang, E.-C., and Martucci, S. A. A fast direct Fourier-based algorithm for subpixel registration of images. *IEEE Transactions on Geoscience and Remote Sensing* 39(10), 2235–2243 (2001). DOI: https://doi.org/10.1109/36.957286.

[57] Foroosh, H., Zerubia, J. B., and Berthod, M. Extension of phase correlation to subpixel registration. *IEEE Transactions on Image Processing* 11(3), 188–200 (2002). DOI: https://doi.org/10.1109/83.988953.

[58] Capon, J. High-resolution frequency-wavenumber spectrum analysis. *Proceedings of the IEEE* 57(8), 1408–1418 (1969).

[59] Frost, O. L. An algorithm for linearly constrained adaptive array processing. *Proceedings of the IEEE* 60(8), 926–935 (1972).

[60] Kay, S. M. *Fundamentals of Statistical Signal Processing, Volume II: Detection Theory*. Prentice Hall (1998).

[61] Scharf, L. L., and Friedlander, B. Matched subspace detectors. *IEEE Transactions on Signal Processing* 42(8), 2146–2157 (1994).

[62] Ward, J. Space-time adaptive processing for airborne radar. MIT Lincoln Laboratory, Tech. Rep. 1015 (1994).

| Label group | Plain-English meaning (what it checks) |
|---|---|
| R–* | Regime preconditions: do the bands exist, do risk features vary, and is there tail headroom worth regularizing? |
| BG–* | Band geometry: after discretization, are Pf/guard/Pa non-empty and non-overlapping for the chosen (PRF, $T$)? |
| OC–* | Operator/covariance constraints: KA can only apply bounded, bandwise structure without uncontrolled mixing or conditioning pathologies. |
| G–* | Gating/localization: KA is allowed to act only where artifacts are likely and positives are unlikely (protected/do-not-penalize set preserved; scores unchanged). |
| Safety–* | Safety guards: if any check indicates score inflation or loss of invariance, KA is forced off. |

Table 1: Appendix label groups in plain English. Failing any condition triggers a conservative fallback ($w = 1$), which is an intended success mode out of regime.

Missing figure `figs/paper/stap_pipeline_bayes_block.pdf`.
Reproduce it with `PYTHONPATH=.pythonscripts/fig_stap_pipeline_bayes_block.py`.

Figure 1: System block diagram for the Knowledge-Aided Space-Time Adaptive Processing (KA–STAP) architecture. Beamformed kilohertz data initially undergoes baseline motion-compensated clutter filtering (MC–SVD), yielding a residual cube $X_{\text{base}}$. The residual is processed via two parallel branches: (top) the uninformed STAP core computes the matched-subspace statistic $S_{\text{stap,pre}}(y, x)$; (bottom) a spatial-spectral feature extractor evaluates localized band-energy statistics to compute a bounded shrink-only weight $w(i) \in (0, 1]$ (equivalently $\gamma(i) = 1/w(i) \geq 1$). The branches recombine to form the strictly bounded post-regularization score $S_{\text{stap,post}}(y, x) = w\, S_{\text{stap,pre}}(y, x)$ (shrink-only).

# A KA Prior (Modality–Agnostic)

In this part we describe regime conditions and operator-level sufficient structure for a knowledge-aided (KA) temporal covariance transform layered on top of a baseline slow-time detector. The analysis is formulated in terms of abstract band projectors $(P_o, P_f, P_g, P_a)$, alias features, and score distributions, and does not depend on any particular imaging modality. Only in later sections do we instantiate these conditions for functional ultrasound (fUS) brain imaging with concrete band choices, acquisition parameters, and baselines.

### How to read this appendix (plain-English map)

This appendix lists explicit preconditions under which the KA prior is allowed to modify scores; if any precondition fails, the prior is disabled ($w \equiv 1$; KA off). The labels below are grouped by purpose.

## A.1 Regime Preconditions (Data–Level, Non–Negotiable)

The most important empirical lesson is: even if the KA operator $Q$ and band design $(P_f, P_a)$ are perfect in an abstract sense, *uplift requires the data themselves to be in a compatible regime.* This section codifies that.

We write $S(f)$ for the (multi–taper) PSD of a baseline–whitened tile, and define

$$E_f := \sum_{f \in \mathcal{B}_f} S(f), \quad E_a := \sum_{f \in \mathcal{B}_a} S(f), \quad E_g := \sum_{f \in \mathcal{B}_g} S(f),$$

with $\mathcal{B}_f, \mathcal{B}_a, \mathcal{B}_g$ the flow, alias, and guard bands in frequency.

### A.2 R–1: Band Occupancy Separation

We require that the fixed bands $(\mathcal{B}_f, \mathcal{B}_a)$ are physically meaningful for the current PRF, slow–time length $T$, and DPSS parameters, in the sense that label statistics show actual occupancy.

On a held–out labeled sample:

$$\mathbb{P}\{f_{\text{peak}} \in \mathcal{B}_f \mid H_1\} \geq p_{f,\min}, \qquad p_{f,\min} \in [0.5, 0.7], \tag{1}$$

$$\mathbb{P}\{f_{\text{peak}} \in \mathcal{B}_a \mid H_0\} \geq p_{a,\min}, \qquad p_{a,\min} \in [0.2, 0.3], \tag{2}$$

where $f_{\text{peak}}$ is the frequency at which $S(f)$ attains its maximum and $H_1/H_0$ denote flow/background tiles.

Interpretation: a majority of flow tiles have their dominant PSD peak *inside* the flow band, and a nontrivial fraction of background tiles have their dominant peak in the alias band. This ensures that the band projectors $(P_f, P_a)$ actually correspond to distinct physical processes.

**Implementation in Brain–* fixed profiles.** For the main Brain-* simulation regimes, the raw k-Wave RF tensor $x(t, a, e)$ (angle index $a$, element index $e$) lives on an angle and element grid that does not align pixelwise with the STAP PD masks ($N_y \times N_x$). We therefore implement R–1 using the whitened multi–taper PSD and band-ratio feature statistics computed on the *same* spatial grid as the exported PD scoring maps (the right-tail score $S_{\text{PD}}$ stored as `score_pd_*.npy`). We take $f_{\text{peak}}$ from the whitened PSD peaks used in the band-ratio recorder and report the fraction of $H_1$ tiles whose peak lies in $\mathcal{B}_f$ and the fraction of $H_0$ tiles whose peak lies in $\mathcal{B}_a$ from those tilewise peak statistics. When the RF tensor is spatially aligned with the masks (e.g. 1D pilots), we instead use a direct FFT on $x(t, \cdot, \cdot)$ as in the definition above.

### A.3 R–2: Alias Score Class Separation

We generalize the alias metric used for localization.

**Alias score.** Let $m_{\text{alias}}(z)$ be any scalar score derived from the baseline PSD and/or covariance satisfying:

- $m_{\text{alias}}$ increases when $E_a$ grows relative to $E_f$,
- $m_{\text{alias}}$ is larger when Pa energy is more coherent or concentrated (e.g. a strong tone),
- $m_{\text{alias}}$ is penalized when guard/edge energy $E_g$ is large.

Examples (not exhaustive):

- Log band ratio: $m_{\text{alias}}(z) = \log \frac{E_a + \epsilon}{E_f + \epsilon}$,
- Combination of band fractions: $m_{\text{alias}}(z) = \frac{E_a}{E_f + E_a + E_g} \cdot c_a$ where $c_a$ is an alias–band coherence metric (e.g. leading eigenvalue fraction of a band–restricted covariance),
- Peak–location based scores (peak in Pa, high tone prominence).

For KA to be meaningfully guided by alias evidence, we require *class–differential separation* of $m_{\text{alias}}$:

$$\text{median}(m_{\text{alias}} \mid H_0) - \text{median}(m_{\text{alias}} \mid H_1) \;\geq\; \Delta_{\min}, \tag{3}$$

with $\Delta_{\min}$ set so that alias dominance differs by at least a factor $\approx e^{\Delta_{\min}}$ in expectation (for log ratio, $\Delta_{\min} \approx 0.3$ corresponds to a $\approx 35\%$ relative difference in $E_a/E_f$ medians).

If (3) fails (alias score almost identical on $H_0$ and $H_1$), the alias-localization rule (Appendix J.11) has no reliable signal and KA uplift cannot be expected.

### A.4 R–3: Baseline Headroom

Let $S^-$ be the baseline score distribution on negatives, and $\text{pAUC}_{\text{base}}(\alpha)$ be the partial AUC up to FPR $\alpha$.

We require:

- Sufficient *quantile density* at the target FPR $\alpha$ (as already assumed implicitly in SL–2): the density $f_-(t_\alpha)$ at the $\alpha$–quantile $t_\alpha$ is bounded away from 0.
- Baseline not already numerically saturated: for a chosen $\alpha_{\max}$ (e.g. $10^{-3}$),

$$\text{pAUC}_{\text{base}}(\alpha \leq \alpha_{\max}) \leq \text{pAUC}_{\max} - \varepsilon_{\text{AUC}}, \qquad \varepsilon_{\text{AUC}} > 0,$$

  where $\text{pAUC}_{\max}$ is the ideal partial AUC under perfect separation.

If the baseline already achieves near–maximal pAUC at the FPR of interest, even a perfect KA transform cannot provide significant gains; in this regime KA can serve as a regularizer or safety mechanism, but not as a source of measurable uplift.

### A.5 Operator–Level Sufficient Structure (Non–Negotiable Core)

This section is unchanged at the core: it describes conditions on $Q$ alone under which KA behaves like a band–wise scalar rescaling of scores, up to small perturbations.

### A.6 OC–1: Approximate Band Commutation

We require $Q$ to be nearly block–diagonal with respect to $(P_o, P_f, P_g, P_a)$:

$$\|[Q, P_f]\|_2 \leq \varepsilon_{\text{mix}}, \qquad \|[Q, P_a]\|_2 \leq \varepsilon_{\text{mix}},$$

for some small $\varepsilon_{\text{mix}} \in [0, 1)$ (typical target $\leq 0.05$).

Interpretation: $Q$ preserves band subspaces up to small mixing; it does not rotate energy from Pf into Pa or vice versa in any substantial way. This rules out the failure mode where KA accidentally feeds alias into the flow numerator.

**Runtime label–free check.** Let $\widehat{Q} := R_\beta^{-1/2}\widehat{R}^{1/2}$ be the realized KA transform. Estimate

$$\widehat{\varepsilon}_{\text{mix}} := \max\left\{ \frac{\left\|P_f \widehat{Q} P_a\right\|_F}{\left\|P_f \widehat{Q} P_f\right\|_F}, \frac{\left\|P_a \widehat{Q} P_f\right\|_F}{\left\|P_a \widehat{Q} P_a\right\|_F} \right\},$$

and require $\widehat{\varepsilon}_{\text{mix}} \leq \varepsilon_{\text{mix}}$.

### A.7 OC–2: Differential Band Gains with Spectral Bounds

We require that $Q^{\mathrm{H}}Q$ is well approximated by bandwise scalars on the ranges of $P_f, P_a, P_o$, i.e., there exist $s_f, s_a, s_o > 0$ and tolerances $\delta_f, \delta_a, \delta_o \in [0, 1)$ such that

$$\begin{aligned} s_f^2(1-\delta_f)\, P_f &\preceq P_f Q^{\mathrm{H}} Q P_f \preceq s_f^2(1+\delta_f)\, P_f, \\ s_a^2(1-\delta_a)\, P_a &\preceq P_a Q^{\mathrm{H}} Q P_a \preceq s_a^2(1+\delta_a)\, P_a, \\ s_o^2(1-\delta_o)\, P_o &\preceq P_o Q^{\mathrm{H}} Q P_o \preceq s_o^2(1+\delta_o)\, P_o, \end{aligned}$$

with *strict separation*

$$\frac{s_a}{s_f} \geq \Lambda > 1 + c\,\varepsilon_{\mathrm{mix}}, \qquad c \in [2, 4].$$

Interpretation: within each band, $Q^{\mathrm{H}}Q$ behaves like a scalar $(s_f^2, s_a^2, s_o^2)$ up to modest anisotropy $\delta_\bullet$, and the alias band gains are strictly larger than flow band gains.

**Runtime label–free check.** Compute eigenvalues of the band–projected operators:

$$\lambda_{\min}^f, \lambda_{\max}^f := \lambda_{\min\,/\,\max}(Q^{\mathrm{H}}Q|_{P_f}),$$

and similarly for $a, o$. Set

$$\widehat{s}_f^2 := \tfrac{1}{2}(\lambda_{\min}^f + \lambda_{\max}^f), \quad \widehat{\delta}_f := \frac{\lambda_{\max}^f - \lambda_{\min}^f}{\lambda_{\max}^f + \lambda_{\min}^f},$$

and analogously for $\widehat{s}_a, \widehat{s}_o$ and $\widehat{\delta}_a, \widehat{\delta}_o$. Require $\widehat{\delta}_\bullet \leq \delta_\bullet$ and $\widehat{s}_a/\widehat{s}_f \geq \Lambda$.

### A.8 Consequence: Approximate Multiplicative Effect on Scores

Under OC–1–OC–2, for any $z$ one has

$$\mathrm{BR}(Qz) = \frac{z^{\mathrm{H}} Q^{\mathrm{H}} P_f Q z}{z^{\mathrm{H}} Q^{\mathrm{H}} P_a Q z} \approx \frac{s_f^2}{s_a^2} \cdot \mathrm{BR}(z) \cdot (1 + \Delta_{\mathrm{BR}}(z))$$

with $|\Delta_{\mathrm{BR}}(z)| \leq C_1(\delta_f + \delta_a) + C_2\varepsilon_{\mathrm{mix}}$, for constants $C_1, C_2$ depending only on band dimensions. A similar approximation holds for $\mathrm{GLRT}_\rho(z)$ with denominators involving $s_a^2, s_o^2$.

Thus, if $\Lambda = s_a/s_f > 1$ and tolerances are small, BR/GLRT on a tile are rescaled approximately by a *bandwise constant* factor, with predictable sign (e.g. downwards when $s_a > s_f$).

### A.9 Band Geometry & Leakage Budget

This section remains as in the original doc: its role is to ensure that the projectors $P_f, P_a$ correspond to well–separated spectral bands given the PRF, $T$, and DPSS parameters, using DPSS tapers and multi-taper spectral estimates in the sense of Slepian and Thomson [47–49].

### A.10 BG–1: Leakage Separation Margin

Let $G(f, g)$ be the multi–taper spectral leakage kernel associated with your DPSS design (known from design). Require

$$\max_{f \in \mathcal{B}_f} \sum_{g \in \mathcal{B}_a} G(f, g) \le \ell_{\max}, \qquad \ell_{\max} \le 0.15.$$

Interpretation: at most 15% of energy from an on–grid flow bin leaks into the alias band, and vice versa. This supports keeping the band mixing in OC–1 small after whitening.

### A.11 BG–2: Band–Edge Guard

Define a guard band $\mathcal{B}_g$ separating $\mathcal{B}_f$ and $\mathcal{B}_a$ such that

$$\operatorname{dist}(\mathcal{B}_f, \mathcal{B}_a) \ge b_g,$$

for some guard width $b_g$ in frequency (or bins). Require that any energy within $b_g$ of $\mathcal{B}_f$ leaks only modestly into $\mathcal{B}_a$:

$$\sum_{g:\, \operatorname{dist}(g, \mathcal{B}_f) \le b_g} G(f, g) \le \ell_{\text{edge}}, \qquad \ell_{\text{edge}} \le 0.1.$$

Interpretation: avoid designing $\mathcal{B}_f, \mathcal{B}_a$ so tightly that off–grid peaks straddle the boundary; the guard ensures band–edge leakage is limited.

### A.12 BG–3: Within–Band Condition Numbers

From the baseline whitened covariance in each band, require

$$\kappa_f := \frac{\lambda_{\max}(\widehat{R}|_{P_f})}{\lambda_{\min}(\widehat{R}|_{P_f})} \le \kappa_f^{\max}, \qquad \kappa_a := \frac{\lambda_{\max}(\widehat{R}|_{P_a})}{\lambda_{\min}(\widehat{R}|_{P_a})} \le \kappa_a^{\max},$$

with $\kappa^{\max} \in [10, 40]$ depending on $T$ and sample support.

Interpretation: this makes it feasible to construct $R_\beta$ that achieves OC–2 with moderate $\beta, \delta$ without overly crushing the flow band.

### A.13 Covariance blending and localization (summary)

KA reshapes covariances only within bands. We use either a commuting band-scalar transform with target gains $(s_f, s_a, s_o)$ or a trace-normalized blend $R_\beta$ with small diagonal lift; both enforce band-wise structure (no off-block priors). Localization restricts KA to alias-dominated, low-coverage tiles using alias scores, guard ratios, optional flow coherence/coverage checks, baseline score windows, depth/PD/registration cues, and is validated via coverage rates $p_-, p_+$. Full localization definitions are provided in Appendix J.11 (PD-only instantiation in Appendix G).

### A.14 Score–Level Guarantees

This section is unchanged in structure; we only make the dependence on $p_-, p_+$ explicit.

### A.15 SL–1: Per–Tile Multiplicative Effect

Under OC–1–OC–2, for any $z$,

$$\mathrm{BR}_{\mathrm{KA}}(z) = \mathrm{BR}(Qz) \approx \frac{s_f^2}{s_a^2} \cdot \mathrm{BR}_{\mathrm{base}}(z) \cdot (1 + O(\delta_\bullet + \varepsilon_{\mathrm{mix}})).$$

Similarly,

$$\mathrm{GLRT}_{\rho,\mathrm{KA}}(z) \approx \frac{s_f^2}{\alpha_a(z)\, s_a^2 + \alpha_o(z)\, s_o^2} \cdot \mathrm{GLRT}_{\rho,\mathrm{base}}(z),$$

where

$$\alpha_a(z) := \frac{z^{\mathrm{H}} P_a z}{z^{\mathrm{H}}(P_a + \rho I)z} \in [0,1], \quad \alpha_o(z) := \frac{\rho\, \|z\|^2}{z^{\mathrm{H}}(P_a + \rho I)z} \in [0,1].$$

On alias–dominant tiles (large $\alpha_a(z)$), both BR and GLRT are approximately multiplied by $\mu := s_f^2/s_a^2 < 1$.

### A.16 SL–2: Tail Threshold Improvement at Fixed FPR

Let $S_{\mathrm{base}}^-$ be the baseline negative score distribution, with CDF $F_-$. Let $t_\alpha$ be the $\alpha$–quantile: $F_-(t_\alpha) = 1 - \alpha$. After KA with gating, the negative CDF becomes a mixture:

$$F'_-(t) = (1 - p_-)F_-(t) + p_- F_-(t/\mu),$$

ignoring higher–order mixing terms and assuming $\mu < 1$ is the multiplicative shrinkage factor on gated negatives. Under mild regularity and for $\alpha$ in the low–FPR regime,

$$t'_\alpha \approx t_\alpha - \frac{p_-(1-\mu)}{f_-(t_\alpha)},$$

so that $t'_\alpha < t_\alpha$ whenever $p_- > 0$ and $\mu < 1$. In the PD-mode experiments in this paper we report operating points using the exported PD flow-evidence score map $S_{\mathrm{PD}}$ (stored as `score_pd_*.npy`; higher = more flow-like): choose $\tau$ so that $\mathbb{P}\{S_{\mathrm{PD}} \geq \tau \mid \mathrm{bg}\} = \alpha$ and evaluate $\mathbb{P}\{S_{\mathrm{PD}} \geq \tau \mid \mathrm{flow}\}$. For compatibility, acceptance bundles may also include `pd_*.npy`, which we denote $\mathrm{PD}^{\downarrow}$ and relate deterministically by $S_{\mathrm{PD}} = -\mathrm{PD}^{\downarrow}$.

**Interpretation (plain language).** KA is designed to shrink scores only on a subset of artifact-dominated background tiles. This selectively compresses the extreme right tail of $H_0$ scores, which lowers the background threshold needed to hit a target FPR. Lowering that threshold can reveal additional true flow pixels without increasing the false-positive rate, because scores never increase on penalized pixels. Equivalently, scaling down a fraction $p_-$ of top–tail negatives removes mass from the negative tail and shifts $t_\alpha$ left; uplift is only plausible when $p_+$ (positive gating rate) is small.

We emphasize that the SL–2 guarantee is conditioned on:

- Regime preconditions R–1/R–2/R–3,
- Gating coverage in the range $0.2 \leq p_- \leq 0.5$, $p_+ \leq 0.1 p_-$.

## A.17 SL–3: Positive Collateral Bound

Let $p_+$ be the positive gating rate and assume gated positives suffer at most a multiplicative drop $\nu := s_f^2/s_a^2 \cdot (1 + O(\delta_\bullet + \varepsilon_{\text{mix}}))$. For any threshold $t'_\alpha$ satisfying $F'_-(t'_\alpha) = 1 - \alpha$, the TPR under KA satisfies

$$\text{TPR}_{\text{KA}}(t'_\alpha) \geq \text{TPR}_{\text{base}}(t_\alpha) - \eta\, \Phi(\nu),$$

where $0 \leq \eta \lesssim \frac{p_+}{p_-}$ (up to geometry) and $\Phi(\nu)$ is the fraction of positives near $t'_\alpha$ that are moved below the threshold. For $p_+ \ll p_-$ and $\nu$ close to 1, the loss term is dominated by the gain from SL–2.

## A.18 Evaluation Discipline

This section remains as in the original document: it covers GLRT parameter sharing, block/cluster–aware tail CIs, and effective negatives. These aspects ensure fair comparison but do not themselves cause uplift.

## A.19 Feature Statistics and Pass/Fail Checklist

At runtime, label–free feature statistics must support the following checks:

(i) Band design: BG–1/BG–2/BG–3 satisfied for current DPSS/PRF/$T$.
(ii) KA operator: $\widehat{\varepsilon}_{\text{mix}} \leq \varepsilon_{\text{mix}}$ (OC–1); $\widehat{s}_a/\widehat{s}_f \geq \Lambda$ and $\widehat{\delta}_\bullet \leq 0.1$ (OC–2).
(iii) Localization primitives: risk scores $m_{\text{alias}}$ and $m_{\text{guard}}$, guard ratios, flow coverage, FPC, depth, PD, registration; localization indicator $g$ constructed via G–1/G–2/G–2'/G–3/G–4.
(iv) Invariance sentinel: if $\widehat{s}_a/\widehat{s}_f \in [1-\epsilon, 1+\epsilon]$ with $\epsilon \leq 0.02$, KA is effectively a scalar; disable it (see Section A.21).

## A.20 Knowledge-Aided Prior: shrink-only score-space penalty

To keep KA conservative under heterogeneity, we implement a deterministic, feature-conditioned *prior* that acts in score space as a shrink-only penalty. Concretely, the post-KA score is defined as

$$S_{\text{post}}(i) = w(i)\, S_{\text{pre}}(i), \qquad w(i) \in (0, 1],$$

equivalently $T_{\text{post}} = T_{\text{pre}} + \log w$ when working with log-likelihood scores (MAP log-odds decomposition). By construction, $w \equiv 1$ (no KA effect) is an explicit, expected outcome whenever the feature statistics are uninformative or fail integrity checks.

### A.20.1 Feature-space regimes (R0/R1/R2)

We partition the feature space into three regimes per volume (or per slice/plane in PD-only mode):

- $\mathcal{R}_0$ **(uninformative).** The prior is uninformative and $w \equiv 1$; output equals the baseline detector (STAP on IQ; PD-only detector on PD-only). Feature statistics are still recorded.
- $\mathcal{R}_1$ **(guarded shrink).** A bounded, shrink-only penalty may be applied on a localized set of high-risk tiles while strictly protecting a high-confidence set. No uplift claim is implied.
- $\mathcal{R}_2$ **(alias-dominant; experimental).** Features strongly indicate alias contamination and additional label-free actionability evidence suggests that shrink preferentially targets false activations. This regime is used only as a conservative tail-hygiene mode to gate reporting of KA deltas; it is not a default deployment mode or the basis for primary performance claims.

For auditability, the implementation also logs a reason label (why $w \equiv 1$) and a *risk mode* (alias vs guard) so that $\mathcal{R}_0$ behavior is interpretable. (For backward compatibility, exported metadata may also include legacy string labels such as `C0_OFF`/`C1_SAFETY`/`C2_UPLIFT` mapped to $\mathcal{R}_0/\mathcal{R}_1/\mathcal{R}_2$.)

### A.20.2 Feature primitives (band-energy statistics)

On each tile $i$ we record band energies using fixed, disjoint projectors $(P_o, P_f, P_g, P_a)$ with $P_o + P_f + P_g + P_a = I$:

$$E_o(i) = z_i^{\mathrm{H}} P_o z_i, \quad E_f(i) = z_i^{\mathrm{H}} P_f z_i, \quad E_g(i) = z_i^{\mathrm{H}} P_g z_i, \quad E_a(i) = z_i^{\mathrm{H}} P_a z_i,$$

where $z_i \in \mathbb{C}^T$ (conjugate-transpose $(\cdot)^{\mathrm{H}}$; in real PD-only instantiations $(\cdot)^{\mathrm{H}}$ reduces to $(\cdot)^\top$). We optionally track a DC subcomponent $E_{\mathrm{dc}}(i)$ (e.g. energy in a narrow DC tone within $P_o$) as an auxiliary feature statistic. We define

$$E_{\mathrm{tot}}(i) = E_o + E_f + E_g + E_a + \epsilon,$$

and form robust, label-free ratios:

$$m_{\mathrm{alias}}(i) = \log \frac{E_a(i) + \epsilon}{E_f(i) + \epsilon}, \quad r_g(i) = \frac{E_g(i)}{E_{\mathrm{tot}}(i)}, \quad r_f(i) = \frac{E_f(i)}{E_{\mathrm{tot}}(i)}.$$

We also use a flow-coverage proxy $c_f(i) \in [0, 1]$ (fraction of tile pixels in the flow mask), a depth proxy $d(i) \in [0, 1]$ when available, and a baseline detector score $S_{\mathrm{base}}(i)$ on which KA operates (defined in a flow-evidence convention so that higher means "more flow-like"; in PD-mode this is $S_{\mathrm{base}} = S_{\mathrm{PD}} = \texttt{score_pd}$).

### A.20.3 Proxy sets and actionability tests

Because class labels are unavailable at runtime, the regime selector uses two proxy sets:

$$\mathcal{T}_{\mathrm{bg}} := \{i : c_f(i) \le c_{\mathrm{bg}}\}, \qquad \mathcal{T}_{\mathrm{flow}} := \{i : c_f(i) \ge c_{\mathrm{flow}}\},$$

with frozen design values (typical $c_{\mathrm{bg}} \approx 0.05$, $c_{\mathrm{flow}} \approx 0.20$). These thresholds are intentionally conservative, order-of-magnitude safety bounds chosen a priori to avoid penalizing dense-flow tiles; they are not optimized to maximize ROC on the evaluated datasets. The uplift-eligibility checks are reframed as *label-free actionability*:

- **Background-vs-flow separation:** $\Delta_{bf} = \mathrm{median}(m_{\mathrm{alias}} \mid \mathcal{T}_{\mathrm{bg}}) - \mathrm{median}(m_{\mathrm{alias}} \mid \mathcal{T}_{\mathrm{flow}})$.
- **Tail association in background:** within $\mathcal{T}_{\mathrm{bg}}$, define a tail subset $\mathcal{T}_{\mathrm{tail}}$ as the top 1% of $S_{\mathrm{base}}$ and a mid subset $\mathcal{T}_{\mathrm{mid}}$ as a central quantile band; require that $m_{\mathrm{alias}}$ is higher in $\mathcal{T}_{\mathrm{tail}}$ than in $\mathcal{T}_{\mathrm{mid}}$ (risk-feature association explains "false-alarm-looking" background tiles).

When guard fractions dominate, the actionable regime $\mathcal{R}_2$ is disallowed but guarded shrink $\mathcal{R}_1$ may still be permitted. In this case we switch to a guard-risk mode using $m_{\mathrm{guard}}(i) = \log \frac{E_g(i)+\epsilon}{E_f(i)+\epsilon}$ (or equivalently $r_g(i)$) and apply the same tail association check using $m_{\mathrm{guard}}$.

### A.20.4 Protected set, candidate window, and shrink mapping

KA must not disturb high-confidence flow tiles. We define a protected set

$$\mathcal{T}_{\text{prot}} := \{i : c_f(i) \geq c_{\text{flow}}\} \cup \{i : S_{\text{base}}(i) \geq Q_{q_{\text{hi}}}(S_{\text{base}})\} \cup \{i : r_f(i) \geq r_{f,\text{prot}}\},$$

and a candidate window $\mathcal{T}_{\text{cand}}$ consisting of non-protected tiles whose baseline scores lie in an "uncertain" quantile band $[Q_{q_{\text{lo}}}, Q_{q_{\text{hi}}}]$. On $\mathcal{T}_{\text{bg}}$ we set an adaptive risk threshold $\tau_{\text{risk}} = Q_{q_{\text{risk}}}(m_{\text{risk}} \mid \mathcal{T}_{\text{bg}})$ where $m_{\text{risk}}$ is $m_{\text{alias}}$ or $m_{\text{guard}}$ depending on risk mode. The localization indicator is

$$g(i) = \mathbf{1}\{i \in \mathcal{T}_{\text{cand}},\ m_{\text{risk}}(i) \geq \tau_{\text{risk}}\}.$$

For penalized tiles (those with $g(i) = 1$) we compute a bounded shrink factor

$$w(i) = \max\left(w_{\min}, \exp(-k\, u(i))\right), \qquad u(i) = \max\left(0, \frac{m_{\text{risk}}(i) - \tau_{\text{risk}}}{\text{MAD}(m_{\text{risk}} \mid \mathcal{T}_{\text{bg}}) + \epsilon}\right),$$

with $w_{\min} \in (0, 1]$ so the corresponding scale $\gamma(i) = 1/w(i)$ is finite, and set $w(i) = 1$ on non-penalized and protected tiles. In Doppler/IQ PD-mode bundles we export the per-pixel scale $\gamma(i) = 1/w(i) \geq 1$ (averaged over overlapping tiles) and apply it on the union of penalized tiles as

$$\text{PD}^{\downarrow}_{\text{post}}(i) = \gamma(i)\, \text{PD}^{\downarrow}_{\text{pre}}(i),$$

which inflates the legacy PD-mode map $\text{PD}^{\downarrow}$ on penalized regions. Since $S_{\text{PD}} = -\text{PD}^{\downarrow}$, this is equivalent to $S_{\text{post}} = w\, S_{\text{pre}}$ and therefore is non-increasing in score space. For PD-only detectors such as PD-z that take both signs, we use hard-suppression/soft-shrink variants (Appendix G) so that shrink-only is literal for signed scores.

### A.20.5 Sentinels, reason codes, and auditability

The regime selector defaults to $\mathcal{R}_0$ ($w \equiv 1$) when any of the following hold: insufficient background proxy support, flat risk statistics (e.g. $\text{IQR}(m_{\text{risk}} \mid \mathcal{T}_{\text{bg}})$ too small), guard domination that is not actionable, coverage outside bounds (localization inert or global), invariance (penalized $w$ nearly constant), or a monotonicity guard failure on near-protected tiles. When $w \not\equiv 1$, we record coverage and strength summaries (e.g. $p_{\text{shrink}} = \mathbb{P}\{g = 1 \mid \mathcal{T}_{\text{cand}}\}$ and quantiles of $w$) along with a short reason label (e.g. `alias_metric_flat`, `guard_dominates_not_actionable`, `coverage_out_of_bounds`, `monotonicity_guard_failed`). This makes KA behavior reviewable and supports the intended posture: $\mathcal{R}_0$ is an expected success regime, $\mathcal{R}_1$ is a conservative safety regime, and $\mathcal{R}_2$ is a research-only regime used to gate uplift reporting.

### A.20.6 PD-only instantiation

On PD-only datasets (Macé/Urban, fUS-BMI, cranial-window PD), the same regime selection is applied after replacing Doppler bands with hemodynamic bands and treating tile-averaged PD time series as the slow-time signal. The feature primitives $(E_f, E_a, E_g)$ and the shrink-only map $w(i)$ are defined identically, and the regime label is produced per plane. This yields a uniform export schema across Doppler IQ and hemodynamic PD regimes.

Figure 2 shows a compact prior/regime summary panel over a representative sweep across regimes: most runs remain in $\mathcal{R}_0$ with an explicit disable reason, while a minority enter $\mathcal{R}_1$ and apply a mild, localized shrink in an alias- or guard-risk lane (never boosting scores).

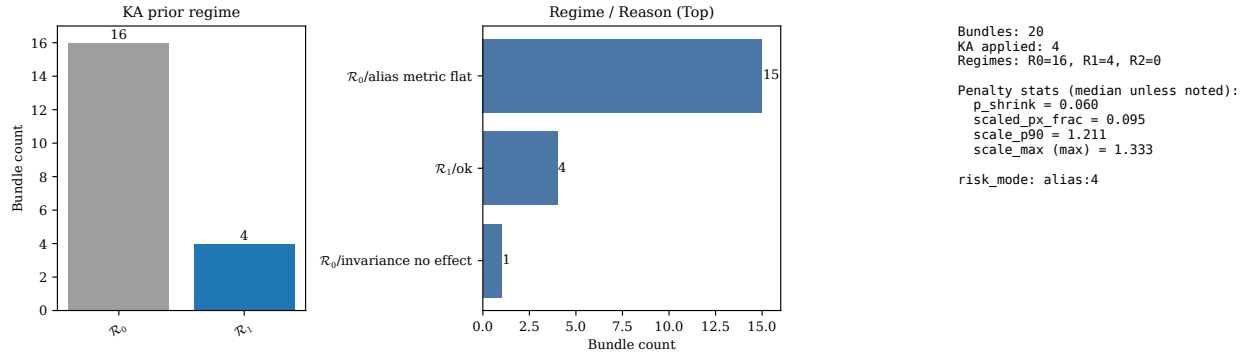


Figure 2: KA prior summary panel on a representative sweep. Left: regime counts. Middle: top (regime, reason) labels. Right: summary of shrink coverage and strength on runs where $w \not\equiv 1$.

## A.21 Regime Sentinel and Safety Guard

Experience shows that even when OC/BG/CB/G conditions are roughly satisfied, there can exist regimes where KA is either inert or harmful (e.g. risk feature flat, localization never triggers or triggers everywhere, baseline ROC saturated). This section formalizes when KA should be *explicitly disabled*.

## A.22 RS–1: Regime Sentinel (Design–Time)

On a held–out labeled configuration, check:

- **RS–1a: Band occupancy.** If R–1 fails (e.g. most flow peaks near DC instead of Pf, or alias peaks rarely in Pa), the band design is not physically realized; do not deploy KA.
- **RS–1b: Alias separation.** If R–2 fails (alias scores $m_{\text{alias}}$ show negligible difference between classes), alias localization has no reliable signal; do not deploy KA.
- **RS–1c: Localization coverage.** If, after choosing $q_{\text{alias}}, c_{f,\max}, c_{\text{pc}}$, the empirical localization rates satisfy $p_- \approx 0$ or $p_- \approx 1$, or $p_+ \approx p_-$, then localization is either inert, global, or non–differential; do not expect uplift and default to $\mathcal{R}_0$.
- **RS–1d: Baseline saturation.** If $\text{pAUC}_{\text{base}}$ in the target FPR range is already at or near maximum and the score ratio distribution under KA has median close to 1 with tiny spread, you are in a ROC–invariant regime; KA can be left off.

## A.23 Safety–1: Score Monotonicity Guard (Runtime)

Even when regime preconditions and localization coverage hold, we require that KA does not produce grossly pathological score transformations on *penalized flow tiles*. Let $S_{\text{base}}, S_{\text{KA}}$ denote baseline and KA scores on the chosen metric (BR or GLRT).

For the set of penalized flow tiles, define

$$r_i := \frac{S_{\text{KA}}(i)}{\max(S_{\text{base}}(i), \varepsilon)}, \quad u_i := \log r_i,$$

with a small $\varepsilon > 0$.

Let

$$\text{median}_f := \text{median}(u_i), \quad \text{IQR}_f := Q_{0.75}(u_i) - Q_{0.25}(u_i),$$

and $\tau_f$ the Kendall $\tau$ rank correlation between $S_{\text{KA}}$ and $S_{\text{base}}$ on these pixels.

We require:

- **Sample size.** Apply the guard only if the number of gated flow pixels exceeds a threshold $N_{\min}$ (e.g. $N_{\min} = 200$). If not, mark KA as "too sparse to evaluate" but do not disable it solely on this basis.
- **Flow monotonicity.** KA is marked unsafe if either

$$\text{IQR}_f > \log 10 \quad (\text{heterogeneity } > \times 10), \quad \text{or}$$
$$\text{median}_f > \log 10 \quad \text{and} \quad |\tau_f| < 0.1,$$

  i.e. if KA systematically boosts flow scores by more than an order of magnitude while destroying rank consistency, or applies extremely heterogeneous multiplicative effects.

If Safety–1 fails, KA is disabled at runtime for that configuration and the system falls back to pure STAP.

## A.24 What the Regime Does Not Promise (Updated)

KA–STAP may *fail to provide uplift* or may be disabled by the regime sentinel / safety guard in the following cases:

- If $\Lambda \approx 1$ (band–constant scaling, invariance sentinel) or $\varepsilon_{\text{mix}}$ is large, global band–wise transforms are nearly ROC–invariant or harmful; expect parity or degradation.
- If R–2 alias separation fails, i.e. $m_{\text{alias}}$ shows no class–differential behavior, alias localization cannot preferentially capture alias–dominated negatives; KA is out of regime.
- If localization is not class–differential (offline $p_+ \approx p_-$) or $p_-$ is either tiny or nearly one, KA is either inert or global; SL–2 no longer applies.
- If baseline ROC is already saturated in the target FPR range, even an ideal KA transform cannot move the operating point significantly.
- If Safety–1 indicates that KA acts in a grossly non–monotone way on penalized flow tiles (e.g. large heterogeneous boosts), KA is disabled for that configuration.

In all these regimes, MC–SVD or STAP–only should be preferred; KA can be viewed as an optional regularizer but not a source of guaranteed uplift.

## A.25 Safety and Regime Sentinel Summary

For deployed use it is helpful to summarize, in plain language, when KA is allowed to act and when it is automatically turned off.

- *When KA is allowed to act.* KA is enabled only in regimes that satisfy the structural preconditions (R–1/R–2/R–3), have useful but not global localization coverage (roughly $0.2 \leq p_- \leq 0.5$ with $p_+ \ll p_-$), and pass the invariance and Safety–1 checks. In this case KA behaves as a controlled, band–aware regularizer on top of STAP: it is permitted to shrink scores on alias–dominated negatives but must remain monotone and mild on flow tiles.
- *When KA is turned off.* KA defaults to $\mathcal{R}_0$ by design when band occupancy or alias separation fail (R–1/R–2), when localization becomes inert or non–differential, when the baseline ROC

is already saturated in the target low–FPR band (R–3), or when Safety–1 detects non–monotone, highly heterogeneous score changes on penalized flow tiles. The invariance sentinel also forces $\mathcal{R}_0$ when the realized gains satisfy $\widehat{s}_a/\widehat{s}_f \approx 1$ so that KA is effectively a scalar rescaling.

- *Behavior in skull/OR–like regimes.* In skull/OR realism regimes such as Brain–SkullOR, alias power deliberately leaks into $H_1$ and R–2 fails by construction. In these regimes KA is expected to be off or ROC–neutral; the primary detector is the fixed STAP PD profile, with MC–SVD/RPCA/HOSVD as reference baselines. This is a safety feature, not a failure of the method: whenever the regime preconditions are not met, the system falls back to STAP alone rather than forcing KA to act.

# B Supplementary Results

This appendix collects additional dataset audits and diagnostic panels that support the main paper's methodology, but are not required for the primary labeled low-FPR detection claims.

**Evaluation discipline (summary).** Quantitative ROC claims are restricted to regimes with independent labels (Brain-* simulator truth; Gammex B-mode structural masks). Operating points are reported under a fixed-profile protocol by selecting thresholds on labeled (or structurally labeled) $H_0$ sets at target FPR and evaluating TPR on the corresponding $H_1$ set; the empirically supported floor $fpr_{\min} = 1/n_{\text{neg}}$ is reported. Uncertainty is quantified at the natural unit of variation (windows for Brain-*; frames for cine acquisitions) rather than treating pooled pixels as independent samples. Conditional execution is treated as a changed decision rule and is reported only with explicit leakage controls. For Twinkling sequences, PRF is treated as an explicit experimental assumption (frozen overrides) and band-discretization sentinels force conservative behavior when discretized bands are invalid. When KA is enabled, it is shrink-only and preserves a protected do-not-penalize set (scores unchanged on high-confidence flow locations); labeled ROC reporting uses the pre-KA STAP score.

**Where to find what.**

- Full ROC curve shapes for labeled regimes: Figure 3 (Brain-*) and Figure 4 (Gammex).
- STAP-only ablation and PRF override sensitivity (Gammex): Figure 7.
- Conditional-execution leakage ablation (Brain-OpenSkull): Table 4.
- Fixed-profile sensitivity sweep (Brain-OpenSkull; $\lambda$, tile size, $L_t$): Figure 6.
- Shin proxy-metric comparisons, runtime, and measured rigid-motion telemetry: Figure 13 and Figure 14.
- Corrected SIMUS/PyMUST moving-scatterer frozen-benchmark comparison and stage-symmetric detector-head audit: Tables 12 and 13.

## B.1 Full ROC curves for labeled regimes

The main text reports strict low-FPR operating points. Here we provide representative ROC-style curves to show the overall curve shape under the same masks and scoring conventions.

ROC curves are reported with the same unit-of-variation discipline as the operating-point tables. For Brain-* simulations, we compute ROC curves per 64-frame window and summarize them as

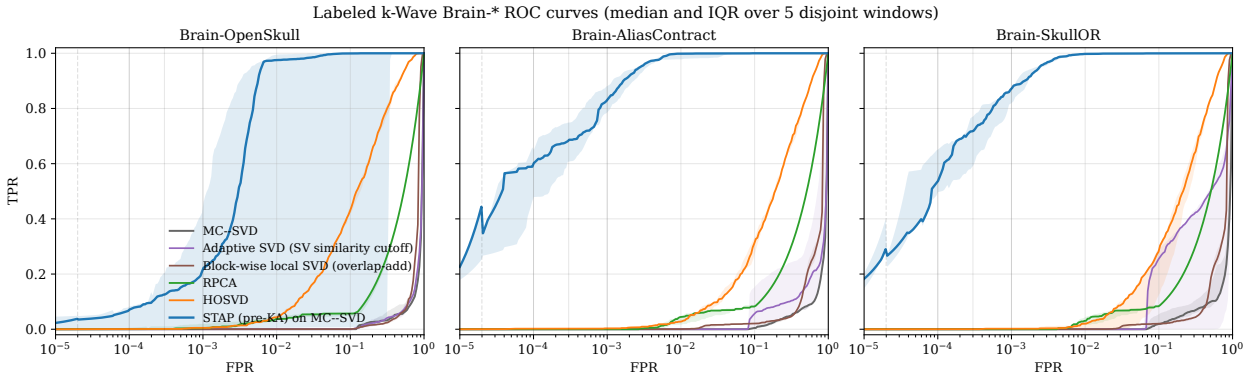


Figure 3: Brain-* k-Wave labeled regimes: ROC-style curves (TPR vs FPR on a log-x axis) under the same 64-frame window protocol used for the Brain-* low-FPR operating-point reporting in the main paper. Curves show the median and interquartile range over five disjoint windows. The vertical dashed line indicates the supported floor $fpr_{\min} = 1/n_{\text{neg}}$.

Missing table `reports/brain_crosswindow_calibration_table.tex`.
Reproduce it with `PYTHONPATH=.pythonscripts/brain_crosswindow_calibration.py--runs-rootruns/pilot/fair_filter_matrix_pd_r3_localbaselines`.

Table 2: Brain-* cross-window threshold calibration audit for the pre-KA STAP score. "Within-window" matches the main reporting protocol (calibrate $\tau_\alpha$ on a window's negatives and evaluate on that same window). "Cross-window" calibrates $\tau_\alpha$ on one window and evaluates on a disjoint window ($n = 20$ ordered window pairs). Reported values are median (IQR) over windows/pairs. Full outputs are written to `reports/brain_crosswindow_calibration.csv` / `reports/brain_crosswindow_calibration_summary.json`.

median and IQR across **windows**; the supported floor $fpr_{\min} = 1/n_{\text{neg}}$ is fixed by the window's negative-set size. For Gammex, thresholds are calibrated on pooled pixels for stability, while uncertainty is quantified via a frame-level bootstrap (resampling **cine frames** as the unit of variation at fixed pooled thresholds).

## B.2 Cross-window threshold calibration audit (Brain-*)

The main Brain-* results calibrate the operating threshold $\tau_\alpha$ on the negative set of the *same* 64-frame window being evaluated. To quantify how sensitive these strict low-FPR operating points are to window-to-window distribution shift (and to avoid over-interpreting quantized tail points), we perform a simple cross-window audit: for each regime and FPR target $\alpha$, we calibrate $\tau_\alpha$ on one window's negatives and apply it to the other windows. Table 2 reports within-window TPR (as in the main reporting protocol) alongside cross-window TPR and the realized cross-window FPR. **Checksum.** The within-window column in Table 2 matches the STAP operating-point values reported in the main paper build under the same score/mask protocol.

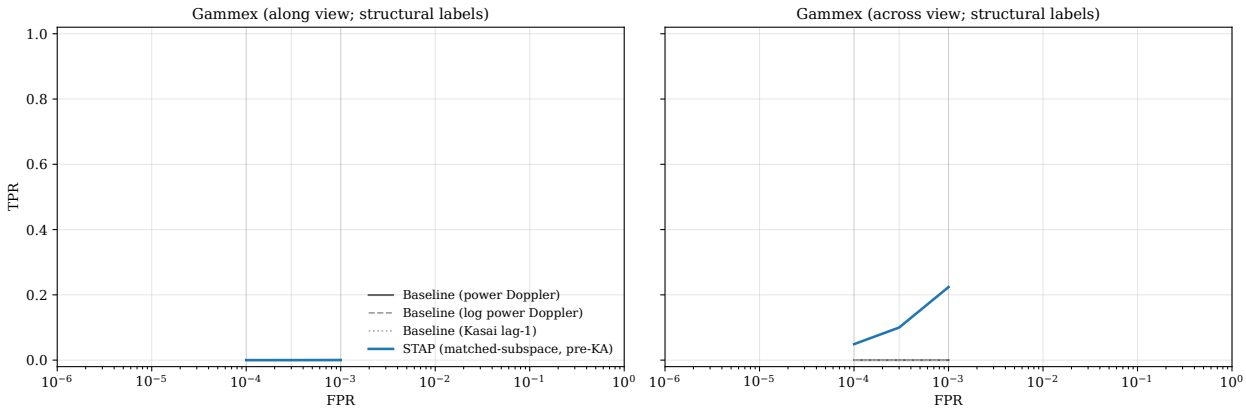


Figure 4: Gammex flow phantom structural labels: pooled ROC-style curves on along and across views, shown for baseline comparators and the pre-KA STAP matched-subspace score. The vertical guides indicate the strict low-FPR operating points reported in the main paper.

### B.3 Baseline sanity checks at relaxed operating points (Brain-*)

Several baseline rows in the main Brain-* strict low-FPR table take the value $\mathrm{TPR}(\alpha) \approx 0$ at $\alpha \leq 10^{-3}$. This can occur even when a baseline is not degenerate: if the background right tail is extreme, the background-calibrated threshold $\tau_\alpha$ can lie above essentially all $H_1$ pixels at strict-tail $\alpha$. To provide a reviewer-facing sanity anchor, Table 3 contrasts one strict-tail operating point ($\alpha = 10^{-3}$) with two relaxed operating points ($\alpha \in \{10^{-2}, 10^{-1}\}$) under the same window protocol and evaluation discipline; $\alpha = 10^{-2}$ is a moderate reference point and $\alpha = 10^{-1}$ is an intentionally relaxed non-degeneracy check.

Missing table `reports/brain_baseline_sanity_relaxed_table.tex`.
Reproduce it with `PYTHONPATH=.pythonscripts/brain_baseline_sanity_table.py`.

### B.4 Fixed-profile parameter sensitivity (Brain-OpenSkull)

Because the detector relies on covariance whitening, a natural concern is sensitivity to conditioning and geometry choices. We therefore run a simple 1D sweep on Brain-OpenSkull (the canonical k-Wave pilot) by varying one parameter at a time around the frozen profile: covariance diagonal loading $\lambda$ (conditioning strength), tile size (with stride chosen to preserve overlap), and slow-time aperture $L_t$. Each point reports within-window TPR medians (IQR) over five disjoint 64-frame windows at fixed strict-tail FPR targets $\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}\}$ under the same baseline and mask protocol as the main Brain-* tables. In this pilot, performance is effectively insensitive to $\lambda$ over the tested range, exhibits modest dependence on tile geometry, and is strongly sensitive to under-sized $L_t$ (e.g. $L_t = 6$ collapses strict-tail TPR), consistent with the need for adequate slow-time support to estimate and invert $R_i$.

### B.5 STAP-only ablation and PRF override sensitivity (Gammex)

To isolate the role of the STAP core from baseline prefiltering, we run a small STAP-only ablation on the Gammex flow phantom by applying STAP directly to raw (registered) CFM IQ without the

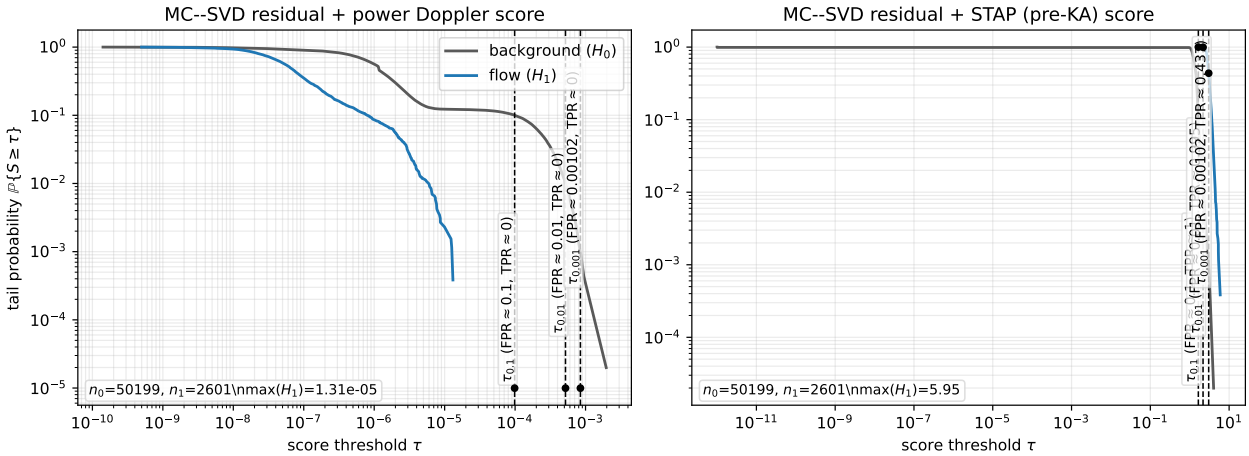


Figure 5: Representative Brain-OpenSkull window illustrating strict-tail collapse of PD-after-filter baselines. Curves show empirical tail probabilities $\mathbb{P}\{S \geq \tau\}$ for background ($H_0$) and flow ($H_1$) score distributions versus score threshold $\tau$, for (left) the baseline power Doppler score on the MC–SVD residual and (right) the pre-KA matched-subspace STAP score computed on the same residual. Vertical lines mark background-calibrated thresholds $\tau_\alpha$ at $\alpha \in \{10^{-1}, 10^{-2}, 10^{-3}\}$ and markers show the corresponding $\mathrm{TPR}(\alpha) = \mathbb{P}\{S \geq \tau_\alpha \mid H_1\}$. For the baseline PD score, $\tau_{10^{-3}}$ lies above $\max(H_1)$, yielding TPR $\approx 0$ by construction; the STAP score retains nonzero sensitivity at the same strict-tail operating point.

SVD band-pass baseline. We also provide a small PRF override sensitivity check on the along view to assess brittleness to the frozen PRF assumption required by Twinkling metadata limitations.

### B.6 Conditional execution leakage ablation (Brain-OpenSkull pilots)

Conditional execution (compute gating) skips STAP outside a flow-proxy region and falls back to the baseline score. Because this changes the overall decision rule, we evaluate it explicitly under leakage controls by varying only the *gating* mask: (i) a proxy mask computed from the same scored window, (ii) a proxy mask carried from a disjoint window, and (iii) a randomized mask with matched coverage. Table 4 reports the corresponding change in TPR relative to full STAP at fixed low-FPR targets.

This behavior is expected because gating changes the decision rule: leakage-controlled gating produces a small TPR decrease in exchange for a large runtime reduction, making it a clear engineering tradeoff rather than a free performance gain.

### B.7 KA hygiene on artifact-heavy calculi sequences (Twinkling)

To illustrate the shrink-only behavior of the KA layer in a strongly artifact-dominated regime, we evaluate Twinkling calculi sequences under a frozen short-ensemble profile. Figure 8 summarizes the KA prior outputs (region and reason) across Gammex and calculi runs, and Figure 9 shows a representative calculi example where the post-KA score compresses the background tail without increasing any scores on penalized pixels.

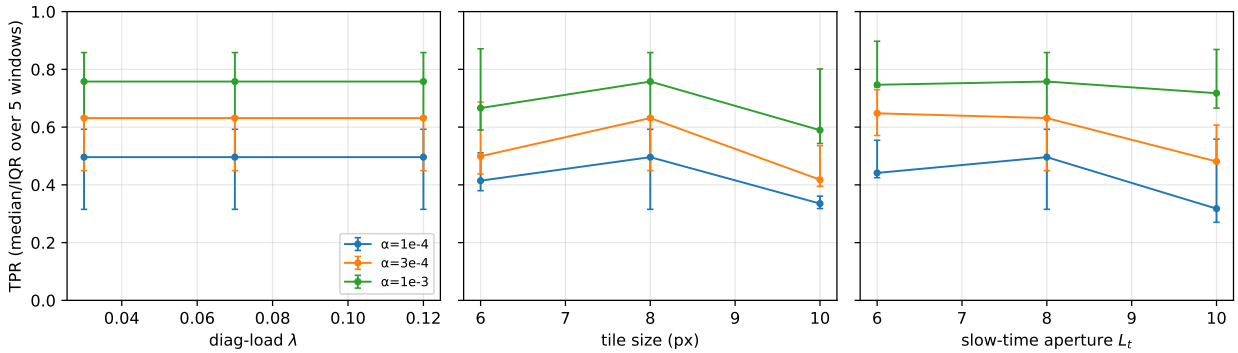


Figure 6: Brain-OpenSkull fixed-profile sensitivity sweep. Left: covariance diagonal loading $\lambda$ (conditioning). Middle: tile size (stride chosen to approximately preserve overlap fraction relative to the default $8 \times 8$/stride-3 profile). Right: slow-time aperture $L_t$. Points show within-window TPR medians with IQR error bars over five disjoint 64-frame windows at strict-tail operating points $\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}\}$.

| Conditional-execution mask | $\Delta$TPR@$10^{-4}$ ($\times 10^{-3}$) | $\Delta$TPR@$3\times10^{-4}$ ($\times 10^{-3}$) | $\Delta$TPR@$10^{-3}$ ($\times 10^{-3}$) |
|---|---|---|---|
| Same-window proxy mask | $+0.31\,[0.23, 0.39]$ | $+0.81\,[0.54, 1.04]$ | $+3.05\,[2.67, 3.43]$ |
| Disjoint-window proxy mask | $-3.05\,[-5.69, -0.92]$ | $-2.90\,[-5.34, -0.88]$ | $-1.74\,[-4.64, 0.28]$ |
| Random mask (matched coverage) | $-3.05\,[-5.65, -0.92]$ | $-2.97\,[-5.68, -0.71]$ | $-2.36\,[-4.96, -0.06]$ |

Table 4: Conditional-execution leakage ablation on Brain-OpenSkull pilots (five independent realizations; 64-frame window; matched-subspace STAP score). Entries report mean $\Delta$TPR (conditional minus full STAP) at fixed FPR targets, with 95% bootstrap CIs over realizations ($n = 5$). This table provides uncertainty details for the conditional-execution tradeoff summarized in the main Results. Evaluation masks are identical across conditions; only the conditional-execution gating mask varies.

### B.8 ULM 7883227: additional motion variants

In addition to the main `brainlike` motion sweep reported in the main paper, we report an `elastic` (nonrigid-only) motion ladder under the same frozen baseline and STAP configuration.

### B.9 Macé whole-brain fUS: hemodynamic band-energy statistics and hold-out check

We treat the Macé/Urban whole-brain dataset as a PD-only, open-skull analogue of Brain-OpenSkull and compute tile-level hemodynamic band-energy statistics with fixed bands $P_f = [0, 0.5]$ Hz, guard $[0.5, 1.0]$ Hz, and $P_a = [1.0, 3.0]$ Hz on $8 \times 8$ tiles (stride 3). Offline we also compute atlas-derived *region strata* (e.g. VIS/SC/LGd versus CP/CA/DG) for descriptive summaries; these strata are not treated as ground-truth flow labels and are not used as KA inputs. Figure 11 summarizes a systematic sweep in "Pf occupancy vs alias separation" space. We provide an explicit atlas-alignment sanity check in Appendix F. Here PD-z denotes a per-tile change-from-baseline

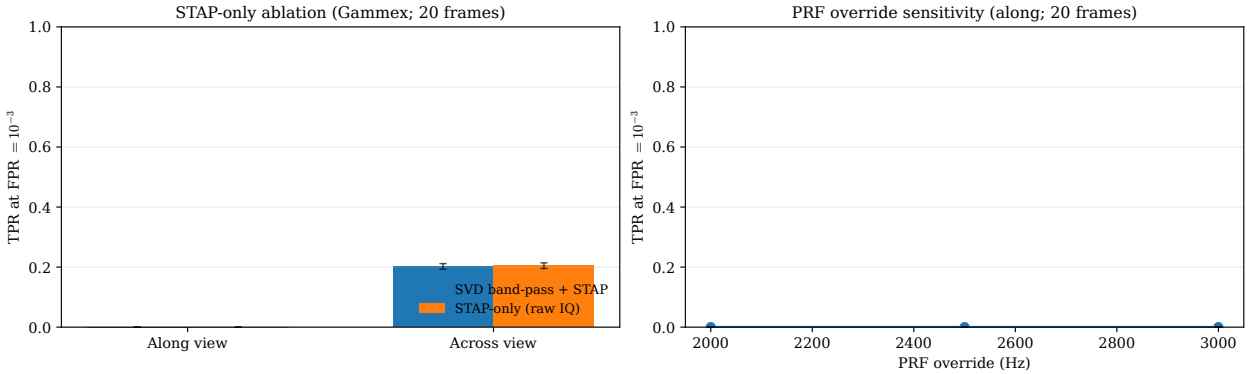


Figure 7: Gammex flow phantom: (left) STAP-only ablation comparing the standard baseline SVD band-pass + STAP pipeline to STAP-only on raw IQ (first 20 cine frames; both views). (right) PRF override sensitivity on the along view (first 20 cine frames). Points show TPR at a fixed FPR target of $10^{-3}$ with 95% frame-bootstrap CIs under the structural tube masks. This is a low-cost sanity audit of the frozen-PRF band discretization (not tuned to improve ROC); PRF is an acquisition constant for a sequence family and band-sanity sentinels are deterministic functions of $(N, \mathrm{PRF})$. Full numeric outputs are in `reports/twinkling_gammex_ablation_prf_sensitivity.csv`.

z-score on the PD time series (distinct from the Doppler/IQ PD-mode score maps $S_{\mathrm{PD}}$ exported by the STAP pipeline).

**Hold-out discipline (avoid plane selection bias and label-tuned thresholds).** To address the selection-bias critique and to avoid per-plane PD-z thresholds chosen using atlas strata (e.g. setting thresholds to force a fixed activation fraction in a preferred stratum), we add a scan-level hold-out evaluation (reproducibility details in Appendix L). We (i) define label-free proxy sets from a mean-PD vascularity mask and set a per-plane PD-z operating point by a fixed background-proxy tail rule $\mathbb{P}\{\text{PD-}z \geq \tau \mid \mathcal{T}_{\mathrm{bg}}\} = \alpha_{\mathrm{pd}}$ (no atlas labels), (ii) define the alias suppression threshold as a fixed background-proxy quantile of $m_{\mathrm{alias}} = \log((E_{\mathrm{Pa}} + \epsilon)/(E_{\mathrm{Pf}} + \epsilon))$, and (iii) select Pf-rich planes in held-out scans using a label-free band-energy statistic (Pf-peak fraction on flow-proxy tiles), with the selection threshold fixed from the training scans. Table 5 summarizes the resulting held-out counts on `scan4`–`scan5` (training: `scan1/2` and `scan3/6`; note `scan2` duplicates `scan1` and `scan6` duplicates `scan3` in this loader).

On held-out scans (`scan4`–`scan5`), this fixed, shrink-only alias suppression reduces proxy activations in the CP/CA/DG stratum while modestly affecting activations in the VIS/SC/LGd stratum at the chosen label-free operating point. At $\alpha_{\mathrm{pd}} = 0.01$ on the background proxy, aggregate (VIS/SC/LGd / CP/CA/DG hit counts) change from 23/142 to 22/95 (VIS/SC/LGd retention $\approx 0.96$). On the Pf-peak-selected subset of held-out planes ($\mathrm{PfPeak}_{\mathrm{flow}} \geq \tau_{\mathrm{sel}}$), hit counts change from 19/22 to 19/19 (VIS/SC/LGd retention = 1.0). We interpret this as a conservative safety-regularization behavior rather than an efficacy claim: Macé is PD-only and provides no Doppler-side flow labels, so its primary role here is to validate the band-energy/proxy machinery and the no-peeking threshold discipline.

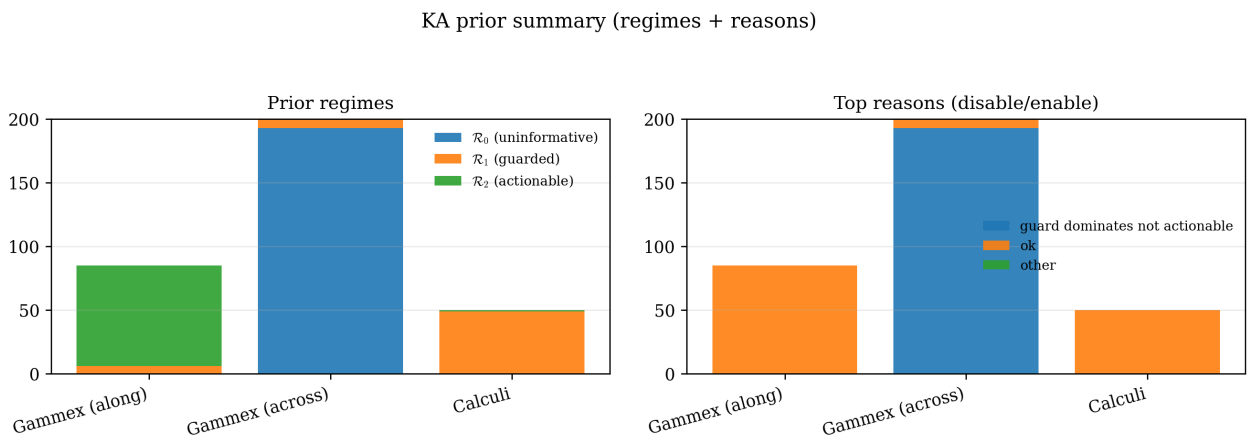


Figure 8: Twinkling KA-prior summary panel: prior (region, reason) distributions across Gammex (structural-label) and calculi (artifact-heavy) runs. Reproducibility details are provided in Appendix L.

| Held-out subset (scan4–scan5) | $n$ | offline VIS/SC/ LGd / CP/CA/DG hits (pre) | offline VIS/SC/ LGd / CP/CA/DG hits (post) | VIS/SC/ LGd retention | CP/CA/DG reduction |
|---|---|---|---|---|---|
| All planes | 40 | 23/142 | 22/95 | 0.957 | 47 |
| Selected ($\text{PfPeak}_{\text{flow}} \geq \tau_{\text{sel}}$) | 5 | 19/22 | 19/19 | 1.000 | 3 |

Table 5: Macé hold-out alias-gate evaluation with label-free thresholding. For each plane we set the PD-z threshold $\tau$ by a fixed background-proxy tail rule $\mathbb{P}\{\text{PD-}z \geq \tau \mid \mathcal{T}_{\text{bg}}\} = \alpha_{\text{pd}}$ with $\alpha_{\text{pd}} = 0.01$, and set the alias suppression threshold as the 0.8 quantile of $m_{\text{alias}} = \log((E_{\text{Pa}} + \epsilon)/(E_{\text{Pf}} + \epsilon))$ on the same background proxy set. "Selected" planes are those whose Pf-peak fraction on flow-proxy tiles exceeds a training-derived threshold $\tau_{\text{sel}}$ (the 0.95 quantile of $\text{PfPeak}_{\text{flow}}$ over the training scans `scan1/2` and `scan3/6`; $\tau_{\text{sel}} \approx 0.145$). Hit counts use atlas strata (VIS/SC/LGd and CP/CA/DG) only for offline descriptive evaluation (not KA inputs and not a Doppler efficacy claim). Reproducibility details are provided in Appendix L.

**Pixel-level separability vs. the vascular atlas (independent structural labels).** To add one real-data quantitative check with an *independent* label source (not derived from PD), we map the dataset-provided Allen atlas vascular volume `atlas.Vascular` into each Macé scan plane using the affine in `Transformation.mat`. We threshold `atlas.Vascular` at 0.5 to define vascular vs. non-vascular pixels and restrict evaluation to pixels that map inside the atlas and to non-background atlas regions. This yields $n_{\text{neg}} \approx 1.7 \times 10^4$ non-vascular pixels per plane (median $fpr_{\min} \approx 5.9 \times 10^{-5}$), so FPR targets down to $10^{-4}$ are empirically meaningful at pixel level. As expected for PD-only hemodynamics, extreme-tail separability is weak; the purpose of this check is independent-label auditability and an empirical FPR scale rather than a performance headline.

We then evaluate three families of scalar maps: (i) a structural baseline based on the mean PD image (reported as a right-tail score using a left-tail convention, $S_{\text{inv_meanPD}} = -\log(\overline{P}+\epsilon)$), (ii) PD-z as a per-pixel change-from-baseline z-score on log-PD (the maximum z-score after an early baseline segment), and (iii) a hemodynamic alias band-energy statistic $m_{\text{alias}} = \log((E_a + \epsilon)/(E_f + \epsilon))$ computed from simple FFT band energies on log-PD with fixed hemodynamic bands $P_f = [0, 0.5]$ Hz

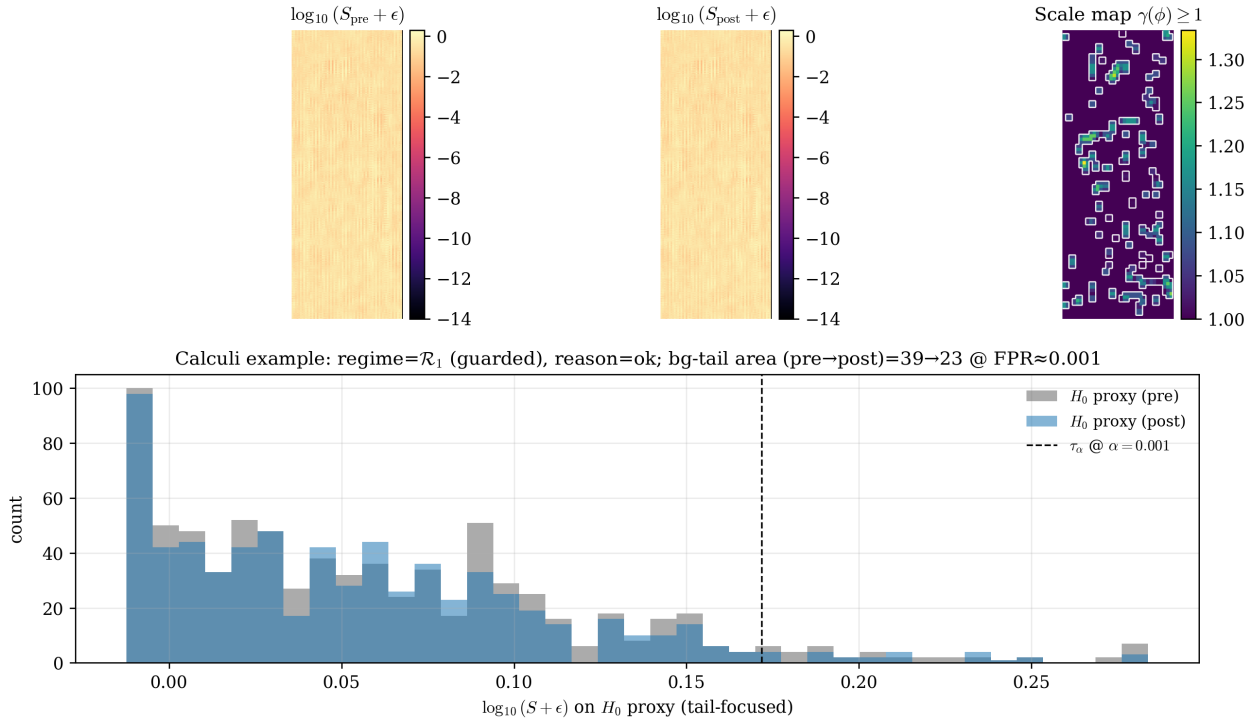


Figure 9: Representative calculi tail example (pre/post KA) showing shrink-only tail hygiene behavior when the KA prior is active ($w < 1$). Reproducibility details are provided in Appendix L.

and $P_a = [1.0, 3.0]$ Hz. Finally, to mirror the conservative posture on real data, we apply a label-free alias penalty as a shrink-only score-space regularizer on PD-z: protect a flow-proxy set defined by a high mean-PD quantile, then down-weight non-proxy pixels whose $m_{\text{alias}}$ lies above the 0.90 quantile of $m_{\text{alias}}$ on the background proxy. Table 6 summarizes the resulting low-FPR operating points.

**PD-only prior summary panel.** To keep PD-only Macé reporting aligned with the same audit posture used for Doppler IQ, we summarize the PD-only KA prior outcomes over all planes using a compact summary panel that mirrors the Doppler-IQ summary but operates per plane. The panel reports (region, reason) histograms and shrink-only coverage/strength band-energy summaries under label-free proxy sets (Appendix G), and presents atlas-stratified summaries only as a clearly separated offline/descriptive panel.

### B.10 Shin RatBrain Fig3 (LOCA-ULM): real kHz IQ viability and KA-prior diagnostics

To demonstrate that the frozen STAP + band-energy + KA-prior stack is operational on real Doppler-like IQ, we use the LOCA-ULM RatBrain Fig3 dataset released with Shin et al. as beamformed, post-compounded complex IQ sequences [34, 35]. This is a microbubble ULM IQ dataset; we use it solely as a kHz-IQ ingestion and band-energy/prior audit anchor, not as a physiological fUS benchmark. The Fig3 archive contains 80 clips (`IQData001.dat`–`IQData080.dat`) and a

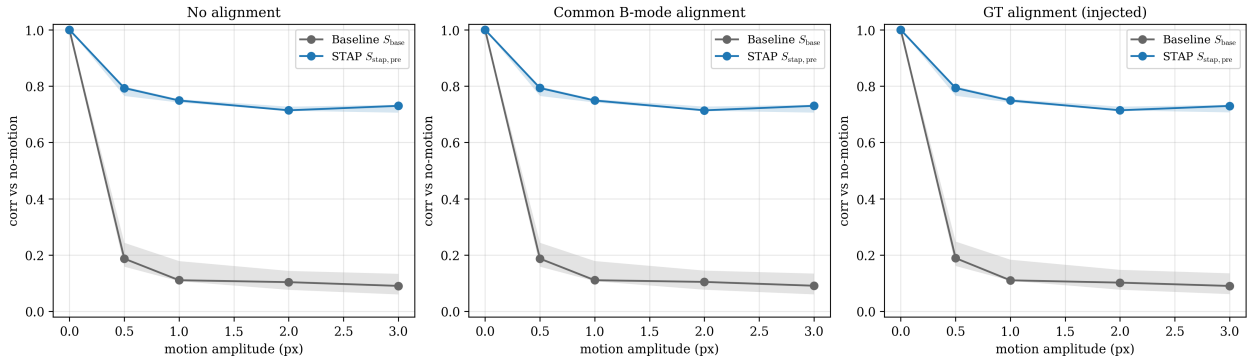


Figure 10: ULM 7883227 motion sweep under `elastic` (nonrigid-only) motion with the same frozen baseline/profile used in the main paper (`blocks 1--3`, frames 0:128, baseline MC–SVD with frozen energy fraction $e = 0.975$). Motion is injected at the beamformed IQ stage as a robustness stress test; it does not model channel-domain decorrelation physics. Reproducibility details are provided in Appendix L.

| Score (vascular vs. non-vascular) | TPR@$10^{-4}$ ($\times 10^{-3}$) | TPR@$3\times 10^{-4}$ ($\times 10^{-3}$) | TPR@$10^{-3}$ ($\times 10^{-3}$) |
|---|---|---|---|
| $S_{\text{inv_meanPD}} = -\log(\overline{P} + \epsilon)$ (structural baseline) | 0.00 (0.00, 0.00) | 0.00 (0.00, 1.04) | 2.76 (1.84, 3.20) |
| PD-z (log-PD; max z after baseline) | 0.00 (0.00, 0.00) | 0.00 (0.00, 0.81) | 0.94 (0.00, 1.78) |
| $m_{\text{alias}} = \log((E_a + \epsilon)/(E_f + \epsilon))$ (hemodynamic alias statistic) | 0.00 (0.00, 0.00) | 0.00 (0.00, 0.75) | 1.05 (0.00, 2.04) |
| PD-z + label-free alias penalty (shrink-only) | 0.00 (0.00, 0.00) | 0.00 (0.00, 0.81) | 0.92 (0.00, 1.74) |

Table 6: Macé pixel-level quantitative evaluation using an independent vascular atlas label source (`atlas.Vascular`; $n = 80$ planes from four deduplicated scans). Entries report median (IQR) TPR at fixed FPR targets for vascular vs. non-vascular pixels. The median negative-pixel count per plane is $\approx 1.69 \times 10^4$ (median $fpr_{\min} \approx 5.9 \times 10^{-5}$). Reproducibility details are provided in Appendix L.

`SizeInfo.dat` file describing the tensor shape. Each clip stores $T = 250$ slow-time frames on a $(238 \times 130)$ spatial grid as little-endian float64 arrays containing I then Q in column-major order (matching the authors' reference MATLAB loader); the effective slow-time sampling is 1 kHz (post-compounding), with center frequency $f_0 \approx 15.6$ MHz [35]. We treat this dataset as a *real-IQ viability and safety anchor* rather than a labeled detection benchmark: there are no $H_1/H_0$ labels, so we do not report ROC uplift; instead we report prior-regime stability (tilewise band-energy summaries and coverage), shrink-only invariants, and label-free endpoints defined on pixel-level proxy masks (flow/background).

**Processing workflow.** We mirror the authors' loader and convert IQ clips into a common analysis format used throughout this work (reproduction details in Appendix L). For a chosen clip and time window we:

- Apply an SVD clutter filter on the Casorati matrix $X \in \mathbb{C}^{T\times HW}$ to form a baseline PD-

derived map $pd_{\text{base}}$ and a residual IQ cube (the input to STAP).

- Construct pixel-level proxy masks from $pd_{\text{base}}$ (used here as a conventional PD-brightness proxy $\text{PD}^{\uparrow}$ distinct from the ROC score maps): a flow/vessel proxy (top quantile; minimum pixel count) and a background proxy (complement of the flow mask in a central crop).
- Run the temporal STAP core on the residual IQ cube with fixed tile geometry ($8 \times 8$ tiles, stride 3) and a longer slow-time aperture $L_t = 64$ suited to 1 kHz slow time (bin spacing 15.6 Hz), producing a pre-KA PD-mode score map.
- Apply the KA spatial-spectral prior (described in the main paper) to produce a region index, reason label, and (when active) a non-increasing post-KA score map together with gate/scale maps.
- Record tilewise band-energy statistics (peak frequency, guard fraction, alias/guard risk metrics) and the penalization maps in the same analysis format used throughout this work.

The proxy masks are used only for label-free proxying/protection; prior-regime decisions and hygiene/robustness proxies follow the same right-tail score convention used elsewhere in this manuscript: operating points are expressed as a threshold on a right-tail PD score map, i.e. $S_{\text{PD}} \geq \tau$ at fixed background-proxy tail rate.

*Scale-map convention.* When the KA prior is active ($w < 1$ on a nontrivial set), we record an equivalent per-pixel scale factor $\gamma(i) \geq 1$ and apply it to the PD score map as

$$\text{PD}_{\text{post}}(i) = \text{PD}_{\text{pre}}(i)/\gamma(i), \qquad \gamma(i) \geq 1,$$

which makes the PD score map non-increasing on penalized regions. For any fixed threshold $\tau$ this is a pure shrink-only penalty; when thresholds are recalibrated post-KA to match a target background tail rate, any TPR change reflects background-tail suppression rather than score inflation.

**Frozen Shin operating profiles and baseline calibration.** To avoid retuning the conservative brain Doppler profile to Shin until outcomes appear favorable, we treat Shin as its own frozen operating regime and explicitly calibrate only the baseline clutter filter once. We sweep the MC–SVD energy-fraction cutoff (smallest number of singular modes removed to exceed the target energy fraction) over a calibration subset (10 representative clips; window 0:128) and select a single frozen baseline setting based on band-energy integrity and stability, not on prior-regime counts. Table 7 reports $\mathcal{R}_1/\mathcal{R}_2$ counts for transparency, but selection is based on the label-free integrity/actionability metrics ($\Delta_{bf}$, $\Delta_{\text{tail}}$) and stability/hygiene proxies. Overly aggressive baseline removal increases Pf peak occupancy but can invert actionability evidence (background-vs-flow separation becomes negative) and suppress $\mathcal{R}_2$ eligibility. We therefore freeze MC–SVD with energy fraction $\approx 0.97$ as a balanced setting for the Shin-U profile (reproducibility details in Appendix L).

**Baseline comparisons (proxy separability on real IQ).** Although Shin provides no independent $H_1/H_0$ labels, it is useful to test whether the STAP core improves *proxy* separation on real complex IQ. We run a fixed scenario matrix on 10 clips and three overlapping windows (0:128, 64:192, 122:250), exporting right-tail score maps for MC–SVD, RPCA, HOSVD, STAP (on the MC–SVD residual), and an *STAP-only* ablation that runs STAP directly on raw/registered IQ. Proxy masks are derived deterministically from the MC–SVD baseline PD map via a `pd_auto` quantile rule (0.995 quantile within depth fraction $[0.25, 0.85]$ with dilation), with the generic default-mask union disabled; masks are then held fixed across methods. We report (i) AUC between flow-proxy and background-proxy score distributions (threshold-free), and (ii) flow-proxy hit rate at matched

| Energy frac | $\mathcal{R}_1/\mathcal{R}_2$ | $\Delta_{bf}$ med | $\Delta_{\text{tail}}$ med | Pf-peak(flow) med | BG clusters $\Delta$ | BG area $\Delta$ |
|---|---|---|---|---|---|---|
| 0.90 | 8/2 | +1.52 | +0.65 | 0.011 | −17 | −32 |
| 0.95 | 8/2 | +1.50 | +0.65 | 0.019 | −17 | −32 |
| 0.97 | 8/2 | +1.49 | +0.73 | 0.050 | −19 | −45 |
| 0.975 | 9/1 | −0.38 | +0.77 | 0.237 | −30 | −89 |
| 0.98 | 10/0 | −0.38 | +0.78 | 0.253 | −30 | −89 |
| 0.99 | 10/0 | −0.47 | +0.58 | 0.510 | −6 | −59 |

Table 7: Baseline clutter-filter calibration for the frozen Shin-U profile (10 clips; window 0:128). $\Delta_{bf}$ is the median background-vs-flow proxy separation of the risk metric; $\Delta_{\text{tail}}$ is the background tail association (actionability) metric; Pf-peak(flow) is the fraction of flow-proxy tiles (tiles with high flow-mask coverage) whose PSD peak lies in Pf. BG cluster/area deltas are computed from a fixed background-proxy tail operating point on the PD score map $S_{\text{PD}}$ (clusters/area after KA minus before KA). We freeze energy fraction $\approx 0.97$ because it yields nontrivial Pf realization while preserving positive actionability.

background tail rates $\alpha \in \{10^{-1}, 10^{-2}, 3\times10^{-3}, 10^{-3}, 3\times10^{-4}, 10^{-4}\}$ (Figure 13; Table 8). In this setup $n_{\text{bg}} \approx 2.6\times10^4$ per scenario ($fpr_{\min} \approx 3.8\times10^{-5}$), so $\alpha = 10^{-4}$ corresponds to roughly 2–3 background pixels.

Across scenarios, STAP improves proxy separation (median AUC 0.951 vs 0.922 for MC–SVD) and yields nonzero flow-proxy hit rates at strict tails where MC–SVD is typically zero (e.g. median hit rate 0.070 at $\alpha = 10^{-3}$). The STAP-only ablation is comparable to STAP-on-residual, indicating that the localized STAP core is the primary driver of the improvement on this dataset. At moderate strictness ($\alpha \approx 10^{-2}$), RPCA/HOSVD can exhibit higher proxy hit rates than STAP despite lower AUC, consistent with a broader, lower-contrast score elevation rather than strict-tail selectivity. Finally, a conditioning-target sweep ($\kappa_{\text{target}} = 80$ vs 200) produced identical STAP score maps under this frozen profile, indicating that the conditioning target is non-binding in this particular configuration. Figure 15 shows a representative qualitative comparison.

**Measured rigid-motion telemetry.** Because Shin clips are short ($\approx 0.25$ s) and there are no external motion sensors, we treat rigid registration telemetry as a descriptive motion proxy rather than a full motion benchmark. Figure 14 summarizes the phase-correlation shifts estimated during baseline preprocessing (RMS shift magnitude per scenario) and relates them to the proxy AUC uplift (STAP minus MC–SVD). In these windows the estimated rigid motion is typically subpixel, so we do not treat Shin as a stratified motion-stress regime; the proxy-separation gains are not explained by measured rigid shifts.

**Proxy circularity audit.** To reduce dependence on any single baseline-derived proxy mask, we report metrics under (i) a flow-proxy mask derived from MC–SVD ($pd_{\text{base}}$ via `pd_auto`) and (ii) a consensus proxy mask formed by majority vote over MC–SVD/RPCA/HOSVD `pd_auto` masks; rankings are consistent under this cross-mask audit.

**Cross-mask audit (proxy-label sensitivity).** Table 8 includes both the baseline-derived mask and the consensus mask results; full cross-mask outputs (including per-scenario thresholds and realized FPRs) are in `reports/shin_ratbrain_crossmask_audit_shinU_consensus3.csv`.

| | MC–SVD proxy mask | | | Consensus proxy mask | | |
|---|---|---|---|---|---|---|
| Method | AUC (med [IQR]) | hit@$\alpha = 10^{-3}$ (med [IQR]) | hit@$\alpha = 10^{-4}$ (med [IQR]) | AUC (med [IQR]) | hit@$\alpha = 10^{-3}$ (med [IQR]) | hit@$\alpha = 10^{-4}$ (med [IQR]) |
| MC–SVD | 0.922 [0.916, 0.934] | 0.000 [0.000, 0.023] | 0.000 [0.000, 0.000] | 0.913 [0.908, 0.924] | 0.000 [0.000, 0.029] | 0.000 [0.000, 0.000] |
| RPCA | 0.902 [0.893, 0.907] | 0.024 [0.000, 0.031] | 0.008 [0.000, 0.016] | 0.909 [0.898, 0.915] | 0.028 [0.000, 0.036] | 0.010 [0.000, 0.020] |
| HOSVD | 0.881 [0.873, 0.906] | 0.040 [0.030, 0.056] | 0.008 [0.000, 0.015] | 0.883 [0.874, 0.909] | 0.055 [0.036, 0.069] | 0.010 [0.000, 0.019] |
| STAP | 0.951 [0.948, 0.956] | 0.070 [0.034, 0.081] | 0.014 [0.006, 0.025] | 0.944 [0.932, 0.947] | 0.051 [0.018, 0.082] | 0.009 [0.003, 0.032] |
| STAP-only | 0.948 [0.944, 0.951] | 0.065 [0.024, 0.078] | 0.020 [0.003, 0.032] | 0.939 [0.933, 0.944] | 0.065 [0.012, 0.083] | 0.021 [0.000, 0.033] |

Table 8: Shin RatBrain Fig3 proxy-metric summary over 30 scenarios (10 clips $\times$ 3 windows) under the frozen Shin-U profile (PRF = 1 kHz, $L_t = 64$, MC–SVD energy fraction $\approx 0.97$). AUC is computed between flow-proxy and background-proxy score distributions; hit@$\alpha$ is the flow-proxy hit rate when the threshold is chosen to achieve background tail rate $\alpha$. Metrics are shown under two proxy-mask definitions: an MC–SVD-derived mask and a consensus mask (majority vote over MC–SVD/RPCA/HOSVD masks). Full per-scenario outputs are in `reports/shin_ratbrain_baseline_matrix_shinU_e970_Lt64_nomaskunion_k80.csv` and `reports/shin_ratbrain_crossmask_audit_shinU_consensus3.csv`.

**Profile variants (Shin-U vs Shin-S/L).** In addition to Shin-U, we evaluated two alternative frozen band profiles (Shin-S and Shin-L) designed to push the "flow" band Pf toward lower frequencies and increase Pf peak occupancy (an interpretability-forward choice for microvascular-like content under 1 kHz slow time). Across the same 10-clip calibration subset, these variants typically increase Pf-peak(flow) but render the contract actionability check $\Delta_{\text{tail}}$ negative under our flow/background proxy sets, so the prior lies predominantly in $\mathcal{R}_1$ (safety-only) rather than $\mathcal{R}_2$ (actionable). We therefore treat Shin-S/L as conservative "deployment-mode" profiles ($\mathcal{R}_1$-only by design) and use Shin-U as the primary "real IQ viability + safety" anchor reported here.

**KA behavior, stability, and hygiene endpoints.** Under the frozen Shin-U profile (MC–SVD energy fraction $\approx 0.97$, STAP $L_t = 64$, fixed Pf/Pa band definitions, and KA enabled in `auto` mode), we run the pipeline on 10 pre-selected clips (`IQData001`–`IQData005`, `IQData010`, `IQData020`, `IQData040`, `IQData060`, `IQData080`) and evaluate stability across three overlapping time windows (0:128, 64:192, 122:250). Under Shin-U, Pf realization is weak (Pf-peak(flow)$\ll p_{f,\min}$), so Shin is not treated as a KA-uplift feasibility case under R–1; we use it to validate end-to-end determinism, invariants, and robustness proxies. Because there are no labels, we report only label-free endpoints (reproducibility details in Appendix L):

- **KA regime stability.** $\mathcal{R}_1$ dominates in all windows (8–10 of 10 clips), while $\mathcal{R}_2$ is rare (0–2 clips) due to the Pf-realization eligibility condition for $\mathcal{R}_2$ based on flow-proxy tiles (tiles with high flow-mask coverage); all decisions emit auditable reason labels.
- **Shrink-only and protected invariance.** When KA is active it applies a mild, localized shrink (median $p_{\text{shrink}} \approx 0.09$; median scaled pixel fraction $\approx 0.20$) and preserves the protected set exactly (maximum absolute PD change on protected pixels is 0 in all evaluated clips).
- **Background-tail hygiene.** A label-free background-tail metric shows consistent tail compression on early windows: for each clip/window we choose a threshold $\tau$ so that the background proxy satisfies $\mathbb{P}\{S_{\text{PD}} \geq \tau \mid \text{bg proxy}\} = 10^{-3}$ on the pre-KA map, then count clusters/area of pixels satisfying $S_{\text{PD}} \geq \tau$ before vs. after KA. Total background-tail clusters decrease from 125 $\rightarrow$ 106 (window 0:128) and 121 $\rightarrow$ 104 (window 64:192), while the late window is closer to neutral (129 $\rightarrow$ 124). This is interpreted as "map hygiene" rather than ROC uplift.

Table 9 summarizes prior-regime counts and key band-energy summaries across windows.

| Window | $\mathcal{R}_0/\mathcal{R}_1/\mathcal{R}_2$ | Pf-peak(flow) med | guard q90 med | $p_{\text{shrink}}$ med | scaled-px frac med | scale p90 ($\gamma$) med |
|---|---|---|---|---|---|---|
| 0:128 | 0/8/2 | 0.050 | 0.104 | 0.091 | 0.201 | 1.167 |
| 64:192 | 0/10/0 | 0.032 | 0.110 | 0.097 | 0.198 | 1.168 |
| 122:250 | 0/9/1 | 0.039 | 0.108 | 0.090 | 0.205 | 1.167 |

Table 9: Shin RatBrain Fig3 prior-regime summary band-energy statistics for the frozen Shin-U profile ($e \approx 0.97$, $L_t = 64$) across three overlapping windows (10 clips each).

| Window | background clusters (pre→post) | background area (pre→post) |
|---|---|---|
| 0:128 | $125 \to 106$ | $263 \to 218$ |
| 64:192 | $121 \to 104$ | $264 \to 212$ |
| 122:250 | $129 \to 124$ | $264 \to 257$ |

Table 10: Shin RatBrain Fig3 label-free hygiene endpoints under Shin-U ($e \approx 0.97$; $L_t = 64$; 10 clips). "Pre" and "post" denote STAP PD-mode outputs before and after the non-increasing KA prior penalty. The background-tail metric targets flow-like tail activations under the right-tail score convention: for each clip/window instance we choose $\tau$ so that the background proxy satisfies $\mathbb{P}\{S_{\text{PD}} \geq \tau \mid \text{bg proxy}\} = 10^{-3}$ on the corresponding pre-KA map, then count connected components (clusters) and pixels (area) with $S_{\text{PD}} \geq \tau$. Reproducibility details are provided in Appendix L.

**All-clips KA regime summary (n=80).** To reduce dependence on the 10-clip subset, we also run a diagnostics-only sweep over all 80 clips (window 0:128), recording the prior regime outputs. In this sweep we skip the STAP pass and do not apply the score-space KA mapping; we treat the result as an activation-rate audit (region/reason frequencies and band-energy summary statistics), not a performance result. On all clips the prior lies in $\mathcal{R}_1$ for 73/80 and in $\mathcal{R}_2$ for 7/80 (all with `reason=ok`, risk mode `alias`); median Pf-peak(flow)$\approx$ 0.091, guard $q_{0.90} \approx 0.105$, and median predicted $p_{\text{shrink}} \approx 0.095$. Reproducibility details are provided in Appendix L.

**Synthetic motion robustness sweeps (label-free).** To probe motion robustness on real IQ without ground-truth labels, we implement controlled motion-injection sweeps on Shin IQ cubes (reproduction details in Appendix L). In addition to simple per-frame subpixel translations, global phase modulation, and spatially smooth low-frequency clutter tones, we define a *brain-like* motion model designed to be mechanistically comparable to Brain-SkullOR: a bulk rigid translation (correctable by phase-correlation registration) plus a smooth, depth-decaying elastic warp with temporally correlated coefficients (not removable by translation-only registration). Motion is applied directly to the beamformed IQ cube (a stress test rather than a full scatterer-level simulator), after which we rerun the same baseline and STAP pipeline (KA disabled to isolate STAP).

We report two label-free robustness endpoints as functions of motion amplitude: (i) PD-mode score-map correlation versus the no-motion reference (after rigid alignment and a boundary crop), and (ii) a flow-proxy hit rate at a matched background-proxy tail operating point (background-proxy tail rate = $10^{-3}$), expressed in score space as $\mathbb{P}\{S_{\text{PD}} \geq \tau \mid \text{bg proxy}\} = 10^{-3}$. Concretely, for each clip/window we define a fixed pixel-level flow-proxy mask from the no-motion baseline PD-derived map $pd_{\text{base}}$ (top quantile; minimum pixel count) and a background-proxy mask from its complement; for each run/method we choose $\tau$ on that run's background proxy and then report the fraction of flow-proxy pixels satisfying $S_{\text{PD}} \geq \tau$. As a degeneracy check (to rule out "stability by collapse"), the sweep also records non-collapse diagnostics (map std/entropy and background

| Window | $\mathrm{corr}_{\mathrm{self}}$ base | $\mathrm{corr}_{\mathrm{self}}$ STAP (pre-KA) | frac(STAP>base) $\mathrm{corr}_{\mathrm{self}}$ | hit $\mathrm{rate}_{\mathrm{flow}}$ base @ bg tail = $10^{-3}$ | hit $\mathrm{rate}_{\mathrm{flow}}$ STAP @ bg tail = $10^{-3}$ | frac(STAP>base) hit $\mathrm{rate}_{\mathrm{flow}}$ |
|---|---|---|---|---|---|---|
| 0:128 | 0.188 | 0.454 | 1.0 | 0.000 | 0.041 | 1.0 |
| 64:192 | 0.186 | 0.399 | 0.9 | 0.000 | 0.040 | 1.0 |
| 122:250 | 0.182 | 0.364 | 0.8 | 0.000 | 0.041 | 1.0 |

Table 11: Shin RatBrain Fig3 motion-sweep endpoints at 2 px amplitude (median over 10 clips) under the frozen Shin-U profile, with rigid registration enabled and KA disabled (STAP reported pre-KA). $\mathrm{corr}_{\mathrm{self}}$ is Pearson correlation between each method's PD-mode score map at the given motion amplitude and its own no-motion reference, after phase-correlation alignment and a boundary crop. hit $\mathrm{rate}_{\mathrm{flow}}$ is computed on a fixed, method-independent flow-proxy mask derived from the no-motion baseline PD-derived map $pd_{\mathrm{base}}$: for each run/method we choose $\tau$ so that the background proxy achieves a tail rate $10^{-3}$ under the right-tail score rule $\mathbb{P}\{S_{\mathrm{PD}} \geq \tau \mid \text{bg proxy}\} = 10^{-3}$, then evaluate the fraction of flow-proxy pixels satisfying $S_{\mathrm{PD}} \geq \tau$. frac(STAP>base) reports the paired fraction of clips where STAP exceeds MC–SVD at 2 px on the corresponding endpoint.

variance ratios). A post-hoc sensitivity sweep recomputes correlation under different crop margins and with alignment disabled. Over 10 pre-selected clips and three overlapping time windows (0:128, 64:192, 122:250) under the frozen Shin-U profile (PRF = 1 kHz, $L_t = 64$, baseline MC–SVD with rigid registration enabled), STAP PD-mode score maps are consistently more stable than MC–SVD as motion increases: at a 2 px motion amplitude, median correlation improves from $\approx 0.18$–$0.19$ (MC–SVD) to $\approx 0.36$–$0.45$ (STAP) across windows, and the flow-proxy hit rate improves from 0 to $\approx 0.04$ at bg tail rate $= 10^{-3}$. Table 11 summarizes these endpoints; Figures 16–17 show motion curves.

Figure 16 shows the canonical window (0:128) motion curves, while Figure 17 shows the same endpoints across all three overlapping windows; Figure 18 provides a rigid-registration ablation (`reg0`).

Figure 19 shows the corresponding KA prior summary panel for the frozen Shin-U profile. Figure 20 shows a representative per-clip multipanel view separating the baseline PD-derived map, the STAP PD-mode score map pre-KA, the KA gate/scale, and the final PD-mode score map post-KA, along with key band-energy maps and the flow-proxy mask that defines the protected set.

**Limitations and claim boundaries (Shin).** The Shin RatBrain Fig3 analysis is intended as an end-to-end *real-IQ viability and generalization* anchor, not a clinical detection benchmark. Because there are no task labels and no ground-truth vessel segmentation, quantitative endpoints reported here are label-free proxies: baseline comparisons use deterministic flow/background proxy masks and report both threshold-free proxy separation (AUC) and tail-calibrated flow-proxy hit rates, while motion-sweep endpoints measure within-method consistency relative to each method's own no-motion reference and persistence of a fixed flow proxy under a matched background-proxy tail operating point. In addition, the "brain-like" motion stress test is an image-domain warp applied to beamformed IQ (rigid + depth-decaying elastic components) rather than a scatterer-level forward model; it is designed to mimic the specific high-rank, spatially varying residual-motion mismatch represented in Brain-SkullOR, not to reproduce full ultrasound physics. Accordingly, we interpret Shin results as evidence of (i) deterministic real-IQ ingestion and band-energy summaries, (ii) KA prior auditability and invariants (shrink-only; protected tiles unchanged), and (iii) self-consistency under controlled residual-motion corruption, while leaving ROC uplift claims to labeled regimes (Brain-* simulations) and to future raw-IQ datasets with task/landmark structure.

| Method | Frozen config | $AUC_{main/bg}$ | $AUC_{main/nuis}$ | $FPR_{nuis}$ @ $TPR_m$ |
|---|---|---|---|---|
| MC–SVD → STAP | `Brain-SIMUS-Clin-MotionMidRobust-v0` | 0.936 | 0.997 | 0.000 |
| MC–SVD | `ef95` | 0.807 | 0.076 | 0.992 |
| HOSVD | `ef95_ds2_t32` | 0.753 | 0.097 | 0.973 |
| RPCA | `lam1_it250_ds2_t32_r4` | 0.736 | 0.064 | 0.992 |
| Local SVD (Fixed Energy) | `tile16_s4_ef95` | 0.717 | 0.426 | 0.652 |
| Adaptive Local SVD | `tile12_s4_bal_r8` | 0.688 | 0.058 | 1.000 |
| Adaptive Global SVD | `conservative_r8` | 0.651 | 0.054 | 0.998 |

Table 12: Corrected SIMUS/PyMUST moving-scatterer frozen-benchmark summary on held-out corrected actual cases (seeds 23–24). Each method family is given one frozen configuration chosen on the corrected development split (seeds 21–22) using the same selection protocol; labels match the implementation taxonomy used in the benchmark scripts. STAP is reported as the chained pipeline MC–SVD → STAP using the pre-KA matched-subspace detector score, while the non-STAP rows report the corresponding baseline score. The selected configurations and exact held-out aggregates are taken from `reports/simus_motion/simus_fair_profile_search_seed2122to2324_expanded_headline.csv`; per-case held-out outputs are in `reports/simus_motion/simus_frozen_benchmark_v1p2_eval_cases.csv`.

### B.11 Corrected SIMUS/PyMUST moving-scatterer benchmark (frozen v1)

To complement the deterministic Brain-* k-Wave pilots with a moving-scatterer forward model, we freeze a corrected SIMUS/PyMUST benchmark after fixing the SIMUS motion-scaling bug uncovered during the motion-ladder audit. The frozen protocol is intentionally symmetric across methods: using corrected actual SIMUS cases from seeds 21–22 as a development split, we select one frozen configuration per method family over the repository's exposed tuning surface, then evaluate those fixed choices on held-out corrected actual cases from seeds 23–24. STAP is scored with its pre-KA matched-subspace detector (`score_stap_preka`); baselines are scored with their native baseline score maps. This benchmark is intended as a moving-scatterer realism and fairness checkpoint, not a replacement for the paper's primary labeled Brain-* and Gammex claims.

For a broader neuroscience audience, it is helpful to separate the pipeline into two roles. First, each *residualizer* suppresses tissue/clutter and passes a residual slow-time cube to a detector head. In our benchmark these residualizers are the conventional global SVD family (MC–SVD and the adaptive-global Baranger-style similarity cutoff [11, 12]), Song-style local SVD with overlap-add reconstruction [14], the stricter adaptive-local hybrid that combines Song-style localization with Baranger-style tilewise rank selection [12, 14], RPCA/PCP [24], and tensor HOSVD clutter suppression [41, 42]. Second, a *detector head* scores that residual cube. The native simple detector heads are power Doppler and Kasai lag-1 autocorrelation magnitude [15]; the proposed detector head is the matched-subspace STAP score [50, 62]. Table 12 freezes one full pipeline per method family. Table 13 then asks the stricter fairness question: if we hold the residualizer fixed and swap only the detector head on the *same* residual cube, does STAP still help?

## C Integrity checks and regime sentinels

We implement a reproducible processing pipeline (decode → baseline → STAP score → spatial-spectral features → optional KA regularization) and treat tail-regime validity as part of the ex-

| Residualizer | Frozen config | Native head | $AUC_{main/bg}$ | $AUC_{main/nuis}$ | $FPR_{nuis}$ @ $TPR_{main} = 0.5$ | STAP head | $AUC_{main/bg}$ | $AUC_{main/nuis}$ | $FPR_{nuis}$ @ $TPR_{main} = 0.5$ |
|---|---|---|---|---|---|---|---|---|---|
| MC–SVD | `ef95` | Kasai | 0.865 | 0.126 | 0.950 | `MotionMidRobust-v0` | 0.938 | 0.997 | 0.000 |
| Adaptive Global SVD | `conservative_r8` | Kasai | 0.694 | 0.172 | 0.896 | `MotionMidRobust-v0` | 0.829 | 0.991 | 0.000 |
| Local SVD (Fixed Energy) | `tile16_s4_ef95` | Kasai | 0.788 | 0.611 | 0.308 | `MotionMidRobust-v0` | 0.764 | 0.997 | 0.000 |
| Adaptive Local SVD | `tile12_s4_bal_r8` | Kasai | 0.732 | 0.148 | 0.959 | `MotionMidRobust-v0` | 0.803 | 0.993 | 0.001 |
| RPCA | `lam1_it250_ds2_t32_r4` | Kasai | 0.777 | 0.163 | 0.916 | `MotionMidRobust-v0` | 0.898 | 1.000 | 0.000 |
| HOSVD | `ef95_ds2_t32` | Kasai | 0.800 | 0.342 | 0.729 | `MotionMidRobust-v0` | 0.891 | 1.000 | 0.000 |

Table 13: Stage-symmetric detector-head audit on the same corrected SIMUS/PyMUST held-out cases (seeds 23–24). Each residualizer is frozen to the configuration selected on the corrected development split (seeds 21–22). The "native head" is the better of PD and Kasai, chosen on that same development split; in this benchmark Kasai is selected for every residualizer family. The STAP column applies the *same single frozen STAP detector head* (`Brain-SIMUS-Clin-MotionMidRobust-v0`) to the same residual cube, rather than changing the upstream residualizer. This is the strict detector-head fairness audit: it gives conventional clutter filters their strongest simple Doppler head while asking whether the matched-subspace STAP detector still improves separation on the identical residual. In five of six residualizer families STAP improves both AUC measures and essentially eliminates nuisance-tail activation; for the fixed-energy local SVD residual, the native Kasai head retains slightly higher $AUC_{main/bg}$, but STAP still sharply improves nuisance separation ($AUC_{main/nuis}$ 0.611→0.997, nuisance FPR 0.308→0.000). Source artifacts are `reports/simus_motion/simus_symmetric_pipeline_compare_seed2122to2324_headline.csv` and `reports/simus_motion/simus_symmetric_pipeline_compare_seed2122to2324.csv`.

perimental design. This appendix enumerates common pitfalls (metadata ambiguity, discretization artifacts, proxy-mask circularity, and compute-gating leakage) and the deterministic, label-free sentinels used to detect them. When a sentinel triggers, we either record provenance and proceed conservatively or force $\phi_i \in \mathcal{R}_0$ (uninformative prior; $w \equiv 1$) with an explicit reason label so that "KA off" is an auditable success mode under flat or non-actionable evidence.

# D Practical Band Design for Brain fUS (PRF 1.5–5 kHz, T 8–16)

We provide practical band guidelines for selecting Pf/guard/Pa with explicit physical anchors.

For brain fUS at center frequencies $f_0 \approx 6$–15 MHz and PRF $\approx 1.5$–5 kHz [1, 3, 6, 7, 28]:

- Microvascular brain flows (arterioles/venules) in humans and rodents are typically in the range $v \approx 3$–20 mm/s for the vessels that dominate functional responses [1, 3]. At $f_0 = 7.5$ MHz and $c \approx 1540$ m/s, this corresponds to Doppler shifts $f_D \approx 30$–200 Hz.
- Low–frequency tissue motion (respiration, cardiac pulsation) produces strong spectral content below $\approx 20$ Hz; this should be contained in Po and largely removed by high–pass filtering and/or SVD.
- Large–vessel flows (e.g. MCA–like arteries) reach $v \approx 0.3$–0.8 m/s, corresponding to $f_D$ in the kHz range [4, 5]. Depending on PRF and angle, a portion of this energy appears in high Doppler bands and can alias into the >400 Hz region corresponding to Pa [18, 20].

Thus a reasonable fixed band design for brain fUS with $T \approx 8$–16 and PRF $\approx 1.5$–3 kHz is:

- Flow band $\mathcal{B}_f$: $[30, 250]$ Hz — microvascular flows $\approx 3$–20 mm/s.
- Guard band $\mathcal{B}_g$: $[250, 400]$ Hz — protects against band–edge leakage.
- Alias band $\mathcal{B}_a$: $[400, \text{Nyquist}]$ Hz — high Doppler where large–vessel/aliased flows and certain motion artifacts live.

In discrete terms, for $T = 8$ at PRF=1.5 kHz (bin spacing 187.5 Hz), this maps to:

| Pitfall (what can go wrong) | Why it matters | Mitigation sentinel (what we enforce and where it is logged) |
|---|---|---|
| Vendor raw-container decode ambiguity (header sizing, line ordering, beam/shot layout) | A mis-decode can silently invalidate structural masks and invalidate Doppler scores and band-energy statistics. | Decoder sanity checks reconstruct B-mode and compare to dataset screenshots; ordering/coherence checks validate CFM layout. Summaries are provided in Appendix E. |
| Missing or incorrect Doppler PRF (Twinkling metadata does not reliably store PRF) | PRF determines discrete band binning and peak-occupancy statistics; wrong PRF can make Pf/Pa empty or overlapping under short ensembles, creating misleading "actionability". | PRF is treated as an explicit, frozen experimental assumption per sequence family (Appendix E) and recorded in bundle metadata. If binned Pf/Pa are empty/overlap, we force $\phi_i \in \mathcal{R}_0$ with an explicit reason. |
| Short-ensemble band discretization (coarse $\Delta f = \mathrm{PRF}/N$) | Hz-narrow bands can collapse to DC-only or become indistinguishable with $N \in \{5, 9, 17\}$. | Band operators for short ensembles are constructed as exact orthogonal DFT-bin projectors with non-overlapping bin partitions; bin membership and overlap counts are exported per run and audited by the band-sanity sentinel (the main paper). |
| Tail sample insufficiency ($n_{\mathrm{tail}} < n_{\mathrm{tail,min}}$) | Tail association metrics (e.g. $\Delta_{\mathrm{tail}}$) become undefined or noisy if the background tail is too small, which can spuriously toggle $\mathcal{R}_2$ eligibility. | Tail sizing is made explicit: we record tail sizes and use a size-aware tail fraction so that $\Delta_{\mathrm{tail}}$ is only computed when supported; otherwise we remain conservative ($\mathcal{R}_0/\mathcal{R}_1$ with an explicit reason). |
| Proxy masks misread as ground truth (real data) | Proxy-derived labels can create circular evaluation or leakage. | We restrict quantitative ROC to regimes with explicit labels: simulator-truth masks (Brain-*) and B-mode structural masks (Gammex flow phantom sequences). Proxy masks are used only for band-energy statistics/protected sets and for conditional execution, never for detection efficacy claims. |
| Conditional execution (compute gating) changes the decision rule | Same-window proxy gating can measurably change operating characteristics (including improving them), so it cannot be presented as a cost-free speedup. | Conditional execution is treated as a separate algorithm variant; any performance reporting is accompanied by leakage controls (same-window vs disjoint-window vs random-mask). Reproducibility details are provided in Appendix L. |

Table 14: Integrity checks and regime sentinels. Each sentinel is deterministic and label-free; it either records provenance/QC metrics or forces conservative KA behavior (e.g. $\phi_i \in \mathcal{R}_0$, $w \equiv 1$) with an explicit reason label.

- $P_f$ on first off–DC bin (with DPSS leakage covering $\approx$ 30–250 Hz),
- guard around the second bin,
- $P_a$ on higher bins with bin–level guard as per BG–1/BG–2.

# E Twinkling PRF overrides and decoder sanity

The Twinkling RawBCF sidecars do not reliably store Doppler PRF, so all Twinkling experiments in this manuscript use an explicit PRF override (`prf_hz`) frozen per sequence family and recorded in each bundle's `meta.json`. Table 15 lists the PRF overrides used for the Gammex structural-label experiments and the calculi hygiene experiments; a full extracted table from bundle metadata is written to `reports/twinkling_prf_override_table.csv` / `reports/twinkling_prf_override_table.json`.

| Sequence family (RawBCF) | $N$ shots | PRF override (Hz) |
|---|---|---|
| Flow in Gammex phantom (along – linear probe; `RawBCFCine.par`) | 17 | 2500 |
| Flow in Gammex phantom (across – linear probe; `RawBCFCine_*_17.par`) | 17 | 2500 |
| Twinkling/Flash artifacts on artificial calculi (`calcifications.par`) | 9 | 500 |
| Twinkling artifact on artificial calculi (`RawBCFCine_29062016_174207.par`) | 9 | 500 |

Table 15: Frozen PRF overrides for Twinkling RawBCF sequences used in this manuscript. For the Gammex flow phantom we use PRF= 2500 Hz as reported for the Gammex example acquisition in [40]. For the calculi sequences the dataset provides screenshots that encode PRF (e.g. `picture (prf=500HZ).png`); we treat PRF as an explicit experimental assumption and enforce band-sanity sentinels to avoid empty/overlapping bands under short ensembles. Values are consistent with `reports/twinkling_prf_override_table.csv`.

| Sequence family | B-mode vs PNG corr (best) | coherence ratio (beam/shot) |
|---|---|---|
| Gammex along – linear17 (`RawBCFCine.par`) | 0.737 | 2.31 |
| Gammex across – linear17 (`RawBCFCine_*_17.par`) | 0.610 | 2.46 |
| Calculi calcifications (`calcifications.par`) | 0.382 | 2.92 |
| Calculi artificial (`RawBCFCine_29062016_174207.par`) | 0.415 | 1.08 |

Table 16: Twinkling RawBCF decoder sanity metrics. "B-mode vs PNG corr" is the correlation between a decoded B-mode envelope image (log-compressed) and a dataset-provided screenshot (after bilinear resize; max over left/right flips). "Coherence ratio" is the ratio of mean temporal coherence under beam-major ordering versus shot-major ordering; values $> 1$ support the beam-major assumption used throughout. Per-sequence decode reports are written to `reports/twinkling_decode_sanity/*/decode_report.json`; the aggregate summary is written to `reports/twinkling_decode_sanity_summary.csv` / `reports/twinkling_decode_sanity_summary.json`.

# F Macé atlas alignment sanity check

To reduce the risk that offline atlas-derived summaries are dismissed as misregistered, Figure 21 overlays the Allen atlas alignment inputs used throughout the Macé section: atlas region masks (`atlas.Regions`) and the vascular atlas volume (`atlas.Vascular`) mapped into scan planes using the provided affine transform (`Transformation.mat`).

# G PD-only KA-prior summary panel (hemodynamic): band-energy statistics and aggregation

This appendix specifies a PD-only instantiation of the KA spatial-spectral prior and the associated summary-panel aggregation used for systematic plane sweeps (e.g. Macé/Urban whole-brain mouse

fUS). The key discipline is unchanged: *runtime* prior selection is label-free (proxy sets), while atlas-defined $H_1/H_0$ labels are used only *offline* for descriptive reporting and are never used as prior inputs.

## A) PD-only band-energy feature definitions (math)

**PD movie and tiling.** Fix one coronal plane (one "configuration") with a PD movie

$$P[t, y, x] \in \mathbb{R}_{\geq 0}, \qquad t = 0, \ldots, T-1,$$

sampled at rate $f_s = 1/dt$ (Macé: $f_s \approx 10$ Hz). Tile each frame into overlapping spatial tiles $\Omega_i$ of size $h \times w$ (Macé: $8 \times 8$) with stride $s$ (Macé: $s = 3$).

**Tile PD time series.** Define the tile-averaged PD time series

$$p_i[t] := \frac{1}{|\Omega_i|} \sum_{(y,x)\in\Omega_i} P[t, y, x].$$

For robustness across tiles, we optionally use a log-stabilized, mean-removed series

$$u_i[t] := \log(p_i[t] + \epsilon_{\mathrm{pd}}), \qquad \tilde{u}_i[t] := u_i[t] - \frac{1}{T}\sum_{t'=0}^{T-1} u_i[t'].$$

**Sliding-window temporal embedding (Hankel snapshots) and whitening.** Fix a slow-time aperture $L_t$ (tens of frames in PD-only). Define $K = T - L_t + 1$ and form multiple slow-time snapshots using a sliding-window temporal embedding (Hankel matrix construction):

$$x_{i,k} := \begin{bmatrix} \tilde{u}_i[k] \\ \tilde{u}_i[k+1] \\ \vdots \\ \tilde{u}_i[k+L_t-1] \end{bmatrix} \in \mathbb{R}^{L_t}, \qquad k = 0, \ldots, K-1.$$

Estimate a per-tile covariance (SCM or robust variant)

$$\widehat{R}_i := \frac{1}{K}\sum_{k=0}^{K-1} x_{i,k} x_{i,k}^{\mathrm{H}},$$

apply conditioning/diagonal loading to obtain $R_i$, and whiten

$$y_{i,k} := R_i^{-1/2} x_{i,k}.$$

We pool whitened snapshots to a single per-tile slow-time vector

$$z_i := \frac{1}{K}\sum_{k=0}^{K-1} y_{i,k} \in \mathbb{R}^{L_t}.$$

**Hemodynamic band projectors and energies.** Let $\mathcal{B}_f, \mathcal{B}_g, \mathcal{B}_a$ denote Pf/guard/Pa hemodynamic bands (Macé: $\mathcal{B}_f = [0, 0.5]$ Hz, $\mathcal{B}_g = (0.5, 1.0]$ Hz, $\mathcal{B}_a = (1.0, 3.0]$ Hz). Build fixed temporal projectors $(P_f, P_g, P_a)$ onto the span of discrete Fourier tones whose frequencies lie in these bands, and define $P_o := I - P_f - P_g - P_a$ ("other"/out-of-band). Define band energies

$$E_f(i) := z_i^{\mathrm{H}} P_f z_i, \quad E_g(i) := z_i^{\mathrm{H}} P_g z_i, \quad E_a(i) := z_i^{\mathrm{H}} P_a z_i, \quad E_o(i) := z_i^{\mathrm{H}} P_o z_i,$$

and $E_{\mathrm{tot}}(i) := E_f + E_g + E_a + E_o + \epsilon$.

**Risk metrics and Pf-peak statistics.** We use the same KA feature primitives as in the Doppler regime:

$$m_{\mathrm{alias}}(i) := \log \frac{E_a(i) + \epsilon}{E_f(i) + \epsilon}, \qquad m_{\mathrm{guard}}(i) := \log \frac{E_g(i) + \epsilon}{E_f(i) + \epsilon}, \qquad r_g(i) := \frac{E_g(i)}{E_{\mathrm{tot}}(i)}.$$

Define a whitened periodogram of the pooled vector $z_i$,

$$\widehat{z}_i[n] := \sum_{\ell=0}^{L_t-1} z_i[\ell]\, e^{-2\pi i n \ell / L_t}, \qquad S_i[n] := |\widehat{z}_i[n]|^2, \qquad n = 0, \ldots, \lfloor L_t/2 \rfloor,$$

and the (non-DC) peak frequency $f_{\mathrm{peak}}(i) = \frac{n_{\mathrm{peak}}}{L_t dt}$ where $n_{\mathrm{peak}}(i) := \arg\max_{n \geq 1} S_i[n]$. The Pf-peak indicator and occupancy on a set $\mathcal{T}$ are

$$\pi_f(i) := \mathbf{1}\{f_{\mathrm{peak}}(i) \in \mathcal{B}_f\}, \qquad \mathrm{PfPeakFrac}(\mathcal{T}) := \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \pi_f(i).$$

## B) PD-only proxy sets and prior inputs (math)

The PD-only prior uses label-free proxy sets based on a PD-derived flow-coverage proxy, analogous to the IQ flow-mask coverage $c_f$. Let $\bar{P}[y, x] := \frac{1}{T} \sum_t P[t, y, x]$ be the mean PD image (optionally restricted to a brain mask). Define a high-quantile flow proxy mask

$$M_{\mathrm{flow}}(y, x) := \mathbf{1}\{\bar{P}[y, x] \geq Q_{q_{\mathrm{flow}}}(\bar{P})\},$$

and per-tile flow coverage

$$c_f(i) := \frac{1}{|\Omega_i|} \sum_{(y,x) \in \Omega_i} M_{\mathrm{flow}}(y, x) \in [0, 1].$$

With frozen design thresholds $c_{\mathrm{bg}}$ and $c_{\mathrm{flow}}$ (typical $c_{\mathrm{bg}} \approx 0.05$, $c_{\mathrm{flow}} \approx 0.20$), define

$$\mathcal{T}_{\mathrm{bg}} := \{i : \ c_f(i) \leq c_{\mathrm{bg}}\}, \quad \mathcal{T}_{\mathrm{flow}} := \{i : \ c_f(i) \geq c_{\mathrm{flow}}\}, \quad \mathcal{T}_{\mathrm{nonbg}} := \{i : \ c_f(i) > c_{\mathrm{bg}}\}.$$

If $\mathcal{T}_{\mathrm{flow}}$ is too small (insufficient proxy support), a conservative fallback is to define proxy sets via tilewise mean-$\bar{P}$ quantiles and record this fallback choice in the exported metadata.

## C) Prior-regime selection for PD-only (math)

**Baseline score for PD-only prior.** For Macé, the baseline scalar detector is PD-z on tile time series: choose a baseline segment $B$ (e.g. first third of frames) and define

$$s_{\text{base}}(i) := \max_{t \notin B} \frac{u_i[t] - \mu_i^{(B)}}{\sigma_i^{(B)} + \epsilon_z},$$

with $\mu_i^{(B)}, \sigma_i^{(B)}$ the mean and standard deviation of $u_i[t]$ over $t \in B$. Higher $s_{\text{base}}$ means more activation.

**Risk-mode selection and integrity checks.** The PD-only prior follows the same posture as the Doppler-IQ prior: if risk statistics are flat (e.g. $\text{IQR}(m_{\text{alias}} \mid \mathcal{T}_{\text{bg}})$ too small) or proxy support is insufficient, we default to the uninformative regime ($\mathcal{R}_0$; $w \equiv 1$) and record an explicit disablement reason in the metadata. If guard dominates (e.g. $Q_{0.90}(r_g \mid \mathcal{T}_{\text{bg}})$ above a frozen threshold), we disallow the experimental alias-dominant regime $\mathcal{R}_2$ and either remain in a guard-risk safety lane ($\mathcal{R}_1$; requiring tail association of $r_g$) or default to $\mathcal{R}_0$ as out of regime.

**Actionability checks (experimental alias-dominant regime $\mathcal{R}_2$, label-free).** The experimental alias-dominant regime $\mathcal{R}_2$ (research-only eligibility; reporting-only) requires additional evidence that risk statistics explain high-score background tails. In the selected risk mode ($m_{\text{risk}} = m_{\text{alias}}$ or $m_{\text{guard}}$), define

$$\Delta_{bf} := \text{median}(m_{\text{risk}} \mid \mathcal{T}_{\text{bg}}) - \text{median}(m_{\text{risk}} \mid \mathcal{T}_{\text{flow}}),$$

and within $\mathcal{T}_{\text{bg}}$ define a tail subset $\mathcal{T}_{\text{tail}}$ as the top 1% of $s_{\text{base}}$ and a mid subset $\mathcal{T}_{\text{mid}}$ as a central quantile band; require

$$\Delta_{\text{tail}} := \text{median}(m_{\text{risk}} \mid \mathcal{T}_{\text{tail}}) - \text{median}(m_{\text{risk}} \mid \mathcal{T}_{\text{mid}}) \geq \Delta_{\text{tail,min}}.$$

Finally, we enforce a Pf-realization eligibility condition for $\mathcal{R}_2$:

$$\text{PfPeakFrac}(\mathcal{T}_{\text{flow}}) \geq p_{f,\text{min}}.$$

If these checks fail, the prior remains in $\mathcal{R}_0$ or $\mathcal{R}_1$ (guarded only) and does not enter $\mathcal{R}_2$.

**Coverage sentinels.** As in the Doppler-IQ prior, penalization must be neither inert nor global: after defining protected/candidate sets (below), the penalized fraction

$$p_{\text{shrink}} := \mathbb{P}\{g = 1 \mid \mathcal{T}_{\text{cand}}\}$$

must lie in frozen bounds (e.g. $p_{\text{min}} \leq p_{\text{shrink}} \leq p_{\text{max}}$), otherwise we default to $\mathcal{R}_0$ ($w \equiv 1$) and record a coverage-out-of-bounds sentinel.

## D) PD-only shrink-only prior mapping (optional; summary-panel parity)

The PD-only instantiation supports both a hard suppression used in the Macé systematic-sweep reporting and a soft shrink mapping for summary-panel parity. Define a protected set $\mathcal{T}_{\text{prot}}$ (tiles with high flow coverage and/or very high baseline scores) and a candidate set $\mathcal{T}_{\text{cand}}$ (non-protected tiles in an uncertain score window). Using a background-calibrated risk threshold $\tau_{\text{risk}} = Q_{q_{\text{risk}}}(m_{\text{risk}} \mid \mathcal{T}_{\text{bg}})$, define the penalization indicator

$$g(i) = \mathbf{1}\{i \in \mathcal{T}_{\text{cand}},\ m_{\text{risk}}(i) \geq \tau_{\text{risk}}\}.$$

*Hard suppression (Macé sweep):* apply the binary indicator $g$ as a hard suppression on positive activations,

$$s_{\text{KA}}(i) = s_{\text{base}}(i) - g(i)\max(s_{\text{base}}(i), 0),$$

which is equivalent to setting positive penalized tiles to zero while leaving negative scores unchanged. *Soft shrink (summary-panel parity):* set $w(i) = \max(w_{\min}, \exp(-k\,u(i)))$ with a robust normalized exceedance $u(i) \geq 0$ and apply $s_{\text{KA}}(i) = s_{\text{base}}(i) - (1 - w(i))\max(s_{\text{base}}(i), 0)$. In both cases, $g(i) = 0$ (and $w(i) = 1$) on $\mathcal{T}_{\text{prot}}$ enforces the do-not-penalize constraint (protected-set scores unchanged) and $s_{\text{KA}}(i) \leq s_{\text{base}}(i)$ enforces shrink-only behavior.

## E) Summary-panel schema and aggregation (PD-only)

For PD-only datasets the summary panel is computed per plane (configuration) and written as a CSV with one row per (scan, plane). Each row records: (i) identifiers and frozen settings (scan/plane, $T$, $f_s$, $L_t$, tile geometry, band edges), (ii) proxy support counts ($|\mathcal{T}_{\text{bg}}|$, $|\mathcal{T}_{\text{flow}}|$), (iii) prior outputs (region $\in \{\mathcal{R}_0, \mathcal{R}_1, \mathcal{R}_2\}$, reason label, risk_mode; exported in code as `state/reason`), (iv) key band-energy statistics (PfPeakFrac($\mathcal{T}_{\text{flow}}$), $Q_{0.90}(r_g \mid \mathcal{T}_{\text{bg}})$, IQR($m_{\text{risk}} \mid \mathcal{T}_{\text{bg}}$), $\Delta_{bf}$, $\Delta_{\text{tail}}$), and (v) KA coverage/strength when active ($p_{\text{shrink}}$ and scale/shrink quantiles). Atlas-labeled descriptive fields are prefixed `offline_` (e.g. `offline_pf_peak_frac_H1`, `offline_pf_peak_frac_H0`, `offline_hits_pre/post`) to make clear that they are not prior inputs.

Summary-panel aggregation mirrors the Doppler-IQ diagnostic panel: histograms of prior regions and (region, reason) pairs, a risk-mode histogram, distributions of key band-energy statistics (Pf peak fraction, guard q90, risk IQR), and (for active planes) shrink coverage/strength summaries. Any atlas-stratified Macé plots (e.g. PfPeak(VIS/SC/LGd) vs PfPeak(CP/CA/DG), alias separation) are presented as a separate, explicitly offline/descriptive panel.

## F) "Do not overclaim" messaging (PD-only summary panel)

When reporting a PD-only prior summary panel, we use the following posture in captions and limitations:

- PD-only band-energy statistics operate on low-rate PD time series and are an auditability check; they do not validate Doppler-side uplift on raw IQ.
- The uninformative regime ($\mathcal{R}_0$; $w \equiv 1$) is an intended success outcome when feature evidence is flat or non-actionable; the summary panel is designed to make these cases explicit via reason labels.
- Atlas-stratified results on Macé are descriptive; prior-positive slices identified by scanning the same dataset are not treated as held-out confirmation.

- The experimental alias-dominant regime ($\mathcal{R}_2$) is a reporting discipline requiring additional label-free actionability evidence; $\mathcal{R}_1$ is guarded only and should not be presented as a clinical performance guarantee.

# H Algorithmic details (supplementary)

## H.1 Knowledge-Aided prior regime selection (frozen defaults)

The KA prior consumes per-tile arrays $S_{\text{base}}$, $m_{\text{alias}}$, $r_g$, $c_f$, and PfPeak and returns a compact regime report (region label, a disablement reason when inert, and diagnostic metrics). The reference implementation is provided with the accompanying code. The frozen default thresholds used throughout this repository are:

- **Proxy thresholds:** $c_{\text{bg}} = 0.05$, $c_{\text{flow}} = 0.20$.
- **Candidate/protected score quantiles:** candidate window $q_{\text{lo}} = 0.30$, $q_{\text{hi,cand}} = 1.0$; optional high-score protection $q_{\text{hi,prot}} = 0.99999$ when the protected set includes extreme-score tiles.
- **Risk threshold:** $q_{\text{risk}} = 0.90$ applied to $m_{\text{alias}}$ on $\mathcal{T}_{\text{bg}}$.
- **Feature integrity (flatness + guard):** $\text{IQR}(m_{\text{alias}} \mid \mathcal{T}_{\text{bg}}) \geq 0.25$; actionable eligibility disabled if $Q_{0.90}(r_g) > 0.25$; "guard safety" lane allowed up to $Q_{0.90}(r_g) \leq 0.70$ if guard is actionable, requiring $\text{IQR}(r_g \mid \mathcal{T}_{\text{bg}}) \geq 0.03$ and $\Delta_{\text{tail,guard}} \geq 0.01$.
- **C2 actionability thresholds:** $\Delta_{bf} \geq 0.30$ and $\Delta_{\text{tail}} \geq 0.20$ (computed on $\mathcal{T}_{\text{bg}}/\mathcal{T}_{\text{flow}}$ proxies as in the main paper).
- **C2 Pf-realization eligibility:** $\text{PfPeakFrac}(\mathcal{T}_{\text{flow}}) \geq 0.05$ with $|\mathcal{T}_{\text{flow}}| \geq 50$.
- **Coverage bounds:** safety lane $p_{\text{shrink}} \in [0.02, 0.25]$ (guard lane $[0.02, 0.15]$), actionable lane $p_{\text{shrink}} \in [0.05, 0.40]$.
- **Invariance sentinel:** if $\text{IQR}(\log w \mid \text{penalized}) < 0.02$ then return the off state with an invariance/no-effect sentinel.
- **Sample support:** $n_{\min} = 500$ valid tiles, $n_{\text{bg,min}} = 200$ background-proxy tiles, $n_{\text{cand,min}} = 200$ candidate tiles, $n_{\text{tail,min}} = 20$, $n_{\text{mid,min}} = 20$.

The report exports one of three states (off / safety / actionable), corresponding to feature-space regimes $\mathcal{R}_0/\mathcal{R}_1/\mathcal{R}_2$ in the main text. When the prior is forced inert, the report includes an explicit reason category (e.g. missing inputs, insufficient proxy support, feature flatness, invariance/no-effect sentinels, or invalid band discretization in short ensembles). Reason labels are enumerated in the code and logged in the exported metadata.

## H.2 Score-space shrink mapping and exported maps

When score-space KA is enabled, per-tile shrink factors $w(i) \in (0, 1]$ are derived deterministically from the regime report and applied in score space as

$$S_{\text{post}}(i) = w(i)\, S_{\text{pre}}(i), \qquad w(i) \in (0, 1].$$

For convenience (and compatibility with legacy `pd` arrays), bundles store an equivalent per-pixel *scale* map $\gamma(i) = 1/w(i) \geq 1$ as `ka_scale_map.npy`, averaged over overlapping tiles and applied only on the union of penalized tiles as

$$\text{PD}_{\text{post}}(i) = \text{PD}_{\text{pre}}(i)/\gamma(i).$$

A binary `ka_gate_map.npy` stores the penalized region. Protected pixels are enforced by setting $\gamma = 1$ on the protected set (flow mask and extreme-score pixels as in the main paper).

**Implementation score conventions.** Throughout the manuscript, quantitative low-FPR operating points are reported using flow-evidence score maps under a right-tail convention (higher = more flow-like; thresholds calibrated on the high-score tail) intended for labeled ROC and robustness reporting (see the score taxonomy table in the main paper). Exported bundles include the baseline score and the pre-KA matched-subspace STAP score, along with optional conventional baseline comparators (e.g. log-PD and Kasai) and diagnostic/proxy maps used for gating and audits. Proxy maps are not used as the primary detector scores for labeled ROC reporting. When KA is enabled, the exported maps enforce a shrink-only mapping on the chosen detector score: in this right-tail score space, $S_{\text{post}} = w\, S_{\text{pre}}$ with $w \in (0, 1]$; for convenience, bundles also store the equivalent per-pixel scale map $\gamma = 1/w \geq 1$ applied on the penalized region.

# I Simulation details (supplementary)

## I.1 k-Wave brain pilots (Brain-* profiles)

The k-Wave brain pilots used in this repository are generated using our simulation pipeline (see Appendix L for the exact commands). The shared acquisition/geometry settings are:

- Grid: $N_x = N_y = 240$ with $dx = dy = 90$ $\mu$m; PML size 16; $c \approx 1540$ m/s; $f_0 = 7.5$ MHz; 3 cycles.
- Angles: $(-12, -6, 0, 6, 12)^\circ$; ensembles: 5; pulses per ensemble: 64 (so $T = 320$ compounded frames); PRF = 1500 Hz.
- Tiling: $8 \times 8$ tiles, stride 3; $L_t = 8$ (conservative profile temporal aperture).

**Synthetic flow/alias/clutter overlays.** To make Doppler/alias structure explicit and reproducible, the pilots apply deterministic slow-time overlays on the beamformed cube (code: `sim/kwave/common.py`). The canonical Brain-OpenSkull overlays in the Makefile are:

- Flow Doppler on the flow mask: `--flow-doppler-min-hz 60`, `--flow-doppler-max-hz 180`.
- Background alias tone: `--bg-alias-hz 650` with fraction 0.3 in depth $[0.30, 0.70]$ and jitter $\pm 50$ Hz.
- Colored temporal clutter: `--clutter-beta` 1.0 with target SNR 20 dB over depth $[0.20, 0.95]$.
- Aperture phase screen: `--aperture-phase-std 0.8` rad, correlation length 14 elements.

Brain-Pial128 uses 4 ensembles with 32 pulses each ($T = 128$ compounded frames) and concentrates shallow alias in depth $[0.15, 0.30]$ with a stronger alias fraction and a vibration tone. Brain-AliasContract and Brain-SkullOR use additional alias/vibration settings and skull/aberration tagging; all simulation parameters are fixed per profile and are specified in the reproducibility manifest (Appendix L).

## I.2 Replay/analysis pipeline used for ROC and latency

Downstream processing is run on the saved pilot angle data using `scripts/replay_stap_from_run.py`, which loads angle directories (or ensembles of angle directories), forms the slow-time cube, applies the chosen baseline, runs STAP, and writes a standard output bundle (`sim/kwave/common.`

py:write_acceptance_bundle). Low-FPR ROC points and feasibility telemetry are extracted using scripts/hab_contract_check.py.

For the conservative-profile STAP PD runs (--stap-profile clinical) used in the latency suite and robustness sweeps, we additionally use scripts/run_brain_replay_sweep.py, which:

- Reuses a fixed pilot per (regime, PRF) and replays baseline+STAP under a frozen profile on 64-frame windows (--time-window-length 64).
- Evaluates deployment-oriented variants used in the latency suite, including conditional execution and a reduced-support mode that caps effective training support by subsampling training snapshots. Exact runtime settings are recorded in bundle metadata and listed in the reproducibility manifest (Appendix L).
- Uses a frozen MC–SVD baseline and simulator-truth evaluation masks. Conditional execution is driven by a deterministic, label-free proxy flow mask derived from baseline PD brightness.

**Latency reporting protocol (steady-state, CUDA graphs).** When GPU acceleration is enabled (e.g. stap_device=cuda), the first replay window can include one-time overheads such as CUDA context initialization, kernel compilation, and optional CUDA-graph capture. For any GPU latency numbers reported from this codebase, we therefore use a steady-state replay protocol: scripts/latency_rerun_check.py replays $N$ windows in a single process (--steady-windows $N$ or --window-offsets) and reports per-window timing, with the publishable latency taken as the mean over windows $2..N$ (window 1 treated as cold-start overhead).

# J Implementation details (supplementary)

## J.1 RPCA baseline (full)

The RPCA baseline used in this repository is implemented as sim/kwave/common.py:_baseline_pd_rpca. It performs a windowed Principal Component Pursuit (PCP) using an inexact augmented Lagrangian method (IALM) on a downsampled Casorati matrix:

- Spatial downsampling: complex average pooling by factor 2 when $\max(H, W) > 128$.
- Temporal windows: default subwindow length $t_{\text{sub}} = 80$ (or shorter if $T < 80$); windows are processed sequentially.
- Optimization: IALM with at most 80 inner iterations per window (max_iters_inner= min(max_iters,80)), tolerance $10^{-4}$.
- Shrinkage: $\lambda = 1/\sqrt{\max(m, n)}$ when not supplied, where $m = T_w$ and $n = H_d W_d$.
- Low-rank step: singular value thresholding using either full SVD or a randomized SVD depending on matrix size, with a per-window rank heuristic $k = \min(8, \max(2, \lfloor T_w/10 \rfloor))$.
- Output PD: sum of sparse-component energy over windows on the downsampled grid, then nearest-neighbor upsampling to the original resolution.

## J.2 HOSVD baseline (full)

The tensor HOSVD baseline is implemented as sim/kwave/common.py:_baseline_pd_hosvd. The cube $X \in \mathbb{C}^{T \times H \times W}$ is optionally spatially downsampled (complex average pooling). Mode-0/1/2 SVDs are computed on the unfoldings and ranks are chosen either explicitly (--hosvd-ranks) or by energy fractions (--hosvd-energy-fracs, default $(0.99, 0.99, 0.99)$). The low-rank approximation is reconstructed by projecting onto the multilinear subspace, and PD is the time-

averaged residual energy $\frac{1}{T}\sum_t |X(t) - L(t)|^2$. In the brain baselines used in this repository, the Makefile target `r4c-replay-hosvd` runs HOSVD with `--hosvd-spatial-downsample 2` and `--hosvd-energy-fracs 0.99,0.99,0.99`.

### J.3 Adaptive SVD baseline (spatial singular-vector similarity cutoff)

To preempt the common critique that global SVD baselines are sensitive to an arbitrary fixed rank/energy threshold, we include an adaptive spatiotemporal SVD baseline inspired by Baranger *et al.* [12]. For a windowed slow-time cube $X \in \mathbb{C}^{T\times H\times W}$ we form the Casorati matrix $C \in \mathbb{C}^{T\times N}$ with $N = HW$ and compute its SVD $C = U\Sigma V^{\mathrm{H}}$. Let $v_k \in \mathbb{C}^N$ denote the $k$th *spatial* singular vector (column of $V$). Because singular vectors are defined up to an arbitrary complex phase, we measure spatial coherence on magnitudes and define

$$s_k := \frac{\langle |v_k|,\ |v_{k+1}|\rangle}{\||v_k|\|\ \||v_{k+1}|\|}, \qquad k = 1, \ldots, T-1.$$

We smooth $\{s_k\}$ by a short median filter and choose the clutter rank $r$ as the first index where $s_k$ falls below a robust threshold $\tau = \mathrm{median}(s) - \kappa \cdot \mathrm{MAD}(s)$, with $r$ clamped to a fixed range $[r_{\min}, r_{\max}]$ (all hyperparameters frozen per regime). For the Brain-* pilots ($T = 64$) we use median-filter width 7, $\kappa = 2.5$, and clamp $[r_{\min}, r_{\max}] = [1, 32]$. The filtered residual is

$$C_{\mathrm{res}} := \left(I - U_r U_r^{\mathrm{H}}\right) C,$$

and the baseline score is the same PD flow-evidence statistic used elsewhere, $S_{\mathrm{base}}(y, x) = \frac{1}{T}\sum_t |X_{\mathrm{res}}(t, y, x)|^2$ after reshaping $C_{\mathrm{res}}$ back to cube form. This baseline is implemented as `sim/kwave/common.py:_baseline_pd_svd_similarity`.

### J.4 Block-wise local SVD baseline (overlap-add reconstruction)

To address reviewer objections that *local* adaptive SVD clutter suppression is already standard in small-vessel ultrasound imaging, we also include a block-wise local SVD clutter filter in the style of Song *et al.* [14]. We tile the spatial grid into overlapping blocks $\{\Omega_i\}$ (block size and stride frozen per regime) and form local Casorati matrices $C_i \in \mathbb{C}^{T\times(hw)}$ by stacking slow-time vectors within each block. For each tile, we compute $C_i = U_i \Sigma_i V_i^{\mathrm{H}}$ and select a tile-local clutter rank $r_i$ using a deterministic, frozen energy-fraction rule:

$$r_i := \min\left\{ r : \frac{\sum_{k=1}^{r} \sigma_{i,k}^2}{\sum_{k=1}^{T} \sigma_{i,k}^2} \geq e \right\},$$

where $e \in (0, 1)$ is fixed. For the Brain-* pilots we use blocks $8 \times 8$, stride 3, $e = 0.90$, and Hann overlap-add weighting. The local residual is $C_{i,\mathrm{res}} = (I - U_{i,r_i} U_{i,r_i}^{\mathrm{H}}) C_i$, reshaped back to a block cube. We reconstruct a full-resolution residual cube by overlap-add with a separable Hann weight inside each block and compute the baseline score as the PD statistic on the reconstructed residual. This baseline is implemented as `sim/kwave/common.py:_baseline_pd_local_svd`.

### J.5 Adaptive-local SVD baseline (tilewise similarity-drop rank selection)

For the stricter SIMUS fairness study we additionally include an adaptive-local SVD baseline that combines the two literature ideas above rather than treating them as mutually exclusive. As in the Song-style local baseline, we tile the spatial grid into overlapping blocks $\{\Omega_i\}$, form local Casorati matrices $C_i \in \mathbb{C}^{T\times(hw)}$, and reconstruct by overlap-add with optional Hann weighting [14]. However, instead of a fixed tilewise energy threshold, each tile chooses its clutter rank from the same Baranger-style spatial singular-vector similarity-drop heuristic used by the adaptive-global baseline [12]. Concretely, for each tile we compute $C_i = U_i \Sigma_i V_i^{\mathrm{H}}$, form tile-local spatial singular-vector similarities

$$s_{i,k} := \frac{\langle |v_{i,k}|,\ |v_{i,k+1}| \rangle}{\| |v_{i,k}| \| \ \| |v_{i,k+1}| \|},$$

smooth $\{s_{i,k}\}$ with the same short median filter used in the global adaptive baseline, and choose

$$r_i := \min \{k : s_{i,k} < \mathrm{median}(s_i) - \kappa \cdot \mathrm{MAD}(s_i)\},$$

with $r_i$ clamped to a frozen range $[r_{\min}, r_{\max}]$. The local residual remains $C_{i,\mathrm{res}} = (I - U_{i,r_i} U_{i,r_i}^{\mathrm{H}}) C_i$, followed by overlap-add reconstruction and PD scoring on the residual cube. This baseline is implemented as `sim/kwave/common.py:_baseline_pd_adaptive_local_svd`. In the benchmark tables we report the three SVD-family baselines separately as *Adaptive Global SVD*, *Local SVD (Fixed Energy)*, and *Adaptive Local SVD*.

### J.6 STAP temporal core (full)

For k-Wave pilots and replay bundles, the reference temporal STAP implementation follows the same conceptual decomposition as the main text: tile extraction and optional sliding-window temporal embedding (Hankel construction) in slow time; per-tile covariance estimation with a robust estimator (Tyler-type or Huber); diagonal loading to enforce conditioning targets; whitening; and band-energy scoring on a fixed Doppler tone grid. Deployment-oriented variants can (i) cap effective training support by subsampling training snapshots and (ii) reduce computation by skipping the STAP stage on tiles with negligible proxy flow support, reverting to the baseline score there. Exact run settings and exported metadata used for the experiments in this manuscript are listed in the reproducibility manifest (Appendix L).

**CUDA acceleration strategy (batched GPU implementation).** For the latency results reported here we implement the STAP PD stage as a GPU-batched pipeline: tiles are extracted on device using tensor *unfold* operators, processed in batches to amortize per-tile overhead, and stitched back to full-resolution maps by overlap-add. This hardware-aware formulation aligns with prior work on GPU eigen-based clutter filtering and randomized/approximate SVD acceleration for real-time Doppler processing [16,17]. Within each batch, the temporal core is expressed as batched linear algebra (sliding-window temporal embedding; Hankel matrix construction; robust covariance estimation; whitening; and band-energy scoring), with rearrangements that favor GEMM-dominated workloads on GPUs. For fixed input shapes, the batched core can optionally be captured as a CUDA graph and replayed to reduce kernel-launch overhead; the first processed window/frame therefore includes one-time GPU overheads and is excluded from reported steady-state means. Exact commands and runtime settings used for the CUDA latency tables are listed in the reproducibility manifest (Appendix L).

### J.7 Covariance blending constraints and KA modes

We permit anisotropic shrinkage, but only *within* bands. In practice we consider two KA paths:

- **Updated (commuting band–scalar).** $Q^{\mathrm{H}}Q$ is forced to be bandwise scalar with target gains $(s_f, s_a, s_o)$ (typical targets: $s_f = s_o \approx 1$, $s_a \approx 1.5$), small mixing, and no off–block priors; the content of $R_0$ beyond $(P_f, P_a)$ is ignored.
- **Blend.** A trace–normalized blend $R_\beta = (1-\beta)R_t^{(n)} + \beta R_0^{(n)}$ with a small diagonal lift. In practice $R_0$ is overridden to a band–scalar form (Pf weight 1, Pa weight $\gamma_a > 1$, Po weight 1) so the geometry is explicit; $\beta$ is treated as a fixed hyperparameter per regime.

Both modes enforce bandwise structure; there are no off–block priors.

### J.8 CB–1: Band–Block Blending

We construct

$$R_\beta := (1-\beta)\,\widehat{R} + \beta\big(R_{0,f} \oplus R_{0,a} \oplus R_{0,o}\big) + \delta I,$$

where $R_{0,f} = P_f R_0 P_f$, etc., for some prior $R_0$. There are no off–block priors; $\beta \in [0,1)$ and $\delta \geq 0$.

Interpretation: this structure makes $Q$ nearly block–diagonal in $(P_o, P_f, P_g, P_a)$ while still allowing bandwise control to realize OC–2.

### J.9 CB–2: Bandwise Spectral Targets

We choose $(\beta, \delta)$ to enforce OC–2 explicitly:

$$\lambda_{\min}(Q^{\mathrm{H}}Q|_{P_f}) \in [s_f^2(1-\delta_f), s_f^2(1+\delta_f)], \quad \text{etc.}$$

with design tolerances $\delta_\bullet \leq 0.1$ and separation $\Lambda = s_a/s_f \geq 1.25$.

### J.10 CB–3: Bandwise Condition Number Safeguard

We constrain bandwise conditioning:

$$\kappa(R_\beta|_{P_f}) \leq \kappa_f^{\max}, \qquad \kappa(R_\beta|_{P_a}) \leq \kappa_a^{\max},$$

avoid an incompatibility between conditioning and preserving flow Rayleigh quotients.

### J.11 Localization (restricting KA in score space)

A global band–constant transform with $Q$ cannot improve ROC if applied everywhere (it is close to a scalar reweighting of BR/GLRT). To create re–ranking, KA must be *localized* to tiles where:

- scores are dominated by alias/undesired components,
- positives are unlikely to lie.

We define localization based on a combination of alias metrics, flow coverage, depth, PD, and registration. This supplements G–1/G–2 with a generalized alias score $m_{\text{alias}}$ from Section A.1.

### J.12 G–1: Alias–dominant tile selection

Given a baseline tile $z$, compute per–band energies $(E_o, E_f, E_g, E_a)$ and a generalized alias score $m_{\text{alias}}(z)$ satisfying R–2.

**Data–adaptive threshold.** For BG tiles (or their proxy), collect $m_{\text{alias}}(z)$ and define

$$\tau_{\text{alias}} := Q_{q_{\text{alias}}}\big(m_{\text{alias}} \mid H_0\big),$$

where $Q_q$ is the $q$–quantile and $q_{\text{alias}} \in [0.7, 0.9]$ is a design choice (typical: 0.8 or 0.9).

Define an alias gate

$$g_{\text{alias}}(z) = \mathbf{1}\left\{m_{\text{alias}}(z) \geq \tau_{\text{alias}}\right\}.$$

**Band–edge purity.** Additionally require that guard energy is small relative to Pf+Pa:

$$\frac{E_g}{E_o + E_f + E_a + E_g} \leq \eta_{\text{edge}}, \quad \eta_{\text{edge}} \leq 0.1.$$

Interpretation: this gate selects the top tail of alias–heavy tiles among negatives, while protecting against band–edge contamination. The distribution of $m_{\text{alias}}$ across classes is governed by R–2; empirical validation (Section A.1) must show a nontrivial difference between $H_0$ and $H_1$.

### J.13 G–2: Flow coherence protection

We can optionally compute a flow coherence metric FPC (first principal component fraction) within the flow band:

$$\text{FPC} := \frac{\lambda_1(\widehat{S}_f)}{\text{tr}(\widehat{S}_f)}, \qquad \widehat{S}_f := \frac{1}{K}\sum_{k=1}^{K} u_k u_k^{\text{H}}, \quad u_k := P_f z^{(k)},$$

where $z^{(k)}$ are projected slow–time vectors across DPSS tapers or within a local ensemble.

Gate *may* require

$$\text{FPC} \leq c_{\text{pc}}, \qquad c_{\text{pc}} \in [0.4, 0.6].$$

Interpretation: flow tiles tend to exhibit higher coherence in Pf; when enabled, we suppress penalization when FPC is large, thereby reducing positive penalization probability $p_+$. Alias/motion tiles have lower flow coherence and pass this condition. In the current runs this check is typically left off; coverage and depth localization carry most of the class separation.

### J.14 G–2': Flow coverage protection

Let $c_f$ be the tile–wise flow coverage fraction:

$$c_f := \frac{\#\{\text{pixels labeled as flow}\}}{\#\{\text{pixels in tile}\}},$$

using a label–free proxy such as a PD–derived flow mask.

We require

$$c_f \leq c_{f,\max}, \qquad c_{f,\max} \in [0.1, 0.2],$$

and optionally $c_f \geq c_{f,\min}$ to avoid pure background noise tiles (typical $c_{f,\min} \approx 0$).

Interpretation: we prefer penalization on tiles with *low to moderate* flow coverage. This is the regime where alias/motion dominates and where down–scaling scores is most beneficial. Dense flow tiles are either left to STAP alone or are protected via the flow coherence check. (In the current implementation, coverage is treated as an *upper* bound; missing coverage defaults to "pass" in this test.)

### J.15 G–3: Baseline score quantile window

Let $S_{\text{base}}(z)$ be the baseline BR/GLRT score. Within a given depth stratum, compute score quantiles $q_{0.3}, q_{0.95}$ over BG tiles. Gate requires

$$S_{\text{base}}(z) \in [q_{0.3}, q_{0.95}].$$

Interpretation: avoid penalization on the noise floor (very low scores) and on already high–confidence positives (very high scores). This focuses KA on the part of the negative tail that matters near the target FPR.

### J.16 G–4: Registration quality (optional)

If motion compensation / registration is used, include a registration quality metric (e.g. PSR) and require

$$\text{PSR}_{\text{reg}} \leq \text{PSR}_{\text{max}},$$

or any similar condition ensuring that tiles with gross registration failure are handled separately.

### J.17 Offline validation of gating

On a held–out labeled configuration, define

$$p_+ := \mathbb{P}\{g = 1 \mid H_1\}, \qquad p_- := \mathbb{P}\{g = 1 \mid H_0\},$$

for the combined gate $g$ that incorporates G–1, G–2, G–2', G–3, G–4. Require:

$$p_- \geq 0.2, \quad p_- \leq 0.5, \quad p_+ \leq 0.1 p_-.$$

Interpretation: a nontrivial fraction of negatives are gated (but not too many), while positives are rarely gated. This condition is central to the SL–2 quantile shift argument.

### J.18 Summary: Compact Feasibility Checklist (Updated)

KA–STAP can be expected to outperform MC–SVD (or STAP–only) in the low-FPR regime if and only if:

1. **Regime preconditions.** Band occupancy R–1, alias separation R–2, and baseline headroom R–3 are satisfied on a held–out validation set.
2. **Band design.** BG–1/BG–2/BG–3 hold for the chosen PRF/$T$/DPSS; Pf/Pa are physically meaningful (e.g. Pf $\approx$ 30–250 Hz, Pa $\gtrsim$ 400 Hz in brain fUS).
3. **Operator geometry.** OC–1 with $\varepsilon_{\text{mix}} \leq 0.05$ and OC–2 with $\Lambda = s_a/s_f \geq 1.25$ and $\delta_\bullet \leq 0.1$.
4. **Blending realizable.** CB–1/CB–2/CB–3 satisfied with chosen $(\beta, \delta)$.
5. **Gating conservative and class–differential.** G–1/G–2/G–2'/G–3/G–4 implemented; offline, $p_- \in [0.2, 0.5]$ and $p_+ \leq 0.1 p_-$.
6. **Regime sentinel.** RS–1 does not trigger (KA not out of regime).
7. **Invariance sentinel.** $\widehat{s}_a/\widehat{s}_f \notin [1-\epsilon, 1+\epsilon]$ with $\epsilon \leq 0.02$; otherwise KA is effectively scalar and can be skipped.
8. **Safety guard.** Safety–1 (monotonicity on gated flow tiles) passes.

| Bundle file(s) | Meaning | Primary use / direction |
|---|---|---|
| `pd_base.npy`, `pd_stap.npy` | PD-mode maps (baseline / legacy "STAP"). | Deterministic preprocessing (proxy masks) and legacy visualization/compatibility; not used as the primary detector score for quantitative reporting in this manuscript. |
| `score_pd_base.npy`, `score_pd_stap.npy` | Legacy right-tail PD score maps (baseline / PD-mode "STAP"). | Optional legacy PD scores with a fixed sign convention (`score_pd`=`pd` in current bundles); used for compatibility and auditing rather than for headline ROC. |
| `pd_stap_pre_ka.npy`, `score_pd_stap_pre_ka.npy` (optional) | Pre-KA STAP maps (when KA is enabled). | KA auditing (pre/post) under the same sign convention. |

Table 17: Acceptance-bundle PD naming/sign schema.

| Bundle file(s) | Meaning | Primary use / direction |
|---|---|---|
| `score_base.npy` | Baseline power Doppler score (right tail). | Structural ROC; tail hygiene; proxy masks. |
| `score_base_pdlog.npy` (optional) | $\log(\texttt{score_base} + \epsilon)$ (right tail). | Baseline fairness / strawman avoidance. |
| `score_base_kasai.npy` (optional) | $\log(\lvert \sum_t y_t \overline{y_{t-1}} \rvert + \epsilon)$ (Kasai lag-1 power; right tail). | Short-ensemble Doppler baseline fairness. |
| `score_stap_preka.npy`, `score_stap.npy` | STAP score maps (pre/post KA; right tail). | Structural ROC; within-ensemble motion ladders; motion robustness; KA auditing. |

Table 18: Acceptance-bundle score-map naming schema for labeled ROC and label-free reporting (right-tail score convention).

9. **Evaluation discipline.** GLRT controls and tail CI practices (see the main paper) in place for fair low–FPR comparison.

When all of the above hold, KA applies a non–mixing, alias–inflating band transform chiefly to alias–dominant tiles, pushing negative scores down while largely leaving positives unchanged. This lowers the negative $\alpha$–quantile and increases TPR at fixed small FPR in a manner consistent with the SL–2 quantile shift analysis.

# K Bundle artifacts and naming conventions

This appendix records the artifact naming used in the released acceptance bundles and the mapping between the mathematical score notation in the main text and the stored arrays. These details are provided for reproducibility and are not required to follow the core derivations.

| Exported artifact | Property / monotonicity | Allowed claim scope (this manuscript) |
|---|---|---|
| `score_stap_preka.npy` | Primary STAP detector score (matched-subspace; right tail). Not a shrink-only reweighting of baseline PD. | Quantitative ROC on labeled simulations and structurally labeled phantoms; label-free motion robustness endpoints on unlabeled IQ. |
| `score_base.npy` | Baseline power Doppler flow-evidence score (higher = more flow-like). | Quantitative ROC as baseline comparator (same labels/masks as STAP). |
| `score_base_pdlog.npy` | Monotone transform of `score_base` (ordering preserved). | Baseline-score fairness check; does not change within-method rankings. |
| `score_base_kasai.npy` | Lag-1 autocorrelation magnitude score (Kasai power; higher = more flow-like). | Short-ensemble baseline-score fairness check (Twinkling). |
| `score_stap.npy` | Post-KA score map. By construction, shrink-only relative to `score_stap_preka` on gated regions ($S_{\text{post}} \leq S_{\text{pre}}$). | KA auditing and tail-hygiene reporting; not used as the primary detector score unless stated explicitly. |
| `pd_base.npy` | Conventional PD brightness proxy $\text{PD}^{\uparrow}$ (nonnegative). | Deterministic preprocessing (proxy masks) and legacy visualization; quantitative reporting uses detector *scores*. |
| `pd_stap.npy` | PD-mode "STAP" map exported for legacy compatibility/visualization. In PD-mode it is a non-increasing reweighting of `pd_base.npy`. | Visualization / compatibility output; can suppress background tails but cannot recover flow removed by baseline clutter suppression. |

Table 19: Score taxonomy and claim scope. Quantitative detection results are reported using flow-evidence detection scores (baseline power Doppler vs matched-subspace STAP; higher = more likely flow). PD-mode maps are treated as legacy/visualization artifacts with explicit monotonicity constraints.

# L Reproducibility Manifest

This appendix records a minimal reproduction recipe: exact commands, data locations, and a pinned repository state. The manifest lists the minimal set of commands/outputs needed to reproduce the main claims; additional exploratory reports referenced in the text are not required unless explicitly listed here. Run `PYTHONPATH=.condarun-nstap-fuspythonscripts/generate_repro_manifest.py` to refresh this manifest.

**Repository state.** Public repository: `https://github.com/ArthurShune/stap-for-fus.git`. Release tag: (not recorded; set `STAP_FUS_RELEASE_TAG` or create a git tag). Commit: `d502adc` (full: `d502adcc04f1c62ad7a40525199ede6c199a2e07`); branch: `main`; dirty: `true`. Dirty details: non-ignored paths=3, ignored build products=0. Note: dirty status ignores common LaTeX build products (e.g., `*.aux, *.log, *.out, *.fls, *.fdb_latexmk, *.synctex.gz, *.pdf`). For archival reproducibility, generate this manifest from a clean, tagged release commit (`dirty=false`) and record the public repository URL and tag.

**Environment.** Conda spec: `environment.yml` (sha256: `d2216d753643a3f62e3f69be66bfff77bb7e4671a2668ecd4d5a16eba6f76c73`). Create env: `condaenvcreate-fenvironment.yml`. Docker: `Dockerfile` (sha256: `1dbf3078785426b45770b85f7b2f7794cd14435c20e1c587a3c38cf097128e16`). Build container: `dockerbuild-tstap-fus-fDockerfile..` Runtime (conda env `stap-fus`): Python: `3.11.14|packagedbyconda-forge|(main,Jan262026,23:48:32)[GCC14.3.0]`. Platform: `Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.39`. Packages: `numpy`=2.2.6, `scipy`=1.14.1, `torch`=2.4.1, `matplotlib`=3.10.0, `cupy`=14.0.1. Torch CUDA: available=`true`, cuda=`12.1`, device0=`NVIDIAGeForceRTX4080SUPER`.

**Datasets.** Paths below are the expected on-disk locations used by the scripts in this repository. Large outputs under `runs/` and `reports/` are not tracked.

- **Brain-* k-Wave pilots (simulated IQ)**: Labeled low-FPR ROC + ablations.
  Local: `runs/pilot/`.
  Source: generated locally via sim/kwave/pilot_motion.py (Makefile targets r4c-*).
- **Mace/Urban whole-brain mouse fUS (PD-only)**: PD-only telemetry + hold-out alias-gate evaluation (no Doppler IQ).
  Local: `data/whole-brain-fUS`.
  Source: `https://doi.org/10.5281/zenodo.4905862`.
- **Shin RatBrain Fig3 (LOCA-ULM) beamformed IQ**: Real-IQ ingestion, telemetry, contract + label-free robustness sweeps.
  Local archive: `data/shin_zenodo_10711806/RatBrain_Fig3.zip`.
  Extracts to: `data/shin_zenodo_10711806/ratbrain_fig3_raw`.
  Source: `https://doi.org/10.5281/zenodo.10711806`.
- **Twinkling artifact dataset (RawBCF phantoms)**: Structurally labeled phantom ROC (Gammex) + within-ensemble motion ladders + KA hygiene (calculi).
  Local: `data/twinkling_artifact/`.
  Source: `https://doi.org/10.17816/DD76511`.
- **ULM Zenodo 7883227 (rat brain kHz IQ)**: Label-free motion robustness + one-time baseline calibration sweep.
  Local archive: `data/ulm_zenodo_7883227/IQ_001_to_025.zip`.
  Extracts to: `tmp/ulm_zenodo_7883227/`.
  Source: `https://doi.org/10.5281/zenodo.7883227`.

**Pre-registered selections used in this manuscript.**

- **Brain low-FPR matrix (fixed-profile pilots)**: OpenSkull seed1; AliasContract seed2; SkullOR seed2; 64-frame windows at offsets 0,64,128,192,256.
- **Shin all-clips telemetry**: IQData001-IQData080 (n=80), window 0:128.
- **Shin motion subset**: IQData001-005, IQData010, IQData020, IQData040, IQData060, IQData080 (n=10).
- **Mace holdout split (deduplicated)**: train: scan1/2 + scan3/6; test: scan4 + scan5.
- **Gammex flow phantom structural ROC (along-linear17)**: frames 0:85 (n=85), PRF=2500 Hz, N=17 shots.
- **Gammex flow phantom structural ROC (across-linear17)**: frames 0:200 (n=200), PRF=2500 Hz, N=17 shots.
- **Gammex flow phantom within-ensemble motion ladder**: along-linear17, frames 0:85,

amps 0..2 px, kinds rw,step.

- **Twinkling calculi KA hygiene**: calcifications sequence, frames 0:50 (n=50), PRF approx 500 Hz, N=9 shots.
- **ULM 7883227 baseline sweep**: blocks 1-3, frames 0:128, MC-SVD energy-frac sweep (label-free).
- **ULM 7883227 motion sweeps**: blocks 1-3, frames 0:128, frozen baseline e=0.975, motion kinds brainlike,elastic.

**Artifact commands.** Each entry below lists the command(s) used to regenerate the referenced paper artifact(s). Most scripts write a JSON summary containing the full parameterization and are safe to rerun.

(R1) **RTX 4080 SUPER audit logs (nvidia-smi + stap-fus conda env versions)** (paper refs: Table: latency_summary_4080super). Outputs:
`reports/hw/4080super_nvidia_smi.txt`
`reports/hw/4080super_env.txt`
Notes: These logs are hardware-dependent and are captured once per machine/driver stack.

```
nvidia-smi > reports/hw/4080super_nvidia_smi.txt

conda run -n stap-fus python -c "import platform,sys; import torch; print('platform:', platform.platform());
```

(R2) **RTX 4080 SUPER refactor gates (quick/phase/full)** (paper refs: Reproducibility gates (Phase 7)). Outputs:
`reports/refactor/4080super_refactor_quick.log`
`reports/refactor/4080super_refactor_phase.log`
`reports/refactor/4080super_refactor_full.log`
Notes: The 'full' gate is optional on constrained machines; see Makefile targets.

```
conda run -n stap-fus make refactor-quick 2>&1 | tee reports/refactor/4080super_refactor_quick.log
conda run -n stap-fus make refactor-phase 2>&1 | tee reports/refactor/4080super_refactor_phase.log
conda run -n stap-fus make refactor-full  2>&1 | tee reports/refactor/4080super_refactor_full.log
```

(R3) **PD-mode sanity checks (PD-mode map and score S=PD)** (paper refs: Notation (PD convention)). Outputs:
`reports/pd_mode_sanity/(optionalfigs)`
Notes: Run on any generated acceptance bundle root containing pw_* directories.

```
PYTHONPATH=. python scripts/pd_mode_sanity.py \
  runs/<bundle_root>/pw_* \
  --out-dir reports/pd_mode_sanity
```

(R4) **Conditional STAP leakage ablation (full vs conditional masks)** (paper refs: Results (Brain-k-Wave): conditional STAP ablation). Outputs:
`reports/condstap_leakage.csv`
`reports/condstap_leakage.json`

```
PYTHONPATH=. python scripts/conditional_stap_leakage_ablation.py \
  --pilots runs/pilot/r4c_kwave_seed1 \
  --profile Brain-OpenSkull \
  --out-root runs/ablation/condstap_leakage \
  --window-length 64 --window-offset 0 --disjoint-offset 64 \
  --summary-csv reports/condstap_leakage.csv \
  --summary-json reports/condstap_leakage.json
```

(R5) **CUDA latency replay (Brain-* k-Wave; steady-state windows 2..5)** (paper refs: Table: latency_summary). Outputs:
`runs/latency_s12_publish_offsets/(Brain-OpenSkull;full)`
`runs/latency_s13_publish_offsets_cond/(Brain-OpenSkull;conditional)`
`runs/latency_s14_aliascontract_full/(Brain-AliasContract;full)`
`runs/latency_s14_aliascontract_cond/(Brain-AliasContract;conditional)`
`runs/latency_s15_skullor_full/(Brain-SkullOR;full)`
`runs/latency_s15_skullor_cond/(Brain-SkullOR;conditional)`
Notes: GPU latency is reported as the steady-state mean over windows 2..5 (window1 is cold: CUDA init, Triton JIT, and CUDA-graph capture). Use the script's 'optimized steady(avg win2..5)' line.

```
PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_open \
  --out-root runs/latency_s12_publish_offsets \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy

PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_open \
  --out-root runs/latency_s13_publish_offsets_cond \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on

PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_aliascontract \
  --out-root runs/latency_s14_aliascontract_full \
  --profile Brain-AliasContract \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy

PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_aliascontract \
  --out-root runs/latency_s14_aliascontract_cond \
  --profile Brain-AliasContract \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on

PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
```

```
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_skullor \
  --out-root runs/latency_s15_skullor_full \
  --profile Brain-SkullOR \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy

PYTHONPATH=. STAP_FAST_CUDA_GRAPH=1 conda run -n stap-fus \
  python scripts/latency_rerun_check.py \
  --src runs/latency_pilot_skullor \
  --out-root runs/latency_s15_skullor_cond \
  --profile Brain-SkullOR \
  --window-length 64 --window-offset 0 \
  --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on
```

(R6) **CUDA latency replay (real data: Shin + Gammex; steady-state frames 2..N)** (paper refs: Table: latency_summary). Outputs:
`runs/latency_s17_realdata_cuda_unfold/(Shin+GammexCUDAlatencyruns)`
Notes: The script prints cold(win1) and steady(avg win2..N); the latency table uses the steady-state mean and excludes window/frame 1 (graph capture + Triton JIT).

```
PYTHONPATH=. \
STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 192 --clean \
  --out-root runs/latency_s17_realdata_cuda_unfold \
  shin --iq-file IQData001.dat \
  --windows 0:128,64:192,122:250 \
  --Lt 64 --svd-energy-frac 0.97

PYTHONPATH=. \
STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 512 --clean \
  --out-root runs/latency_s17_realdata_cuda_unfold \
  gammex --frames-along 0:6 --frames-across 0:6
```

(R7) **RTX 4080 SUPER latency reruns + consolidated summaries (legacy vs optimized; steady-state windows/frames 2..N)** (paper refs: Table: latency_summary_4080super). Outputs:
`reports/hw/4080super_nvidia_smi.txt`
`reports/hw/4080super_env.txt`
`runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256/`
`runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256_cond/`
`runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256/`
`runs/latency_4080super_brain_aliascontract_`

w64_tb192_off0_64_128_192_256_cond/
runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256/
runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256_cond/
runs/latency_4080super_shin_fig3_w128_tb192/
runs/latency_4080super_gammex_linear17_f0_6_tb512/
reports/latency/4080super_latency_summary.csv
reports/latency/4080super_latency_summary.json

Notes: The replay scripts print cold(win1) and steady(avg win2..N); the paper tables report steady-state means. For Brain k-Wave, scripts/latency_rerun_check.py forces legacy vs optimized CUDA-graph settings internally (legacy: STAP_FAST_CUDA_GRAPH=0; optimized: STAP_FAST_CUDA_GRAPH=1).

```
# Hardware audit (record driver/GPU; record stap-fus env versions):
nvidia-smi > reports/hw/4080super_nvidia_smi.txt
conda run -n stap-fus python -c 'import platform, sys; import torch; print("python", sys.version.splitlines()

# Brain k-Wave latency reruns (offsets 0/64/128/192/256; tile_batch=192):
PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_seed1 \
  --out-root runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256 \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_seed1 \
  --out-root runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256_cond \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0 \
  --stap-conditional on

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_seed2 \
  --out-root runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256 \
  --profile Brain-AliasContract \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_seed2 \
  --out-root runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256_cond \
  --profile Brain-AliasContract \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0 \
  --stap-conditional on

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_v3_skull_seed2 \
  --out-root runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256 \
  --profile Brain-SkullOR \
```

```
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_v3_skull_seed2 \
  --out-root runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256_cond \
  --profile Brain-SkullOR \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --tile-batch 192 \
  --cuda-warmup-heavy --stap-debug-samples 0 \
  --stap-conditional on

# Shin RatBrain Fig3 latency (requires dataset; see docs/data_download.md):
PYTHONPATH=. STAP_FAST_PATH=1 STAP_TILING_UNFOLD=0 STAP_FAST_CUDA_GRAPH=0 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 192 --clean \
  --out-root runs/latency_4080super_shin_fig3_w128_tb192/legacy \
  shin --data-root data/shin_zenodo_10711806/ratbrain_fig3_raw \
  --iq-file IQData001.dat \
  --windows 0:128,64:192,122:250 \
  --prf-hz 1000 --Lt 64 --svd-energy-frac 0.97

PYTHONPATH=. STAP_FAST_PATH=1 STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 192 --clean \
  --out-root runs/latency_4080super_shin_fig3_w128_tb192/optimized \
  shin --data-root data/shin_zenodo_10711806/ratbrain_fig3_raw \
  --iq-file IQData001.dat \
  --windows 0:128,64:192,122:250 \
  --prf-hz 1000 --Lt 64 --svd-energy-frac 0.97

# Gammex linear17 latency (along/across frames 0..5):
PYTHONPATH=. STAP_FAST_PATH=1 STAP_TILING_UNFOLD=0 STAP_FAST_CUDA_GRAPH=0 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 512 --clean \
  --out-root runs/latency_4080super_gammex_linear17_f0_6_tb512/legacy \
  gammex --frames-along 0:6 --frames-across 0:6

PYTHONPATH=. STAP_FAST_PATH=1 STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 512 --clean \
  --out-root runs/latency_4080super_gammex_linear17_f0_6_tb512/optimized \
  gammex --frames-along 0:6 --frames-across 0:6

# Collect consolidated CSV/JSON (steady windows/frames 2..N; includes parity checks):
PYTHONPATH=. conda run -n stap-fus python scripts/collect_latency_summary.py
```

(R8) **RTX 4080 SUPER CUDA latency replay (Brain-* k-Wave; legacy vs optimized; steady-state windows 2..5)** (paper refs: Table: latency_summary_4080super). Outputs: `runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256/` `runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256_cond/`

`runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256/`
`runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256_cond/`
`runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256/`
`runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256_cond/`
Notes: The script writes both legacy/ and optimized/ subtrees per out-root and reports cold(win1) and steady(avg win2..N). Publishable latency uses windows 2..N.

```
PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_seed1 \
  --out-root runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256 \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional off

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_seed1 \
  --out-root runs/latency_4080super_brain_openskull_w64_tb192_off0_64_128_192_256_cond \
  --profile Brain-OpenSkull \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_seed2 \
  --out-root runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256 \
  --profile Brain-AliasContract \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional off

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_seed2 \
  --out-root runs/latency_4080super_brain_aliascontract_w64_tb192_off0_64_128_192_256_cond \
  --profile Brain-AliasContract \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_v3_skull_seed2 \
  --out-root runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256 \
  --profile Brain-SkullOR \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional off

PYTHONPATH=. conda run -n stap-fus python scripts/latency_rerun_check.py \
  --src runs/pilot/r4c_kwave_hab_v3_skull_seed2 \
  --out-root runs/latency_4080super_brain_skullor_w64_tb192_off0_64_128_192_256_cond \
```

```
  --profile Brain-SkullOR \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --stap-device cuda --stap-debug-samples 0 \
  --tile-batch 192 --cuda-warmup-heavy \
  --stap-conditional on
```

(R9) **RTX 4080 SUPER CUDA latency replay (real data: Shin + Gammex; legacy vs optimized; steady-state frames 2..N)** (paper refs: Table: latency_summary_4080super).
Outputs:
`runs/latency_4080super_shin_fig3_w128_tb192/legacy/`
`runs/latency_4080super_shin_fig3_w128_tb192/optimized/`
`runs/latency_4080super_gammex_linear17_f0_6_tb512/legacy/`
`runs/latency_4080super_gammex_linear17_f0_6_tb512/optimized/`
Notes: These are 'latency-only' replay settings (e.g., STAP_TYLER_MAX_ITER=1, STAP_SNAPSHOT_STR
to isolate compute throughput; see per-bundle meta.json for exact telemetry.

```
PYTHONPATH=. \
STAP_TILING_UNFOLD=0 STAP_FAST_CUDA_GRAPH=0 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 192 --clean \
  --out-root runs/latency_4080super_shin_fig3_w128_tb192/legacy \
  shin --iq-file IQData001.dat \
  --windows 0:128,64:192,122:250 \
  --Lt 64 --svd-energy-frac 0.97

PYTHONPATH=. \
STAP_FAST_PATH=1 STAP_TILING_UNFOLD=0 STAP_FAST_CUDA_GRAPH=0 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 512 --clean \
  --out-root runs/latency_4080super_gammex_linear17_f0_6_tb512/legacy \
  gammex --frames-along 0:6 --frames-across 0:6

PYTHONPATH=. \
STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 192 --clean \
  --out-root runs/latency_4080super_shin_fig3_w128_tb192/optimized \
  shin --iq-file IQData001.dat \
  --windows 0:128,64:192,122:250 \
  --Lt 64 --svd-energy-frac 0.97

PYTHONPATH=. \
STAP_FAST_PATH=1 STAP_TILING_UNFOLD=1 STAP_FAST_CUDA_GRAPH=1 \
STAP_SNAPSHOT_STRIDE=4 STAP_TYLER_MAX_ITER=1 STAP_TYLER_EARLY_STOP=0 \
STAP_TYLER_TRITON_CAPTURE=1 \
conda run -n stap-fus python scripts/latency_realdata_rerun_check.py \
  --stap-device cuda --tile-batch 512 --clean \
  --out-root runs/latency_4080super_gammex_linear17_f0_6_tb512/optimized \
  gammex --frames-along 0:6 --frames-across 0:6
```

(R10) **RTX 4080 SUPER latency summary aggregation (CSV/JSON)** (paper refs: Table: latency_summary_4080super). Outputs:

`reports/latency/4080super_latency_summary.csv`

`reports/latency/4080super_latency_summary.json`

Notes: Reads existing runs/latency_4080super_* folders and writes steady-state windows 2..N summaries.

```
PYTHONPATH=. conda run -n stap-fus python scripts/collect_latency_summary.py
```

(R11) **Brain-* baseline fairness: MC–SVD(e) tune-once sweep** (paper refs: Baseline fairness (Brain-*)). Outputs:

`reports/brain_mcsvd_energy_sweep_seed1.csv`

`reports/brain_mcsvd_energy_sweep_seed1.json`

```
PYTHONPATH=. python scripts/brain_mcsvd_energy_sweep.py \
  --pilot runs/pilot/r4c_kwave_seed1 \
  --profile Brain-OpenSkull \
  --out-root runs/sweep/mcsvd_energy_brain_seed1 \
  --window-length 64 --window-offset 0 \
  --energy-fracs 0.90,0.95,0.97,0.975,0.98,0.99 \
  --fprs 1e-4,3e-4,1e-3 \
  --summary-csv reports/brain_mcsvd_energy_sweep_seed1.csv \
  --summary-json reports/brain_mcsvd_energy_sweep_seed1.json
```

(R12) **Brain-* low-FPR baseline matrix (vnext; includes adaptive/local SVD baselines)**

Paper refs:

Table: brain_kwave_vnext_baselines

Figure: `brain_kwave_roc_curves.pdf`

Outputs:

`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/(generatedbundles)`

`reports/fair_matrix_vnext_r3_localbaselines.csv`

`reports/fair_matrix_vnext_r3_localbaselines.json`

`reports/brain_kwave_vnext_baselines_table.tex`

Notes: Generates per-window acceptance bundles and writes a vnext-style strict-tail report, then renders the LaTeX table used in the main Brain-* baseline table; this run root is also consumed by the ROC and cross-window calibration scripts.

```
bash scripts/reproduce_table5_brain_kwave.sh

# Manual breakdown (equivalent to the script above):
PYTHONPATH=. conda run -n stap-fus python scripts/fair_filter_comparison.py \
  --mode matrix --eval-score vnext \
  --matrix-regimes open,aliascontract,skullor \
  --matrix-seeds-open 1 --matrix-seeds-aliascontract 2 --matrix-seeds-skullor 2 \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --matrix-use-profile \
  --matrix-mcsvd-energy-frac 0.90 --matrix-mcsvd-baseline-support window \
  --methods mcsvd,svd_similarity,local_svd,rpca,hosvd,stap_full \
  --generated-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --autogen-missing --stap-device cuda \
  --out-csv reports/fair_matrix_vnext_r3_localbaselines.csv \
  --out-json reports/fair_matrix_vnext_r3_localbaselines.json

PYTHONPATH=. conda run -n stap-fus python scripts/brain_kwave_vnext_baselines_table.py \
  --fair-matrix-json reports/fair_matrix_vnext_r3_localbaselines.json \
  --out-tex reports/brain_kwave_vnext_baselines_table.tex
```

(R13) **Brain-* ROC curve figure (median+IQR over disjoint windows)** (paper refs: Figure: `brain_kwave_roc_curves.pdf`). Outputs:
`figs/paper/brain_kwave_roc_curves.pdf`
Notes: Reads per-window score/mask arrays under the Brain-* pilot run root.

```
PYTHONPATH=. python scripts/fig_brain_kwave_roc_curves.py \
  --runs-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --out-pdf figs/paper/brain_kwave_roc_curves.pdf
```

(R14) **Brain-* cross-window threshold-transfer audit (STAP)** (paper refs: Appendix: cross-window threshold calibration audit (Brain-*)). Outputs:
`reports/brain_crosswindow_calibration.csv`
`reports/brain_crosswindow_calibration_summary.json`
`reports/brain_crosswindow_calibration_table.tex`
Notes: Calibrates thresholds on one 64-frame window's negatives and evaluates on disjoint windows (ordered pairs).

```
PYTHONPATH=. python scripts/brain_crosswindow_calibration.py \
  --runs-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --alphas 1e-4,3e-4,1e-3 \
  --out-csv reports/brain_crosswindow_calibration.csv \
  --out-json reports/brain_crosswindow_calibration_summary.json
```

(R15) **Brain-* baseline sanity table (strict vs relaxed operating points)** (paper refs: Appendix: baseline sanity checks at relaxed operating points (Brain-*)). Outputs:
`reports/brain_baseline_sanity_relaxed.csv`
`reports/brain_baseline_sanity_relaxed.json`
`reports/brain_baseline_sanity_relaxed_table.tex`
Notes: Reads per-window score/mask arrays under the Brain-* pilot run root.

```
PYTHONPATH=. python scripts/brain_baseline_sanity_table.py \
  --fair-matrix-json reports/fair_matrix_vnext_r3_localbaselines.json \
  --alphas 0.001,0.01,0.1 \
  --out-csv reports/brain_baseline_sanity_relaxed.csv \
  --out-json reports/brain_baseline_sanity_relaxed.json \
  --out-tex reports/brain_baseline_sanity_relaxed_table.tex
```

(R16) **Brain-* detector-component ablation (whitening vs band selectivity)** (paper refs: Table: brain_detector_ablation (detector-component ablation)). Outputs:
`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/*_mcsvd_det_whitened_power/(generatedbundles)`
`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/*_mcsvd_det_unwhitened_ratio/(generatedbundles)`
`reports/fair_matrix_detector_ablations.csv`
`reports/fair_matrix_detector_ablations.json`
`reports/brain_detector_ablation.csv`
`reports/brain_detector_ablation.json`
`reports/brain_detector_ablation_table.tex`
Notes: Ablates detector components on the identical MC–SVD residual: whitened total power (no band partition) and unwhitened matched-subspace ratio (R=I).

```
PYTHONPATH=. conda run -n stap-fus python scripts/fair_filter_comparison.py \
  --mode matrix --eval-score vnext \
  --matrix-regimes open,aliascontract,skullor \
  --matrix-seeds-open 1 --matrix-seeds-aliascontract 2 --matrix-seeds-skullor 2 \
```

```
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --matrix-use-profile \
  --matrix-mcsvd-energy-frac 0.90 --matrix-mcsvd-baseline-support full \
  --methods mcsvd,stap_det_whitened_power,stap_det_unwhitened_ratio,stap_full \
  --generated-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --autogen-missing --stap-device cuda \
  --out-csv reports/fair_matrix_detector_ablations.csv \
  --out-json reports/fair_matrix_detector_ablations.json
PYTHONPATH=. python scripts/brain_detector_ablation_table.py \
  --fair-matrix-json reports/fair_matrix_detector_ablations.json \
  --out-csv reports/brain_detector_ablation.csv \
  --out-json reports/brain_detector_ablation.json \
  --out-tex reports/brain_detector_ablation_table.tex
```

(R17) **Brain-* detector-swap fairness check (matched-subspace scoring on competing residuals)** (paper refs: Table: brain_detector_swap (paired residual detector swap)). Outputs:
`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/*_hosvd_stap_full/(generatedbundles)`
`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/*_rpca_stap_full/(generatedbundles)`
`reports/fair_matrix_detector_swap.csv`
`reports/fair_matrix_detector_swap.json`
`reports/brain_detector_swap.csv`
`reports/brain_detector_swap.json`
`reports/brain_detector_swap_table.tex`
Notes: Paired-residual comparison: each run generates both PD (score_base.npy) and STAP (score_stap_preka.npy) scores on the same RPCA/HOSVD residual for each window.

```
PYTHONPATH=. conda run -n stap-fus python scripts/fair_filter_comparison.py \
  --mode matrix --eval-score vnext \
  --matrix-regimes open,aliascontract,skullor \
  --matrix-seeds-open 1 --matrix-seeds-aliascontract 2 --matrix-seeds-skullor 2 \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --matrix-use-profile \
  --methods rpca_pair,rpca_stap,hosvd_pair,hosvd_stap \
  --generated-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --regen-runs --autogen-missing --stap-device cuda \
  --out-csv reports/fair_matrix_detector_swap.csv \
  --out-json reports/fair_matrix_detector_swap.json
PYTHONPATH=. python scripts/brain_detector_swap_table.py \
  --fair-matrix-json reports/fair_matrix_detector_swap.json \
  --out-csv reports/brain_detector_swap.csv \
  --out-json reports/brain_detector_swap.json \
  --out-tex reports/brain_detector_swap_table.tex
```

(R18) **Brain-* covariance-training trim ablation (self-training sensitivity)** (paper refs: Results: covariance-training contamination ablation (Brain-*)). Outputs:
`runs/pilot/fair_filter_matrix_pd_r3_localbaselines/*_mcsvd_covtrim_q05/(generatedbundles)`
`reports/fair_matrix_covtrim.csv`
`reports/fair_matrix_covtrim.json`
`reports/brain_cov_train_ablation.csv`
`reports/brain_cov_train_ablation.json`

`reports/brain_cov_train_ablation_table.tex`
Notes: Enables the ablation via replay_stap_from_run.py –stap-cov-trim-q (plumbed through fair_filter_comparison method key stap_covtrim_q05). This excludes the top 5% highest-energy Hankel snapshots per tile when estimating R_i (guard/trim style training contamination check).

```
PYTHONPATH=. conda run -n stap-fus python scripts/fair_filter_comparison.py \
  --mode matrix --eval-score vnext \
  --matrix-regimes open,aliascontract,skullor \
  --matrix-seeds-open 1 --matrix-seeds-aliascontract 2 --matrix-seeds-skullor 2 \
  --window-length 64 --window-offsets 0,64,128,192,256 \
  --matrix-use-profile \
  --matrix-mcsvd-energy-frac 0.90 --matrix-mcsvd-baseline-support full \
  --methods stap_full,stap_covtrim_q05 \
  --generated-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --autogen-missing --stap-device cuda \
  --out-csv reports/fair_matrix_covtrim.csv \
  --out-json reports/fair_matrix_covtrim.json
PYTHONPATH=. python scripts/brain_cov_train_ablation_table.py \
  --runs-root runs/pilot/fair_filter_matrix_pd_r3_localbaselines \
  --alphas 1e-4,3e-4,1e-3 \
  --out-csv reports/brain_cov_train_ablation.csv \
  --out-json reports/brain_cov_train_ablation.json \
  --out-tex reports/brain_cov_train_ablation_table.tex
```

(R19) **Brain-OpenSkull fixed-profile parameter sensitivity sweep (diag-load, tile size, Lt)** Paper refs:

Appendix: fixed-profile parameter sensitivity (Brain-OpenSkull)

Results: covariance-training contamination ablation (Brain-*)

Outputs:

`runs/pilot/brain_openskull_profile_sensitivity_r1/(generatedbundles)`

`reports/brain_openskull_profile_sensitivity.csv`

`reports/brain_openskull_profile_sensitivity.json`

`figs/paper/brain_openskull_profile_sensitivity.pdf`
Notes: Runs 1D sweeps (vary one parameter at a time) around the frozen Brain-OpenSkull operating profile, reporting within-window strict-tail TPR medians/IQR over five disjoint windows. Sweep ranges are configurable via –diag-loads/–tile-sizes/–lts.

```
PYTHONPATH=. conda run -n stap-fus python scripts/brain_openskull_profile_sensitivity.py \
  --autogen-missing --stap-device cuda
```

(R20) **Brain-* strict-tail collapse visual (representative window; baseline vs STAP)** (paper refs: Appendix: baseline sanity checks at relaxed operating points (Brain-*)).

Outputs:

`figs/paper/brain_tail_collapse_visual.pdf`
Notes: Uses one representative Brain-OpenSkull window to visualize background-tail dominance (baseline PD) and tail reshaping (STAP).

```
ROOT="runs/pilot/fair_filter_matrix_pd_r3_localbaselines/open_seed1_mcsvd_full"
BUNDLE_DIR="$ROOT/pw_7.5MHz_5ang_5ens_64T_seed1_win0_off0"
PYTHONPATH=. python scripts/fig_brain_tail_collapse_visual.py \
  --bundle-dir "$BUNDLE_DIR" \
  --alphas 0.1,0.01,1e-3 \
  --out figs/paper/brain_tail_collapse_visual.pdf
```

(R21) **Knowledge-aided prior falsifiability ablation (STAP-only vs contract vs forced)** (paper refs: KA evaluation discipline). Outputs:

reports/ka_v2_ablation.csv
reports/ka_v2_ablation.json

```
PYTHONPATH=. python scripts/ka_contract_v2_ablation.py \
  --pilot runs/pilot/r4c_kwave_hab_contract_seed2_v2 \
  --profile Brain-AliasContract \
  --out-root runs/ablation/ka_v2_falsifiability \
  --stap-device cpu \
  --window-length 64 \
  --window-offsets 0 64 128 192 256 \
  --summary-csv reports/ka_v2_ablation.csv \
  --summary-json reports/ka_v2_ablation.json
```

(R22) **Knowledge-aided prior positive-control ablation (OpenSkull shallow-alias + high-rank ghosts)** (paper refs: KA evaluation discipline (positive control)). Outputs:
reports/ka_v2_ablation_openskull_shallowalias_e50.csv
reports/ka_v2_ablation_openskull_shallowalias_e50.json
Notes: Synthetic, replay-only KA-positive regime used as a contract-mechanics positive control; not a realism benchmark.

```
PYTHONPATH=. python scripts/ka_contract_v2_ablation.py \
  --pilot runs/pilot/r4c_kwave_seed1 \
  --profile Brain-OpenSkull \
  --out-root runs/ablation/openskull_shallowalias_e50 \
  --stap-device cpu \
  --window-length 64 \
  --window-offsets 0 64 128 192 256 \
  --summary-csv reports/ka_v2_ablation_openskull_shallowalias_e50.csv \
  --summary-json reports/ka_v2_ablation_openskull_shallowalias_e50.json \
  --replay-extra \
    --svd-energy-frac 0.50 \
    --flow-doppler-min-hz 60 --flow-doppler-max-hz 180 \
    --flow-doppler-noise-amp 60 --flow-doppler-noise-mode fft_band \
    --bg-alias-hz 650 --bg-alias-fraction 0.7 \
    --bg-alias-depth-min-frac 0.12 --bg-alias-depth-max-frac 0.28 \
    --bg-alias-jitter-hz 35 \
    --flow-mask-suppress-alias-depth \
    --clutter-beta 1.0 --clutter-snr-db 20 \
    --clutter-depth-min-frac 0.20 --clutter-depth-max-frac 0.95 \
    --aperture-phase-std 0.8 --aperture-phase-corr-len 14 \
    --bg-alias-highrank-mode gw_reverb_add \
    --bg-alias-highrank-coverage 0.3 --bg-alias-highrank-amp 0.3 \
    --bg-alias-highrank-margin-px 3 \
    --bg-alias-highrank-freq-jitter-hz 25 \
    --bg-alias-highrank-drift-step-hz 12 \
    --bg-alias-highrank-pf-leak-eta 0.0 \
    --ka-score-contract-v2-mode auto
```

(R23) **Mace PD-only holdout alias-gate evaluation (label-free thresholds)** (paper refs: Table: Mace hold-out alias gate). Outputs:
reports/mace_alias_gate_holdout.csv
reports/mace_alias_gate_holdout.json

```
PYTHONPATH=. python scripts/mace_alias_gate_holdout.py \
  --out-csv reports/mace_alias_gate_holdout.csv \
  --out-json reports/mace_alias_gate_holdout.json
```

(R24) **Mace pixel-level vascular-atlas check (independent structural labels)** (paper refs:

Mace section: vascular atlas pixel-level check). Outputs:

`reports/mace_vascular_pixel_eval.csv`

`reports/mace_vascular_pixel_eval.json`

Notes: Maps atlas.Vascular into each scan plane using Transformation.mat; evaluates vascular vs non-vascular pixels (structural, not clinical efficacy).

```
PYTHONPATH=. python scripts/mace_vascular_pixel_eval.py \
  --out-csv reports/mace_vascular_pixel_eval.csv \
  --out-json reports/mace_vascular_pixel_eval.json
```

(R25) **Mace atlas-alignment sanity overlay figure** (paper refs: Appendix: Mace atlas alignment sanity check). Outputs:

`figs/paper/mace_atlas_overlay.png`

Notes: Overlays atlas ROIs and atlas.Vascular contours on the anatomical reference volume for representative planes (sanity check of Transformation.mat alignment).

```
PYTHONPATH=. python scripts/mace_atlas_overlay_fig.py \
  --scan-name scan_anatomy \
  --plane-indices 5 10 15 \
  --out-png figs/paper/mace_atlas_overlay.png
```

(R26) **Telemetry regime comparison (sim vs real; contract-v2 scalars)** (paper refs: Figure: `telemetry_regime_compare.png`). Outputs:

`figs/paper/telemetry_regime_compare.png`

Notes: Overlays PfPeakFrac(flow), guard_q90, and iqr_alias_bg across Brain-* simulations, Shin real IQ, and Mace PD-only planes; intended as telemetry anchoring (not a claim that sims match real data).

```
# Sample Brain-* replay bundles (write meta.json with ka_contract_v2 telemetry):
PYTHONPATH=. python scripts/replay_stap_from_run.py \
  --src runs/pilot/r4c_kwave_seed1 \
  --out runs/telemetry_regime_compare/brain_open_seed1 \
  --profile Brain-OpenSkull \
  --time-window-length 64 \
  --time-window-offset 0 --time-window-offset 64 --time-window-offset 128 \
  --time-window-offset 192 --time-window-offset 256
PYTHONPATH=. python scripts/replay_stap_from_run.py \
  --src runs/pilot/r4c_kwave_hab_contract_seed2_v2 \
  --out runs/telemetry_regime_compare/brain_aliascontract_seed2 \
  --profile Brain-AliasContract \
  --time-window-length 64 \
  --time-window-offset 0 --time-window-offset 64 --time-window-offset 128 \
  --time-window-offset 192 --time-window-offset 256
PYTHONPATH=. python scripts/replay_stap_from_run.py \
  --src runs/pilot/r4c_kwave_hab_v3_skull_seed2_v2 \
  --out runs/telemetry_regime_compare/brain_skullor_seed2 \
  --profile Brain-SkullOR \
  --time-window-length 64 \
  --time-window-offset 0 --time-window-offset 64 --time-window-offset 128 \
  --time-window-offset 192 --time-window-offset 256

# Generate the overlay histogram figure:
# (reads Shin bundles and Mace CSV from other artifacts)
PYTHONPATH=. python scripts/telemetry_regime_compare_fig.py \
  --out-png figs/paper/telemetry_regime_compare.png
```

(R27) **Mace PD-only prior dashboard (plane sweep + paper figure)** (paper refs: Figure: `mace_pdonly_contract_v2_dashboard.png`). Outputs:

reports/mace_pdonly_contract_v2.csv
figs/paper/mace_pdonly_contract_v2_dashboard.png

```
PYTHONPATH=. python scripts/mace_pdonly_contract_v2_sweep.py \
  --out-csv reports/mace_pdonly_contract_v2.csv

PYTHONPATH=. python scripts/mace_pdonly_contract_v2_dashboard_fig.py \
  --in-csv reports/mace_pdonly_contract_v2.csv \
  --out-png figs/paper/mace_pdonly_contract_v2_dashboard.png
```

(R28) **Shin all-clips prior-regime telemetry (n=80; telemetry-only)** (paper refs: Shin section: all-clips telemetry audit). Outputs:
reports/shin_ratbrain_shinU_e970_telemetry_only_0_128_all80.csv
reports/shin_ratbrain_shinU_e970_telemetry_only_0_128_all80_summary.json

```
PYTHONPATH=. python -m scripts.shin_ratbrain_allclips_contract_hygiene \
  --extract-from-zip --no-run-stap --no-score-ka-v2 \
  --profile U --svd-energy-frac 0.97 --frames-list 0:128 \
  --out-root runs/shin_ratbrain_shinU_e970_telemetry_only_0_128_all80 \
  --out-csv reports/shin_ratbrain_shinU_e970_telemetry_only_0_128_all80.csv \
  --out-json reports/shin_ratbrain_shinU_e970_telemetry_only_0_128_all80_summary.json
```

(R29) **Shin brainlike motion sweep (10 clips) + aggregate plots** Paper refs:
Table: Shin motion endpoints
Figure: shin_motion_brainlike_batch_U_agg.png
Figure: shin_motion_brainlike_compare_reg1.png
Figure: shin_motion_brainlike_compare.png
Outputs:
reports/shin_motion_brainlike_batch_U/*.csv
reports/shin_motion_brainlike_batch_U_agg.csv
figs/paper/shin_motion_brainlike_batch_U_agg.png
figs/paper/shin_motion_brainlike_compare_reg1.png
figs/paper/shin_motion_brainlike_compare.png
Notes: Window-specific batch runs (64:192, 122:250) follow the same pattern.

```
# Per-clip sweeps (canonical window 0:128; KA disabled):
for f in IQData{001,002,003,004,005,010,020,040,060,080}.dat; do
  PYTHONPATH=. python -m scripts.shin_ratbrain_motion_sweep \
    --iq-file "$f" --frames 0:128 --profile U --svd-energy-frac 0.97 \
    --motion-kind brainlike --amp-px-list 0,0.5,1,2 \
    --no-score-ka-v2 --out-root runs/shin_motion_brainlike_batch_U \
    --out-csv "reports/shin_motion_brainlike_batch_U/${f%.dat}.csv" \
    --out-png "reports/shin_motion_brainlike_batch_U/${f%.dat}.png";
done

# Aggregate curve (writes CSV + plot):
PYTHONPATH=. python -m scripts.shin_ratbrain_motion_aggregate \
  --in-dir reports/shin_motion_brainlike_batch_U \
  --out-csv reports/shin_motion_brainlike_batch_U_agg.csv \
  --out-png reports/shin_motion_brainlike_batch_U_agg.png

# Compare across windows / reg ablations (writes comparison plots):
PYTHONPATH=. python -m scripts.shin_ratbrain_motion_compare \
  --curve "0:128=reports/shin_motion_brainlike_batch_U_agg.csv" \
  --curve "64:192=reports/shin_motion_brainlike_batch_U_f64_192_reg1_agg.csv" \
  --curve "122:250=reports/shin_motion_brainlike_batch_U_f122_250_reg1_agg.csv" \
```

```
  --out-png reports/shin_motion_brainlike_compare_reg1.png
cp -f \
  reports/shin_motion_brainlike_compare_reg1.png \
  figs/paper/shin_motion_brainlike_compare_reg1.png
```

(R30) **Shin motion crop/alignment sensitivity (post-hoc)** (paper refs: Shin motion section: crop sensitivity note). Outputs:
`reports/shin_motion_brainlike_batch_U_crop_sensitivity.csv`

```
PYTHONPATH=. python -m scripts.shin_ratbrain_motion_sensitivity_posthoc \
  --run-root runs/shin_motion_brainlike_batch_U \
  --amps 0,0.5,1,2 \
  --crop-margins 0,4,8,12 \
  --align-maps-list 0,1 \
  --out-csv reports/shin_motion_brainlike_batch_U_crop_sensitivity.csv
```

(R31) **Leading structural-fidelity figure (Gammex phantom; matched-FPR decision differences)** (paper refs: Figure 1: `leading_structural_fidelity_gammex.pdf`). Outputs:
`figs/paper/leading_structural_fidelity_gammex.pdf`
Notes: Generates a single-frame qualitative panel using the structurally labeled tube mask; the script writes a temporary one-frame acceptance bundle under runs/_tmp_leading_structural_fidelity/.

```
SEQ_DIR="data/twinkling_artifact/Flow in Gammex phantom"
SEQ_DIR="$SEQ_DIR/Flow in Gammex phantom (along - linear probe)"
PYTHONPATH=. conda run -n stap-fus python scripts/fig_leading_structural_fidelity.py \
  --seq-dir "$SEQ_DIR" \
  --frame-idx 0 --mask-ref-frames 0:10 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 6 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --fd-span-mode psd --feasibility-mode legacy \
  --svd-keep-min 2 --svd-keep-max 17 \
  --flow-band-hz 150 450 --alias-band-hz 700 1200 \
  --fpr 1e-2 \
  --stap-device auto \
  --out figs/paper/leading_structural_fidelity_gammex.pdf
```

(R32) **Twinkling decode sanity (RawBCF -¿ B-mode/CFM)** (paper refs: Twinkling section: RawBCF decode sanity). Outputs:
`reports/twinkling_decode_sanity/*/decode_report.json`

```
SEQ_DIR="data/twinkling_artifact/Flow in Gammex phantom"
SEQ_DIR="$SEQ_DIR/Flow in Gammex phantom (along - linear probe)"
PYTHONPATH=. python scripts/twinkling_decode_sanity.py \
  --seq-dir "$SEQ_DIR" \
  --frame-idx 0 \
  --out-dir reports/twinkling_decode_sanity
```

(R33) **Gammex flow phantom structural ROC (along-linear17, PRF=2500)** Paper refs: Twinkling section: Table (structural ROC)
Figure: `twinkling_gammex_along_mask_overlay.png`
Outputs:
`runs/real/twinkling_gammex_alonglinear17_prf2500_str6_msd_ka/`
`reports/twinkling_gammex_alonglinear17_prf2500_str6_msd_ka_with_baselines.csv`

reports/twinkling_gammex_alonglinear17_prf2500_str6_
msd_ka_with_baselines_summary.json
figs/paper/twinkling_gammex_along_mask_overlay.png
Notes: Along-linear uses stride 6; this matches the fixed geometry-only stride policy (largest stride with ¿=500 tiles).

```
SEQ_DIR="data/twinkling_artifact/Flow in Gammex phantom"
SEQ_DIR="$SEQ_DIR/Flow in Gammex phantom (along - linear probe)"
OUT_ROOT="runs/real/twinkling_gammex_alonglinear17_prf2500_str6_msd_ka"
MASK_GLOB=$OUT_ROOT/*__mask_debug

PYTHONPATH=. python scripts/twinkling_make_bundles.py \
  --seq-dir "$SEQ_DIR" \
  --out-root "$OUT_ROOT" \
  --frames 0:85 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 6 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 17 \
  --score-mode msd --score-ka-v2-enable --score-ka-v2-mode auto

PYTHONPATH=. python scripts/twinkling_eval_structural.py \
  --root "$OUT_ROOT" \
  --out-csv reports/twinkling_gammex_alonglinear17_prf2500_str6_msd_ka_with_baselines.csv \
  --out-summary-json \
    reports/twinkling_gammex_alonglinear17_prf2500_str6_msd_ka_with_baselines_summary.json

PYTHONPATH=. python scripts/fig_twinkling_mask_overlay.py \
  --bmode $MASK_GLOB/bmode_roi_ref_norm.png \
  --mask-flow $MASK_GLOB/mask_flow.png \
  --mask-bg $MASK_GLOB/mask_bg.png \
  --out figs/paper/twinkling_gammex_along_mask_overlay.png \
  --min-width 1200 --dpi 300
```

(R34) **Gammex flow phantom structural ROC (across-linear17, PRF=2500)** Paper refs:
Twinkling section: Table (structural ROC)
Figure: twinkling_gammex_across_mask_overlay.png
Outputs:
runs/real/twinkling_gammex_across17_prf2500_str4_msd_ka/
reports/twinkling_gammex_across17_prf2500_str4_msd_ka_with_baselines.csv
reports/twinkling_gammex_across17_prf2500_str4_msd_
ka_with_baselines_summary.json
figs/paper/twinkling_gammex_across_mask_overlay.png
Notes: Across-linear uses stride 4; this matches the fixed geometry-only stride policy (largest stride with ¿=500 tiles).

```
SEQ_DIR="data/twinkling_artifact/Flow in Gammex phantom"
SEQ_DIR="$SEQ_DIR/Flow in Gammex phantom (across - linear probe)"
DAT_PATH="$SEQ_DIR/RawBCFCine_08062017_145434_17.dat"
PAR_PATH="$SEQ_DIR/RawBCFCine_08062017_145434_17.par"
OUT_ROOT="runs/real/twinkling_gammex_across17_prf2500_str4_msd_ka"
MASK_GLOB=$OUT_ROOT/*__mask_debug

PYTHONPATH=. python scripts/twinkling_make_bundles.py \
  --seq-dir "$SEQ_DIR" \
  --dat-path "$DAT_PATH" \
```

```
  --par-path "$PAR_PATH" \
  --out-root "$OUT_ROOT" \
  --frames 0:200 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 4 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 17 \
  --score-mode msd --score-ka-v2-enable --score-ka-v2-mode auto

PYTHONPATH=. python scripts/twinkling_eval_structural.py \
  --root "$OUT_ROOT" \
  --out-csv reports/twinkling_gammex_across17_prf2500_str4_msd_ka_with_baselines.csv \
  --out-summary-json \
    reports/twinkling_gammex_across17_prf2500_str4_msd_ka_with_baselines_summary.json

PYTHONPATH=. python scripts/fig_twinkling_mask_overlay.py \
  --bmode $MASK_GLOB/bmode_roi_ref_norm.png \
  --mask-flow $MASK_GLOB/mask_flow.png \
  --mask-bg $MASK_GLOB/mask_bg.png \
  --out figs/paper/twinkling_gammex_across_mask_overlay.png \
  --min-width 780 --dpi 300
```

(R35) **Gammex within-ensemble motion ladder (along-linear17; PRF=2500)** Paper refs: Twinkling section: Figure (within-ensemble motion ladder)
Figure: `twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_rw_ci_v2.png`
Figure: `twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_step_ci_v2.png`
Outputs:
`runs/real/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_rw_v2/`
`runs/real/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_step_v2/`
`reports/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_rw_ci_v2.json`
`reports/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_step_ci_v2.json`
`figs/paper/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_rw_ci_v2.png`
`figs/paper/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_ladder_step_ci_v2.png`

```
SEQ_DIR="data/twinkling_artifact/Flow in Gammex phantom"
SEQ_DIR="$SEQ_DIR/Flow in Gammex phantom (along - linear probe)"
REF_ROOT="runs/real/twinkling_gammex_alonglinear17_prf2500_str6_msd_nomotion_ref_fast_f020"
OUT_RW="runs/real/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_rw_v2"
OUT_STEP="runs/real/twinkling_gammex_alonglinear17_prf2500_within_ensemble_motion_step_v2"
REP_DIR="reports"
FIG_DIR="figs/paper"
BASE="twinkling_gammex_alonglinear17_prf2500"
SUF_RW="within_ensemble_motion_ladder_rw_ci_v2"
SUF_STEP="within_ensemble_motion_ladder_step_ci_v2"
RW_PREFIX="$REP_DIR/${BASE}_${SUF_RW}"
```

```
STEP_PREFIX="$REP_DIR/${BASE}_${SUF_STEP}"
RW_FIG="$FIG_DIR/${BASE}_${SUF_RW}.png"
STEP_FIG="$FIG_DIR/${BASE}_${SUF_STEP}.png"

# Amp=0 reference bundles for equivalence checks (20 frames):
PYTHONPATH=. python scripts/twinkling_make_bundles.py \
  --seq-dir "$SEQ_DIR" \
  --out-root "$REF_ROOT" \
  --frames 0:20 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 6 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 17 \
  --score-mode msd

# Within-ensemble motion sweeps (85 frames; rw and step):
PYTHONPATH=. python scripts/twinkling_gammex_within_ensemble_motion_sweep.py \
  --seq-dir "$SEQ_DIR" \
  --out-root "$OUT_RW" \
  --frames 0:85 --mask-ref-frames 0:20 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 6 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 17 \
  --score-mode msd \
  --motion-kind rw --amp-px-list 0,0.25,0.5,1,1.5,2
PYTHONPATH=. python scripts/twinkling_gammex_within_ensemble_motion_sweep.py \
  --seq-dir "$SEQ_DIR" \
  --out-root "$OUT_STEP" \
  --frames 0:85 --mask-ref-frames 0:20 \
  --prf-hz 2500 --Lt 16 \
  --tile-hw 8 8 --tile-stride 6 \
  --cov-estimator tyler_pca --diag-load 0.07 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 17 \
  --score-mode msd \
  --motion-kind step --amp-px-list 0,0.25,0.5,1,1.5,2

# Aggregate + bootstrap CIs + plots:
PYTHONPATH=. python scripts/twinkling_eval_motion_ladder.py \
  --root "$OUT_RW" \
  --out-csv "${RW_PREFIX}.csv" \
  --out-json "${RW_PREFIX}.json" \
  --out-png "$RW_FIG" \
  --bootstrap 1000 --bootstrap-seed 0 \
  --amp0-ref-root "$REF_ROOT" \
  --amp0-ref-frames 0:20 --amp0-ref-tol 5e-3
PYTHONPATH=. python scripts/twinkling_eval_motion_ladder.py \
  --root "$OUT_STEP" \
  --out-csv "${STEP_PREFIX}.csv" \
  --out-json "${STEP_PREFIX}.json" \
  --out-png "$STEP_FIG" \
  --bootstrap 1000 --bootstrap-seed 0 \
  --amp0-ref-root "$REF_ROOT" \
  --amp0-ref-frames 0:20 --amp0-ref-tol 5e-3
```

(R36) **Twinkling calculi KA hygiene + contract telemetry (calcifications; PRF approx**

**500)** Paper refs:
Twinkling section: KA hygiene on calculi
Figure: `twinkling_calculi_tail_example.png`
Outputs:
`runs/real/twinkling_calculi_calcifications_prf500_str4_Lt8_msd_ka_scm_dl015_f050/`
`reports/twinkling_calculi_calcifications_prf500_str4_Lt8_msd_ka_scm_dl015_f050_roc.csv`
`reports/twinkling_calculi_calcifications_prf500_str4_Lt8_msd_ka_scm_dl015_f050_summary.json`
Notes: Calculi runs use proxy masks (not tube structural masks). Used for contract/tail-hygiene instrumentation (not clinical detection claims).

```
BASE_DIR="data/twinkling_artifact/Twinkling artifact on calculi"
SEQ_DIR="$BASE_DIR/Twinkling and Flash artifacts on artificial calculi"
RUN_TAG="twinkling_calculi_calcifications_prf500_str4_Lt8_msd_ka_scm_dl015_f050"
OUT_ROOT="runs/real/${RUN_TAG}"
REP_PREFIX="reports/${RUN_TAG}"

PYTHONPATH=. python scripts/twinkling_make_bundles.py \
  --seq-dir "$SEQ_DIR" \
  --out-root "$OUT_ROOT" \
  --frames 0:50 \
  --prf-hz 500 --Lt 8 \
  --tile-hw 8 8 --tile-stride 4 \
  --cov-estimator scm --diag-load 0.15 \
  --baseline-type svd_bandpass --svd-keep-min 2 --svd-keep-max 9 \
  --score-mode msd --score-ka-v2-enable --score-ka-v2-mode auto \
  --mask-mode none

PYTHONPATH=. python scripts/twinkling_eval_structural.py \
  --root "$OUT_ROOT" \
  --out-csv "${REP_PREFIX}_roc.csv" \
  --out-summary-json "${REP_PREFIX}_summary.json"
```

(R37) **Twinkling contract figure pack (states/reasons + calculi tail example)** Paper refs:
Twinkling section: Figure (twinkling_contract_states_reasons.png)
Twinkling section: Figure (twinkling_calculi_tail_example.png)
Outputs:
`figs/paper/twinkling_contract_states_reasons.png`
`figs/paper/twinkling_calculi_tail_example.png`
`reports/twinkling_contract_figpack.json`

```
ALONG_BASE="twinkling_gammex_alonglinear17_prf2500_str6_msd_ka_with_baselines"
ACROSS_BASE="twinkling_gammex_across17_prf2500_str4_msd_ka_with_baselines"
ALONG_SUM="reports/${ALONG_BASE}_summary.json"
ACROSS_SUM="reports/${ACROSS_BASE}_summary.json"
CALC_TAG="twinkling_calculi_calcifications_prf500_str4_Lt8_msd_ka_scm_dl015_f050"
CALC_SUM="reports/${CALC_TAG}_summary.json"
CALC_ROOT="runs/real/${CALC_TAG}"
OUT_STATE="figs/paper/twinkling_contract_states_reasons.png"
OUT_EX="figs/paper/twinkling_calculi_tail_example.png"
OUT_JSON="reports/twinkling_contract_figpack.json"

PYTHONPATH=. python scripts/twinkling_contract_figpack.py \
```

```
  --along-summary "$ALONG_SUM" \
  --across-summary "$ACROSS_SUM" \
  --calculi-summary "$CALC_SUM" \
  --calculi-root "$CALC_ROOT" \
  --out-state-reason-png "$OUT_STATE" \
  --out-calculi-example-png "$OUT_EX" \
  --out-json "$OUT_JSON"
```

(R38) **ULM 7883227 one-time baseline sweep (MC-SVD energy fraction)** (paper refs: ULM section: baseline calibration (frozen svd_energy_frac=0.975)). Outputs: `runs/real/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128/` `reports/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128.csv` `reports/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128.json`

```
PYTHONPATH=. python scripts/ulm_zenodo_7883227_baseline_sweep.py \
  --block-ids 1,2,3 --frames 0:128 \
  --svd-energy-frac-list 0.90,0.95,0.97,0.975,0.98,0.99 \
  --profile ULM --prf-hz 1000 --Lt 64 \
  --tile-h 8 --tile-w 8 --tile-stride 3 \
  --reg-enable --reg-subpixel 4 \
  --no-run-stap --bg-tail-fpr 0.001 --stability-split \
  --cache-dir tmp/ulm_zenodo_7883227 \
  --out-root runs/real/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128 \
  --out-csv reports/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128.csv \
  --out-json reports/ulm7883227_baseline_mcsvd_energy_sweep_ULM_0000_0128.json
```

(R39) **ULM 7883227 motion sweeps (brainlike + elastic; fairness alignment + degeneracy)** Paper refs:
ULM section: Figure (ulm7883227_motion_sweep_*_brainlike_e975.png)
ULM section: Figure (ulm7883227_motion_sweep_*_elastic_e975.png)
Outputs:
`reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.csv`
`reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.json`
`figs/paper/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.png`
`reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.csv`
`reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.json`
`figs/paper/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.png`

```
PYTHONPATH=. python scripts/ulm_zenodo_7883227_motion_sweep.py \
  --block-ids 1,2,3 --frames 0:128 \
  --profile ULM --prf-hz 1000 --Lt 64 \
  --tile-h 8 --tile-w 8 --tile-stride 3 \
  --baseline-type mc_svd --svd-energy-frac 0.975 \
  --reg-enable --reg-subpixel 4 \
  --cov-estimator scm --diag-load 0.07 \
  --motion-kind brainlike --amp-px-list 0,0.5,1,2,3 \
  --out-root runs/real/ulm7883227_motion_sweep_ULM_brainlike_e975 \
  --out-csv reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.csv \
  --out-json reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.json \
  --out-png figs/paper/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_brainlike_e975.png
PYTHONPATH=. python scripts/ulm_zenodo_7883227_motion_sweep.py \
  --block-ids 1,2,3 --frames 0:128 \
  --profile ULM --prf-hz 1000 --Lt 64 \
  --tile-h 8 --tile-w 8 --tile-stride 3 \
```

```
  --baseline-type mc_svd --svd-energy-frac 0.975 \
  --reg-enable --reg-subpixel 4 \
  --cov-estimator scm --diag-load 0.07 \
  --motion-kind elastic --amp-px-list 0,0.5,1,2,3 \
  --out-root runs/real/ulm7883227_motion_sweep_ULM_elastic_e975 \
  --out-csv reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.csv \
  --out-json reports/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.json \
  --out-png figs/paper/ulm7883227_motion_sweep_ULM_blocks1_3_0000_0128_elastic_e975.png
```

(R40) **SIMUS/PyMUST moving-scatterer sanity link + bundle contract (micro + alias; non-performance claims)** Paper refs:
Simulation spec: moving-scatterer physical Doppler regime (SIMUS)
Appendix (QC): physical-doppler sanity-link telemetry
Outputs:
`reports/physdoppler_sanity_link/simus_micro_paper_seed0_vs_shin_gammex_summary.json`
`reports/physdoppler_sanity_link/simus_micro_paper_seed0_vs_shin_gammex_table.json`
`reports/physdoppler_sanity_link/simus_alias_paper_seed0_vs_shin_gammex_summary.json`
`reports/physdoppler_sanity_link/simus_alias_paper_seed0_vs_shin_gammex_table.json`
`reports/simus_baseline_compare/simus_pymust_paper_seed0_compare.csv`
`reports/simus_baseline_compare/simus_pymust_paper_seed0_compare.json`
`reports/simus_contract/simus_pymust_paper_micro_seed0_hab_contract.txt`
`reports/simus_contract/simus_pymust_paper_alias_seed0_hab_contract.txt`
`runs/sim/simus_pymust_paper_micro_seed0/bundle/simus_pymust_paper_micro_seed0/`
`runs/sim/simus_pymust_paper_alias_seed0/bundle/simus_pymust_paper_alias_seed0/`
`runs/sim_eval/simus_baseline_compare_r1/`
Notes: SIMUS/PyMUST runs are used as a moving-scatterer credibility anchor. Sanity-link summaries compare PSD-band / coherence / low-rank proxies against open real IQ; contract checks verify bundle integrity and KA-friendly regime telemetry. No performance claims are made from these comparisons.

```
PYTHONPATH=. conda run -n stap-fus python sim/simus/pilot_pymust_simus.py \
  --out runs/sim/simus_pymust_paper_micro_seed0 --preset microvascular_like --tier paper --seed 0 --skip-bund
PYTHONPATH=. conda run -n stap-fus python sim/simus/pilot_pymust_simus.py \
  --out runs/sim/simus_pymust_paper_alias_seed0 --preset alias_stress --tier paper --seed 0 --skip-bundle

PYTHONPATH=. conda run -n stap-fus python scripts/icube_make_bundle.py \
  --run runs/sim/simus_pymust_paper_micro_seed0 --stap-device cpu
PYTHONPATH=. conda run -n stap-fus python scripts/icube_make_bundle.py \
  --run runs/sim/simus_pymust_paper_alias_seed0 --stap-device cpu

PYTHONPATH=. conda run -n stap-fus python scripts/hab_contract_check.py \
  runs/sim/simus_pymust_paper_micro_seed0/bundle/simus_pymust_paper_micro_seed0 \
  2>&1 | tee reports/simus_contract/simus_pymust_paper_micro_seed0_hab_contract.txt
PYTHONPATH=. conda run -n stap-fus python scripts/hab_contract_check.py \
  runs/sim/simus_pymust_paper_alias_seed0/bundle/simus_pymust_paper_alias_seed0 \
  2>&1 | tee reports/simus_contract/simus_pymust_paper_alias_seed0_hab_contract.txt
```

```
PYTHONPATH=. conda run -n stap-fus python scripts/physical_doppler_sanity_link.py \
  --sim-run runs/sim/simus_pymust_paper_micro_seed0 \
  --shin-root data/shin_zenodo_10711806/ratbrain_fig3_raw --shin-iq-file IQData001.dat \
  --shin-frames 0:128 --shin-prf-hz 1000 \
  --gammex-seq-root "data/twinkling_artifact/Flow in Gammex phantom" \
  --gammex-frames-along 0 --gammex-frames-across 0 --gammex-prf-hz 2500 \
  --gammex-mask-mode bmode_tube --gammex-mask-ref-frames 0:6 \
  --pf 30 250 --pg 250 400 --pa-lo 400 --tile-hw 8 8 --tile-stride 3 \
  --tag simus_micro_paper_seed0_vs_shin_gammex
PYTHONPATH=. conda run -n stap-fus python scripts/physical_doppler_sanity_link.py \
  --sim-run runs/sim/simus_pymust_paper_alias_seed0 \
  --shin-root data/shin_zenodo_10711806/ratbrain_fig3_raw --shin-iq-file IQData001.dat \
  --shin-frames 0:128 --shin-prf-hz 1000 \
  --gammex-seq-root "data/twinkling_artifact/Flow in Gammex phantom" \
  --gammex-frames-along 0 --gammex-frames-across 0 --gammex-prf-hz 2500 \
  --gammex-mask-mode bmode_tube --gammex-mask-ref-frames 0:6 \
  --pf 30 250 --pg 250 400 --pa-lo 400 --tile-hw 8 8 --tile-stride 3 \
  --tag simus_alias_paper_seed0_vs_shin_gammex

PYTHONPATH=. conda run -n stap-fus python scripts/icube_baseline_compare.py \
  --run runs/sim/simus_pymust_paper_micro_seed0 \
  --run runs/sim/simus_pymust_paper_alias_seed0 \
  --out-root runs/sim_eval/simus_baseline_compare_r1 \
  --out-csv reports/simus_baseline_compare/simus_pymust_paper_seed0_compare.csv \
  --out-json reports/simus_baseline_compare/simus_pymust_paper_seed0_compare.json \
  --tag clinical_like --stap-device cpu
```

(R41) **Paper build modes (paper vs supplement vs full)** (paper refs: Build packaging (Phase 7)). Outputs:

`stap_fus_paper.pdf`

`stap_fus_supplement.pdf`

`stap_fus_methodology.pdf`

```
pdflatex stap_fus_methodology.tex   # full (default)
pdflatex stap_fus_paper.tex         # paper-only (no appendices)
pdflatex stap_fus_supplement.tex    # supplement-only (appendices only)
```

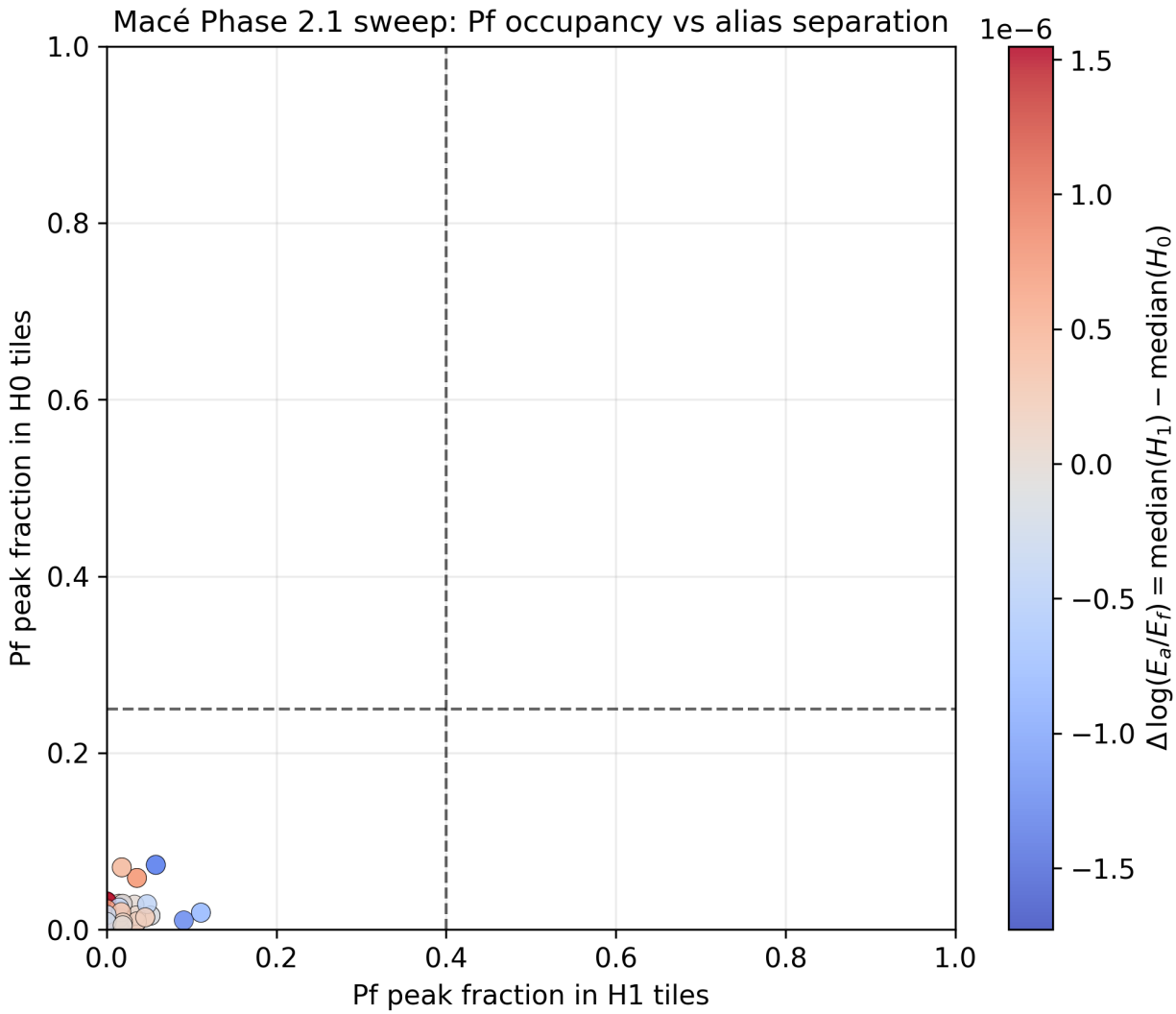


Figure 11: Macé systematic band-energy summary (six files; 84 planes; four unique acquisitions in this loader): Pf peak occupancy vs alias separation under a fixed hemodynamic STAP configuration (tiles $8 \times 8$, stride 3; $P_f = [0, 0.5]$ Hz, guard $[0.5, 1.0]$ Hz, $P_a = [1.0, 3.0]$ Hz). Each point is one (scan, plane) instance; the x-axis is the Pf peak fraction in the VIS/SC/LGd atlas stratum and the y-axis the Pf peak fraction in the CP/CA/DG stratum (offline, descriptive), and color encodes $\Delta \log(E_a/E_f) = \text{median}(\text{VIS/SC/LGd}) - \text{median}(\text{CP/CA/DG})$ (negative means alias ratios are lower in VIS/SC/LGd). Dashed lines indicate an offline descriptive threshold pair ($x > 0.4$, $y < 0.25$); they are not used for the held-out, label-free operating-point evaluation in Table 5.

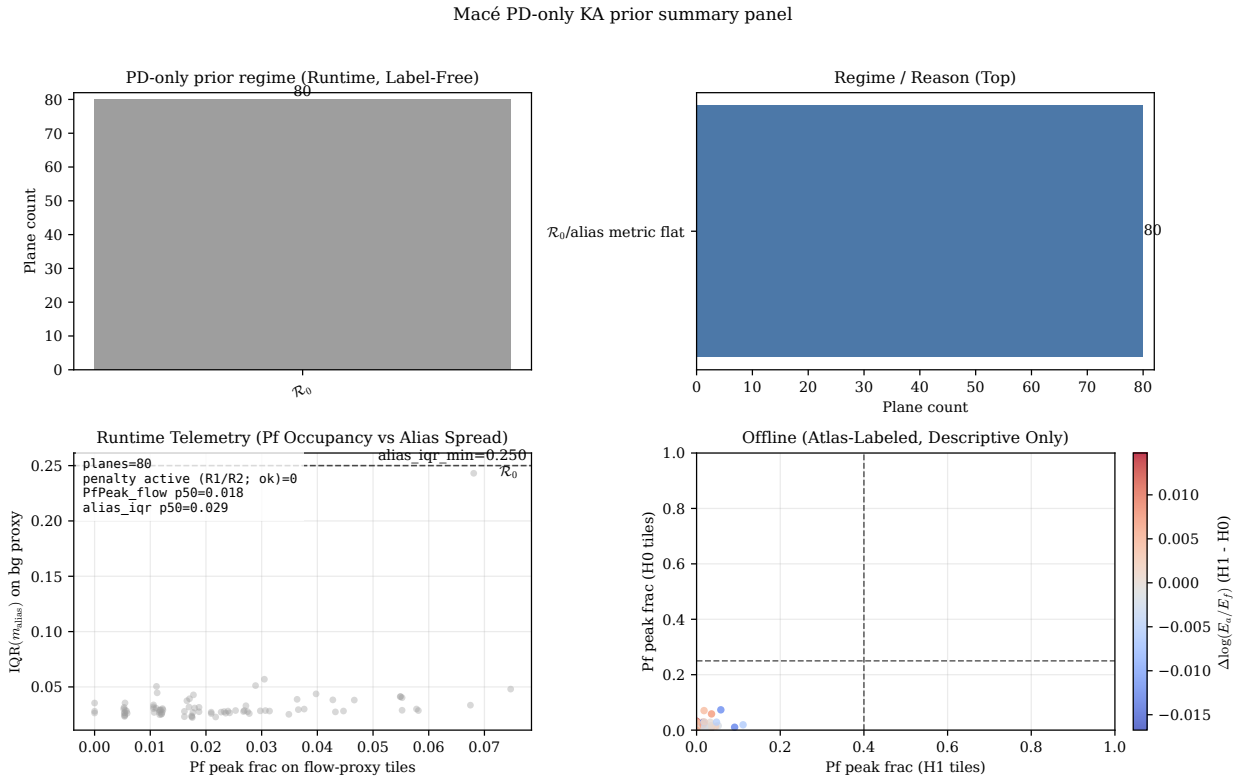


Figure 12: Macé PD-only prior summary panel over all deduplicated scan/plane instances. Top: prior regions and the most common (region, reason) pairs (runtime, label-free). Bottom-left: runtime label-free telemetry summarizing either shrink coverage/strength when penalization is active or, when the contract exits earlier, the upstream Pf-occupancy / alias-spread telemetry that drove the regime assignment. Bottom-right: offline atlas-stratified Pf-peak fractions (VIS/SC/LGd vs CP/CA/DG tiles) colored by $\Delta\log(E_a/E_f)$ (descriptive only; not a KA input). Reproducibility details are provided in Appendix L.

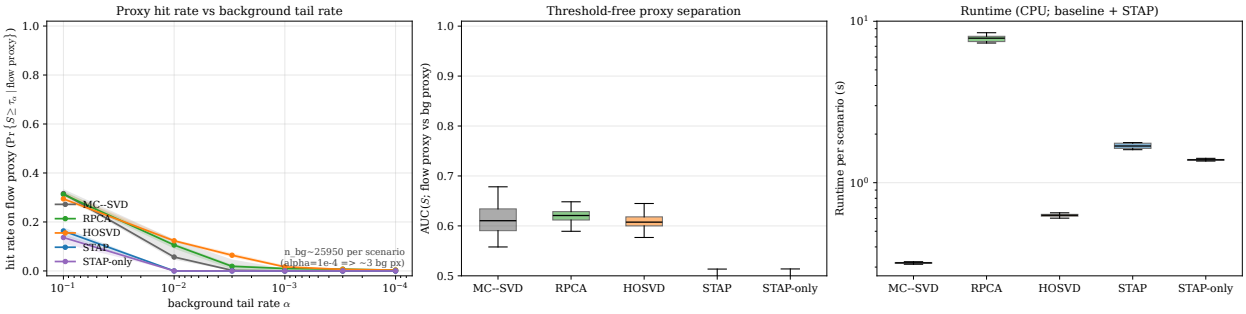


Figure 13: Shin RatBrain Fig3 baseline comparisons under label-free proxy masks (10 clips; three windows; Shin-U profile). Left: flow-proxy hit rate vs background tail rate $\alpha$ (median and IQR over scenarios). Middle: threshold-free AUC between flow-proxy and background-proxy score distributions. Right: runtime distribution per scenario in this CPU-only environment (baseline time for MC–SVD/RPCA/HOSVD; baseline+STAP total time for STAP variants), using compute-time telemetry captured in bundle metadata (excludes disk I/O and figure rendering). Full per-scenario outputs are in `reports/shin_ratbrain_baseline_matrix_shinU_e970_Lt64_nomaskunion_k80.csv`.

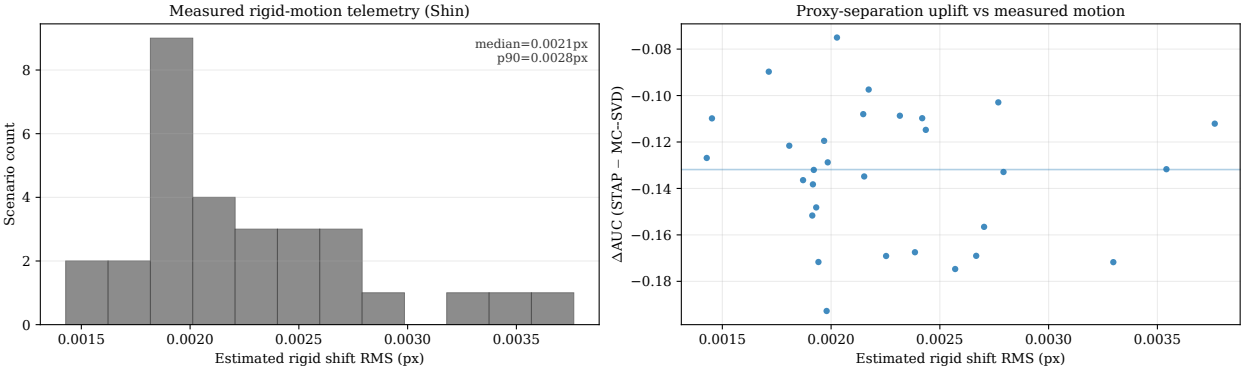


Figure 14: Shin RatBrain Fig3 measured rigid-motion telemetry across the 30 baseline-matrix scenarios (10 clips $\times$ 3 windows). Left: histogram of estimated rigid shift RMS (pixels) from phase-correlation registration used in baseline preprocessing. Right: proxy-separation uplift $\Delta$AUC (STAP minus MC–SVD) versus the same shift statistic. Full extracted telemetry is in `reports/shin_motion_telemetry.csv`.

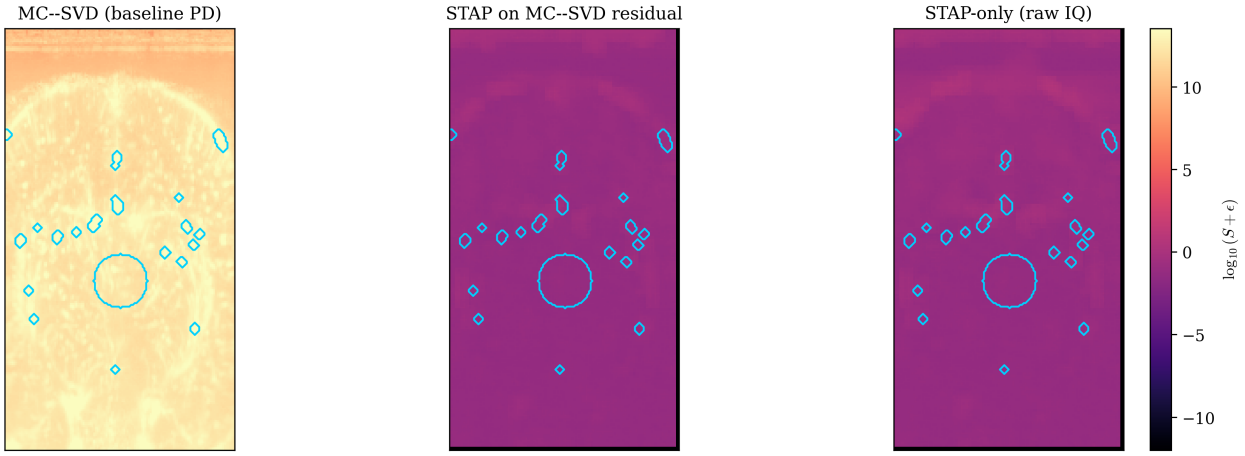


Figure 15: Representative Shin RatBrain Fig3 qualitative comparison (clip `IQData001`, window 0:128). Panels show log-scaled right-tail score maps for MC–SVD baseline PD, STAP on the MC–SVD residual, and STAP-only on raw/registered IQ, with the shared flow-proxy mask overlaid (cyan contour).

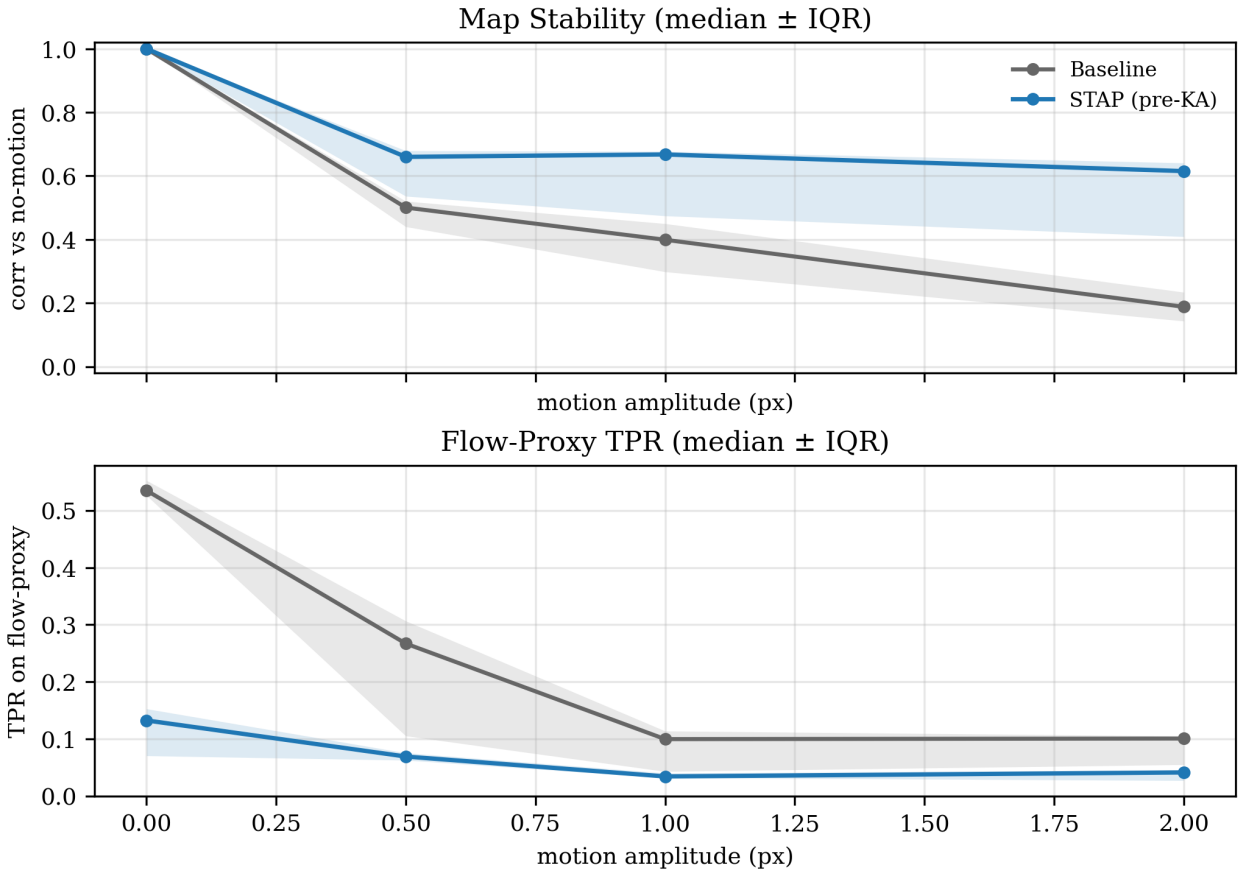


Figure 16: Shin RatBrain Fig3: brain-like residual-motion sweep (canonical window 0:128; 10 clips; median ± IQR) under the frozen Shin-U profile with rigid registration enabled and KA disabled (STAP reported pre-KA). Top: PD-mode score-map stability measured as Pearson correlation between each method's PD-mode score map at a given motion amplitude and its own no-motion reference, after phase-correlation alignment and a boundary crop. Bottom: flow-proxy hit rate (median ± IQR) at matched background-proxy tail rate = $10^{-3}$, computed under the right-tail score rule $\mathbb{P}\{S_{\mathrm{PD}} \geq \tau \mid \text{bg proxy}\} = 10^{-3}$ with a fixed flow-proxy mask derived from the no-motion baseline $pd_{\mathrm{base}}$.

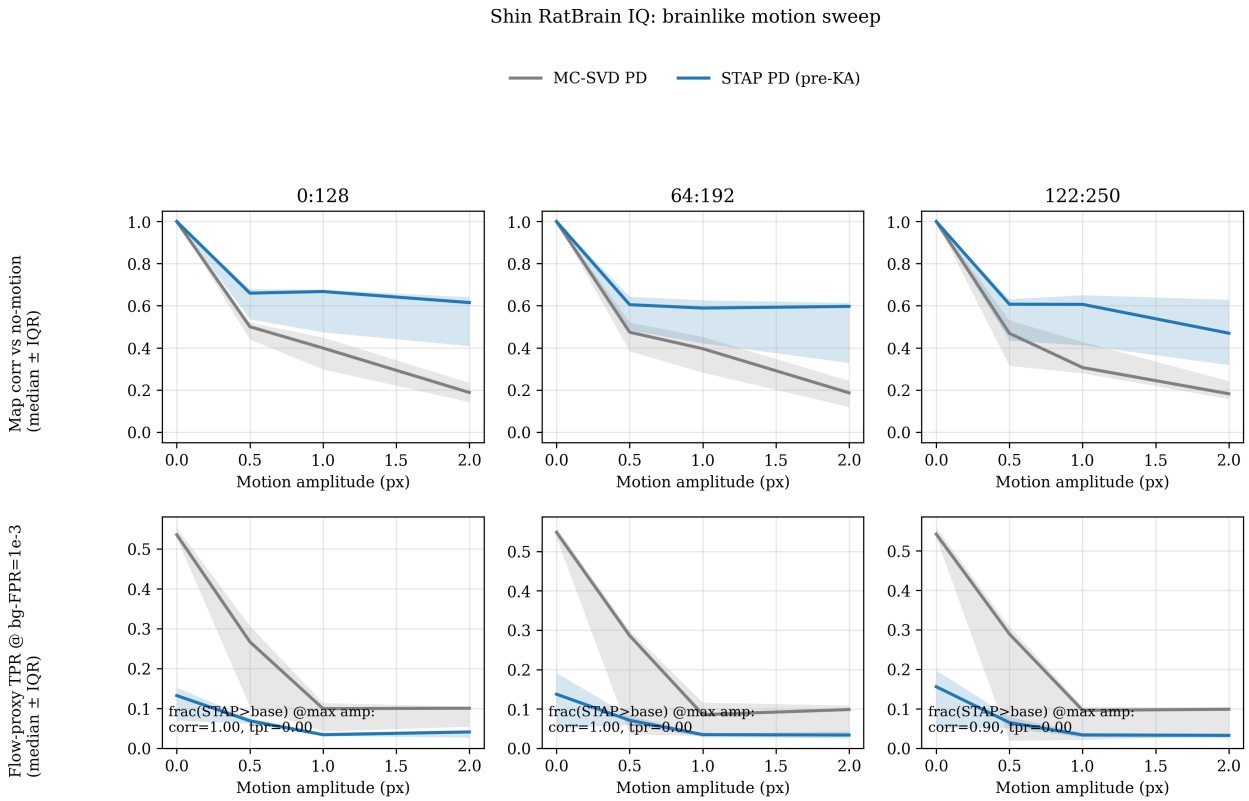


Figure 17: Shin RatBrain Fig3: brain-like residual-motion sweep across three overlapping windows (10 clips; median ± IQR) under the frozen Shin-U profile with rigid registration enabled and KA disabled (STAP reported pre-KA). Motion amplitude (x-axis) is the target displacement scale in pixels. "Brain-like" motion is applied directly to the beamformed IQ cube and combines (i) a rigid random-walk translation (70% of the amplitude), (ii) a smooth, depth-decaying elastic warp (30%; spatial smoothing $\sigma \approx 24$ px; depth weight $\propto \exp(-z/0.8)$; random-walk temporal coefficients), and (iii) per-frame micro-jitter (std $\approx 0.08\times$ amplitude). Endpoints match Figure 16. This stress test is image-domain (IQ warp) rather than a scatterer-level simulator; it is intended to mimic the "residual rigid + elastic motion after registration" mismatch in Brain-SkullOR.

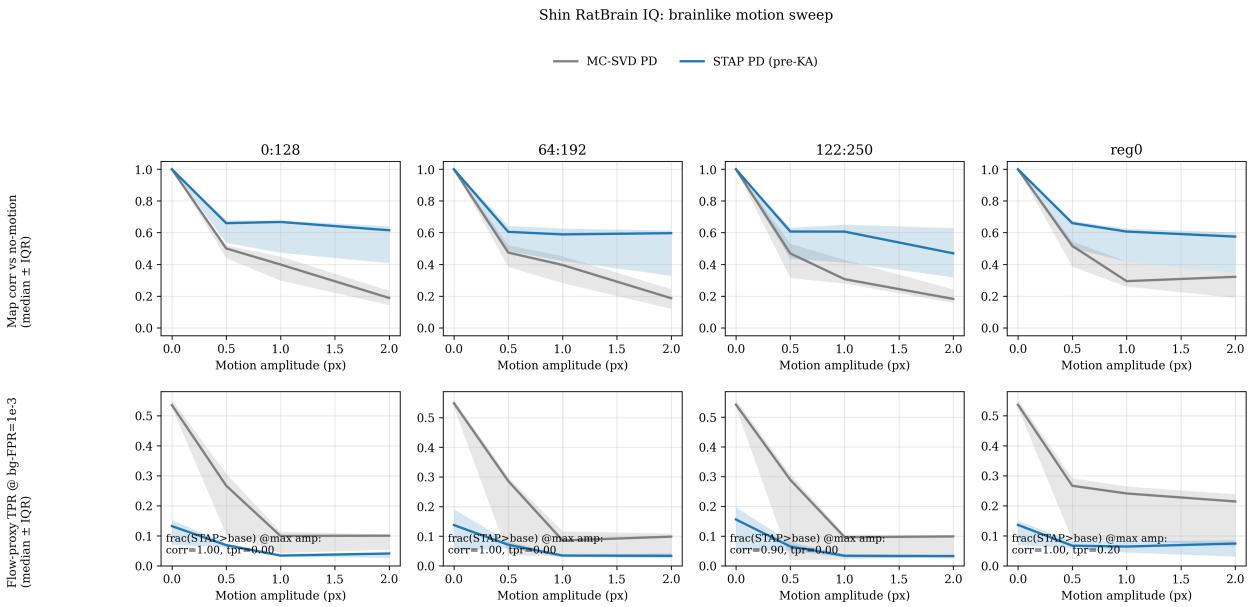


Figure 18: Shin RatBrain Fig3: registration ablation for the brain-like residual-motion sweep. The rightmost column (`reg0`) repeats the canonical window (0:128) with rigid registration disabled in the baseline and STAP pipelines, illustrating that the motion stress test contains a substantial bulk rigid component that phase-correlation registration can correct. Even with registration enabled, a smooth depth-decaying elastic warp remains (by design) and drives the robustness gap between MC–SVD and STAP.

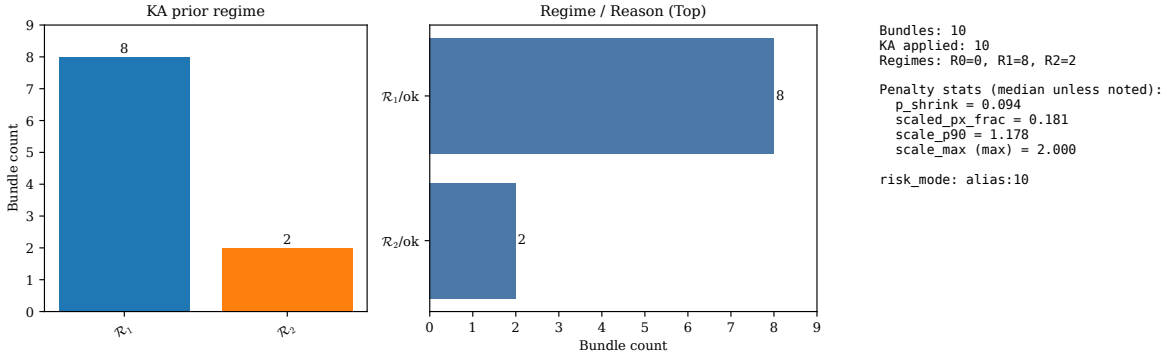


Figure 19: Shin RatBrain Fig3 (10 clips; window 0:128) KA prior summary panel under the frozen Shin-U profile (MC–SVD energy fraction $\approx 0.97$, $L_t = 64$). The panel summarizes prior regions and reasons, Pf peak fractions on flow/non-background proxies, and KA shrink coverage/strength.

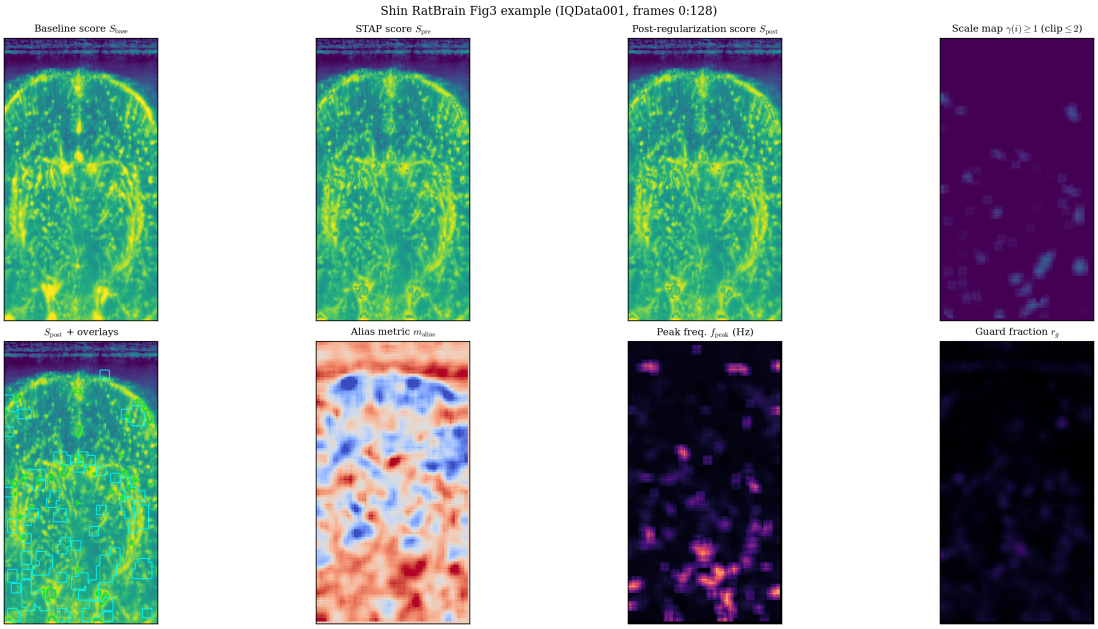


Figure 20: Representative Shin RatBrain Fig3 example (clip `IQData001`; window 0:128) illustrating the end-to-end real-IQ pipeline: baseline PD-derived map, STAP PD-mode score map (pre-KA), KA gate/scale overlay, and final PD-mode score map (post-KA), together with band-energy maps and the flow-proxy mask that defines protected (do-not-penalize) tiles. The KA layer is shrink-only and leaves protected tiles unchanged. Reproducibility details are provided in Appendix L.

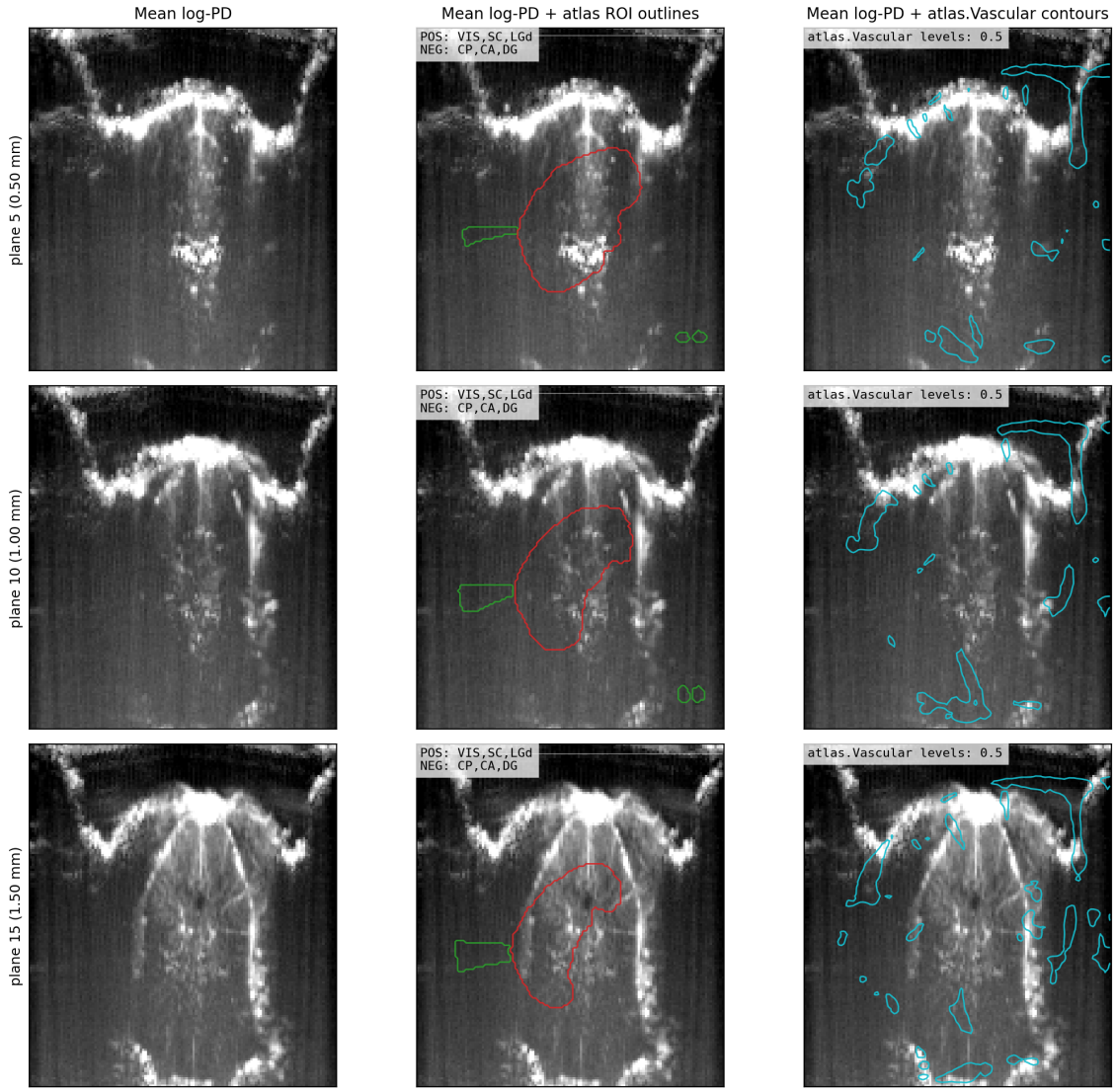


Figure 21: Macé atlas-alignment sanity figure (representative planes from the anatomical reference volume `scan_anatomy.mat`). Left: anatomical scan-space image. Middle: the same image with outlines of selected atlas ROIs mapped into the scan plane using `Transformation.mat` (VIS/SC/LGd in green; CP/CA/DG in red). Right: the anatomical image with contours of `atlas.Vascular` (cyan; level 0.5) mapped into the same plane with the same affine. This figure is generated by `scripts/mace_atlas_overlay_fig.py` and written to `figs/paper/mace_atlas_overlay.png`.